VALENCIA COLLEGE

Department of Electrical & Computer Engineering Technology (ECET)
School of Engineering, Technology, and Advanced Manufacturing (ETAM)

EET 4950

Senior Design Project

Safe Dip

A Safety-Oriented Hot-Dip Pick-and-Place System

Submitted by

Josué Alejandro Bouchard

Adriano Duque Mena

Supervised by

Mayra Socarras

November 14, 2026

Abstract

Manual hot-dip coating operations expose workers to elevated temperatures, repetitive strain, and the risk of burns from hot, viscous materials. This project presents Safe Dip, a two-axis Cartesian gantry designed to automate the pick, dip, and place cycle while reducing direct operator exposure. The system provides controlled workpiece positioning, immersion depth, dwell time, and transfer between fixed input, dipping, and output stations.

An STM32 microcontroller coordinates the complete sequence through a cooperative, nonblocking control architecture. Horizontal motion is produced by a stepper-driven timing belt and linear rails, while a stepper-driven leadscrew controls the vertical axis. A conventional electromagnet handles ferromagnetic workpieces, a part-ready trigger input lets the external feeding system start each cycle, and normally-closed limit switches provide homing and travel references. Serial and Modbus TCP communications report operating states, sensor information, and fault conditions.

Safety is treated as a primary design consideration. A hardwired Category 0 emergency-stop circuit removes power from the motor drivers independently of the microcontroller, while the electromagnet remains on a separate supply to avoid intentionally releasing a suspended workpiece during a motion stop. The laboratory prototype was evaluated without a heated coating bath, and its tests cover the integration of the mechanical, electrical, sensing, control, communication, and safety subsystems. Supporting calculations evaluate the design's scalability toward the intended 10 kg payload.

Acknowledgements

We would like to express our sincere gratitude to Professor Mayra Socarras for her supervision, guidance, and constructive feedback throughout the development of Safe Dip. Her questions and recommendations encouraged us to examine the project more critically, clarify its scope, and improve the technical, safety, and documentation aspects of the design.

We also thank Dean Lara Sharp and the members of the project review board for dedicating their time to evaluating our work and providing feedback. Their observations have helped us identify areas requiring greater clarity and have contributed to the continued refinement of the project.

We are grateful to Valencia College and the Department of Electrical and Computer Engineering Technology for providing the instruction, facilities, laboratory resources, and technical foundation that made this project possible. We also appreciate the faculty, staff, classmates, and laboratory personnel who shared their knowledge and assistance during the development of this proposal.

Finally, we would like to acknowledge our families, friends, coworkers, and employers for their patience, encouragement, and understanding while we balanced the academic and professional responsibilities associated with this project. Their continued support has been invaluable throughout this process.

Table of Contents

Abstract ..... **i**
Acknowledgements ..... **ii**

Table of Contents ..... **iii**
List of Figures ..... **vii**
List of Tables ..... **ix**

Chapter 1 Introduction **1**
1.1 Background and Problem Statement 2
1.2 Need and Motivation 2
1.3 Similar Products and Comparison 3
1.3.1 Aramfix EFFCO DS-250 Dip-Spin Machine 4
1.3.2 MS800 Tiltable Dip-Spin Machine 5
1.3.3 Monorail Headstand Screw Coating Machine 5
1.3.4 Thread Locker Coating Machine — TH-2004L3 6
1.3.5 Full Automatic Thread Coater 6
1.3.6 Emmeti WAX-ON Machine 7
1.3.7 South Shore Controls Robotic Dip Cell 7
1.3.8 Other Alternatives 8
1.3.8.1 Mechanical Architecture 8
1.3.8.2 Effector Alternatives 9
1.3.8.3 E-Stop Alternatives 9
1.4 System Requirements and Success Criteria 10
1.5 Operating Limitations 14
1.6 System Block Diagram 15
1.7 Engineering Specifications 18
1.7.1 Power Distribution 18
1.7.2 E-Stop 19
1.7.3 Processing 19
1.7.4 Sensing 20
1.7.5 Movement 20
1.7.6 End Effector 21
1.7.7 Communications 22

Chapter 2 Background Research 23
2.1 Concepts, Algorithms, and Equations 24
2.1.1 Axis Motion and Resolution 24
2.1.2 Leadscrew Payload 25
2.1.3 Step Pulse Generation 26
2.1.4 Motion Profile 26
2.1.5 Electrical Loads 26
2.2 Hardware Components and Mechanical Design (CAD) 26
2.2.1 Mechanical structure 27
2.2.2 E-Stop and Protection module 29
2.2.3 Sensing Module 30
2.2.4 Power Distribution Module 31
2.2.5 Processing Module 33
2.2.6 Movement Module 34
2.2.7 End Effector Module 34
2.2.8 Communications Module 35
2.3 Software and Programming Languages 36
2.4 Power Budget 37

Chapter 3 Project Contribution 40
3.1 Design Integration and Flow Charts 41
3.1.1 Normal Cycle Sequence 43
3.1.2 Operation Interface 44
3.1.3 Cycle Sequencer 45
3.1.4 Move and Homing Operations 46
3.1.5 Motion Controller 49
3.1.6 Step Pulse Generation 51
3.1.7 Fault Handling and Emergency Stop 52
3.1.8 Communications 52
3.2 Implementation 53
3.2.1 X Axis and Frame 53
3.2.2 Y Axis 55
3.2.3 End Effector 55
3.2.4 Power Distribution and Enclosure 56
3.2.5 Emergency Stop and Relays 57
3.2.6 Sensing 57

3.2.7 Motion Control Firmware 58
3.2.8 Cycle and Fault Firmware 59
3.2.9 Communications and Monitoring 60
3.2.10 Firmware Size Over Time 62
3.3 Troubleshooting 63
3.3.1 Limit Switch Triggering a Constant Fault 63
3.3.2 Noisy Limit Switches 63
3.3.3 Broken Ball-Bearing Housing 64
3.3.4 Belt Tension and Skipped Steps 64
3.3.5 Electromagnet Overheating 64
3.3.6 Failed Motor-Supply Relay 65
3.3.7 Modbus Connections Dropping 65
3.3.8 Modbus Stalling the Main Loop 65
3.3.9 Defects Found in Code Review 66
3.3.10 No Step Pulses From the Timer 66
3.3.11 Stepper Overshoot 66
3.3.12 Inverted Button Inputs 66
3.3.13 3D Printing and Part Fit 67
3.3.14 Damaged Leadscrew Holder 67
3.3.15 Frame Wobble 68
3.4 Requirement Testing and Results 68
3.4.1 R-01: Part-Ready Trigger 69
3.4.2 R-02: Pickup Position Accuracy 69
3.4.3 R-03: Workpiece Retention 70
3.4.4 R-04: Immersion Depth 70
3.4.5 R-05: Dwell Time 70
3.4.6 R-06: Full Unattended Cycle 71
3.4.7 R-07: Emergency-Stop Power Removal 71
3.4.8 R-08: Restart Inhibit 71
3.4.9 R-09: State and Fault Reporting 72
3.4.10 R-10: Ten Consecutive Cycles 72
3.4.11 R-11: Rated Payload 73
3.5 Verification Summary 73
3.6 Final Design 75

Chapter 4 Non-Technical Considerations 77

4.1 Project Timeline 78
4.2 Project Expenses 81
4.2.1 Proposed Budget 81
4.2.2 Total Expenses to Date 82
4.2.3 Cost of the Final Design 84
4.3 Distribution of Tasks 85
4.4 Health and Safety 88
4.5 Ethical Aspects 89
4.6 Environmental Aspects 90
4.7 Social Aspects 90
4.8 Sustainability 91

Chapter 5 Conclusion 92
5.1 Summary and Conclusion 93
5.2 Suggestions for Future Work 93

References 95
Appendix A - Programming Code 97
Appendix B - Engineering Codes and Standards 111
Appendix C - Datasheets 113
Appendix D - Electrical Schematic 115
Appendix E - Team Biographies 116

List of Figures

Fig. 1.1 Universal Robots UR5e Collaborative Robot Arm 8
Fig. 1.2 System-wide block diagram 15
Fig. 1.3 Hardware architecture 17
Fig. 2.1 CAD model of the Safe Dip mechanical arrangement - Perspective view 28
Fig. 2.2 CAD model of the Safe Dip mechanical arrangement - Front view 28
Fig. 2.3 E-stop mushroom button 30
Fig. 2.4 HiLetgo KW12-3 micro limit switch with roller lever 31
Fig. 2.5 Mean Well LRS-350-24 AC-DC power supply 32
Fig. 2.6 ALITOVE DC 12V 5A Power Supply 32
Fig. 2.7 HiLetgo buck converter 33
Fig. 2.8 STM32 Nucleo-F446RE microcontroller board 33
Fig. 2.9 NEMA 23 23HS22-2804S stepper motor 34
Fig. 2.10 DM542TE stepper controller 34
Fig. 2.11 Baomain BM-P 80-38 electromagnet 35
Fig. 2.12 W5500 Ethernet module 36
Fig. 3.1 The firmware's main loop 41
Fig. 3.2 Normal pick-dip-place operation sequence 43
Fig. 3.3 One step of the cooperative cycle sequencer 45
Fig. 3.4 A move operation 48
Fig. 3.5 Homing one axis 49
Fig. 3.6 One speed update of the motion controller 51
Fig. 3.7 Full-scale CAD model of the frame with the linear rails and bearing blocks . 53
Fig. 3.8 X-axis parts in CAD 54
Video 3.1 First full test of both axes with the monitor interface 55
Video 3.2 Fitting the ball bearing, adapter, and gear 55
Fig. 3.9 Vertical assembly plate in CAD 55
Fig. 3.10 Y-axis magnet holder in CAD 56
Fig. 3.11 Electronics in the container during installation 57
Fig. 3.12 The limit switches as mounted, in a frame from the video of the prototype running. The X switch is at the end of the upper rail, and the Y switch is on the carriage plate 58

Fig. 3.13 Bench setup for the open-loop test: 24 V supply, DM542TE driver, NEMA 23 motor with a paper flag to show its position, and the Nucleo board with the W5500 module 59
Video 3.3 Open-loop positioning test of one stepper motor 59
Video 3.4 Bench simulation of both axes and the effector signal 59
Fig. 3.14 The Modbus monitor while the cycle ran 61
Fig. 3.15 The Modbus monitor after an emergency stop: the emergency-stop fault is latched and marked as the first fault 62
Fig. 3.16 Lines of firmware source over time, counted at every commit 62
Fig. 3.17 3D printing of the vertical assembly part 67
Fig. 3.18 Lower section of the holder reinforced with two L-brackets 68
Fig. 3.19 Metal L-support used to brace the frame 68
Video 3.5 Checkpoint 2 viability demonstration of the integrated prototype 69
Fig. 3.20 Safe Dip gantry: aluminum frame, two horizontal rails, and the Y-axis assembly with its stepper motor 75
Fig. 3.21 Details of the assembled prototype 76
Fig. 3.22 Frame from a video of the assembled prototype running, with the limit switches fitted 76
Fig. 4.1 Sixteen-week Safe Dip implementation schedule. 79
Fig. 4.2 Modified Safe Dip implementation schedule. 80
Listing A.1 The timer interrupt that generates the STEP pulses 98
Listing A.2 Braking distance and speed ramp of the motion controller 99
Listing A.3 Conversion between millimeters and steps 100
Listing A.4 Debounce filter for the limit switches 101
Listing A.5 The fault latch and the safe state 102
Listing A.6 The interface that every operation of the cycle implements 103
Listing A.7 One step of the cycle sequencer 104
Listing A.8 One poll of the homing operation 106
Listing A.9 Sending a Modbus TCP request and checking the reply in the monitor 108
Listing A.10 Decoding the bits and registers of a Modbus reply in the monitor 109
Listing A.11 Reading the state of the machine in the monitor 110
Fig. D.1 Complete electrical schematic of the Safe Dip prototype 115

List of Tables

Table 1.1 Comparison of commercial dip coating systems against Safe Dip 3
Table 1.2 System requirements and verification criteria 11
Table 1.3 Power distribution specifications 18
Table 1.4 E-stop specifications 19
Table 1.5 Processing specifications 19
Table 1.6 Sensing specifications 20
Table 1.7 Movement specifications 20
Table 1.8 End effector specifications 21
Table 1.9 Communications specifications 22
Table 2.1 Conversion between stepper driver pulses and axis travel 25
Table 2.2 Principal dimensions of the Safe Dip prototype. 29
Table 2.3 Power budget for the Safe Dip prototype. 38
Table 2.4 Maximum power available from the Safe Dip AC-DC power adapters. 39
Table 2.5 Load on each power rail compared with the rating of its AC-DC power adapter. 39
Table 3.1 Operations available to the sequencer 45
Table 3.2 Verification summary of the system requirements 74
Table 4.1 Proposed component budget, from the Senior Design Proposal. 81
Table 4.2 Total expenses to date. 82
Table 4.3 Cost of the final design. 84
Table 4.4 Preliminary distribution of project responsibilities. 86
Table 4.5 Current distribution of project responsibilities. 87
Table 4.6 Primary prototype hazards, controls, and residual limitations. 88
Table B.1 Engineering codes and standards applicable to the Safe Dip project. 111

Chapter 1

Introduction

Summary

In this chapter, we introduce Safe Dip, a safety-oriented hot-dip pick-and-place system designed to remove operators from the hazardous environment of manual wax coating operations. We establish the motivation for automating this process, identify the risks associated with manual dipping, and review existing commercial solutions. We conclude by identifying the gap in the market that Safe Dip is designed to fill.

1.1 Background and Problem Statement 2
1.2 Need and Motivation 2
1.3 Similar Products and Comparison 3
1.4 System Requirements and Success Criteria 10
1.5 Operating Limitations 14
1.6 System Block Diagram 15
1.7 Engineering Specifications 18

1.1 Background and Problem Statement

Hot-melt dip coating, also referred to as hot dipping or hot-melt wax coating, involves submerging a part into a heated liquid medium such as molten wax, resin, or a similar coating compound. Upon removal, the liquid solidifies as it cools, forming a protective layer around the part that can be cleanly stripped away when needed. This process is commonly applied before long-term storage or bulk packaging to shield parts from corrosion, prevent contact damage that could dull sharp edges, and, in some cases, enhance a tool's grip or handling comfort [1], [2].

Safe Dip seeks to address this gap by providing an affordable, safety-conscious, and instrumented automation solution tailored to this class of hot-dip operations. It targets the small-to-medium production scale where off-the-shelf industrial robots are cost-prohibitive and general-purpose systems lack the domain-specific safety integration required by industrial electrical standards.

1.2 Need and Motivation

In facilities where this process is still performed manually, workers face sustained exposure to elevated temperatures, repetitive motion injuries, and the risk of burns or chemical contact. As soon as the weight of the objects surpasses a couple of kilograms, long-handled tongs or clippers are introduced, adding to the physical strain. Finally, most coating products are not only hot but also viscous, meaning that even the smallest splash can cause serious injuries. Together, these hazards make manual dipping one of the more accident-prone tasks on the shop floor.

Automating the dipping cycle would remove the operator from the hazard zone entirely, yet many smaller manufacturers have not adopted automated solutions due to cost barriers, system complexity, and the lack of scalable, application-specific equipment suited to their production scale.

In addition to improving operator safety, an effective automation solution should remain useful when the coating process changes. Many commercially available systems are designed around a particular chemical, workpiece geometry, or production workflow, making them difficult or expensive to repurpose. Safe Dip instead treats the dipping liquid as an external process variable: the system performs the same controlled pick, immersion, dwell, and placement sequence regardless of the specific coating, cleaning, or treatment medium being used. This chemical flexibility allows laboratories, prototype facilities, and small manufacturers to adapt to new formulations, suppliers, or applications without redesigning the automation equipment.

By combining this flexibility with a comparatively simple Cartesian architecture, Safe Dip seeks to address the gap between manual dipping and highly specialized industrial machinery. The objective is not to reproduce the throughput or complexity of a complete robotic coating cell, but to provide a lower-cost platform that improves safety, process repeatability, and adaptability at a scale accessible to smaller operations.

1.3 Similar Products and Comparison

Table 1.1
Comparison of commercial dip coating systems against Safe Dip

Product	**Price**	**Conveyor Feed**	**Variable Dip Depth**	**Pick & Place**	**Chemical Flexibility**
Safe Dip (Ours)	~$500-$600	YES	YES	YES	YES
Dip-Spin (EFFCO DS-250)	est. $15k–$25k	NO	NO	NO	YES
MS800 Tiltable Dip-Spin	est. $40k–$80k	NO	NO	NO	YES
Monorail Headstand Screw Machine	$2,999–$3,999	YES	NO	NO	Partial
Thread Locker Machine (TH-2004L3)	On Request	YES	NO	NO	Partial

Product	Price	Conveyor Feed	Variable Dip Depth	Pick & Place	Chemical Flexibility
Full Auto Thread Coater	$20k–$30k	YES	NO	NO	NO
Emmeti WAX-ON	est. $50k–$150k+	YES	YES	YES	Partial
South Shore Robotic Dip Cell	Custom Quote	NO	YES	YES	YES

Chemical flexibility is the ability of a system to operate with different coating, cleaning, and treatment liquids without requiring changes to its mechanical architecture or control logic. The system interacts with the process only through part detection, controlled handling, programmable immersion, and placement; therefore, the chemical composition of the liquid is treated as an external process variable rather than a machine-specific design input. Compatibility is limited primarily by the temperature tolerance and chemical resistance of the end effector and other exposed hardware, rather than by the operating principle of the system itself.

This separation between the machine and the dipping chemistry is important because it allows the same platform to be repurposed as process requirements, suppliers, or formulations change. Instead of designing dedicated equipment for each liquid or application, users can adapt the process by changing the dip medium and operating parameters while retaining the same automation hardware. This reduces dependence on a single chemical process, lowers the risk of future redesign, and makes Safe Dip more suitable for laboratories, prototyping environments, and small-scale manufacturing where process requirements may change frequently.

1.3.1 Aramfix EFFCO DS-250 Dip-Spin Machine

The EFFCO DS-250 is a batch-type industrial dip-spin coating machine designed for high-volume processing of small metal components such as fasteners, nuts, bolts, and stampings. The machine operates by loading parts into a perforated basket, submerging the entire basket into a coating tank, and then spinning the basket at high RPM to centrifugally remove excess

coating and achieve a uniform film thickness. This process enables extremely consistent results across thousands of parts per cycle, making it well-suited for mass production environments. The DS-250 features a heated coating tank and programmable dip and spin cycles, allowing operators to fine-tune coating thickness to specification. However, its batch-based design means individual part handling and positional control are not possible, as all parts in the basket are processed simultaneously without differentiation. The machine requires manual loading and unloading of the basket, meaning human intervention is necessary at each production cycle. Its industrial footprint and price range of $15,000 to $25,000 place it well beyond the reach of small-scale or research-oriented operations. Unlike Safe Dip, the DS-250 cannot adapt to varying part geometries within a single run or interact with a conveyor-based workflow.

1.3.2 MS800 Tiltable Dip-Spin Machine

The MS800 is a fully automatic, tiltable dip-spin coating machine designed for heavy-duty industrial production of fasteners and small precision components. Its distinguishing feature is a motorized tilting mechanism that repositions the basket during the drain cycle, improving coating drainage and reducing waste compared to standard vertical-axis spin machines. The system is PLC-controlled, offering precise regulation of dip duration, spin speed, tilt angle, and temperature, resulting in coating uniformity that is difficult to match with manual or semi-automatic methods. With a throughput capacity in the thousands of parts per hour, the MS800 is optimized for large-scale manufacturing lines rather than flexible or small-batch applications. The machine's price range of $40,000 to $80,000 reflects its industrial-grade construction and automation level. Like other dip-spin systems, it is fundamentally a batch processor and cannot handle individual components, vary dip depth per part, or interface with a pick-and-place conveyor system. Its complexity also demands trained operators and dedicated maintenance protocols. While it represents the high end of coating machine capability, it addresses a fundamentally different operational context than Safe Dip.

1.3.3 Monorail Headstand Screw Coating Machine

The monorail headstand screw coating machine is a continuous-flow system designed specifically for the automated surface treatment of headed fasteners such as screws, bolts, and rivets. Parts are loaded onto an overhead rail conveyor by their heads, hanging inverted as they travel through a dip station where their threaded sections are coated in the desired chemical or lubricant. The continuous rail feed allows throughput rates of 10,000 to 20,000 parts per hour, making it one of the most efficient solutions for large-volume screw coating applications. Because parts self-hang from their heads, no gripper or pick-and-place mechanism is

required, which simplifies the mechanical design significantly. However, this same feature makes it incompatible with any component that lacks a head or flange to hang from, limiting its applicability to a narrow category of fastener geometry. The coating type per production run is fixed, and changing the chemical requires a full tank flush and reconfiguration. Priced between $2,999 and $3,999, it is the most affordable automated option reviewed, but its single-purpose nature and geometric constraints make it unsuitable for the diverse component handling that Safe Dip is designed to support.

1.3.4 Thread Locker Coating Machine — TH-2004L3

The TH-2004L3 is a purpose-built automatic dispensing machine designed to apply thread-locking adhesive, such as Loctite or Vibra-Seal equivalents, directly onto the threads of screws and bolts prior to assembly. The machine integrates a bowl feeder for bulk part loading, a precision dispensing head that applies a metered quantity of adhesive to each thread zone, and a conveyor exit for processed parts. Its dispensing mechanism is calibrated to apply only to the thread region, avoiding contamination of the head or shank, which is critical for assembly line compatibility. The system handles fastener sizes from M3 to M36 and operates at high throughput, making it appropriate for automotive, electronics, and aerospace assembly environments. Priced between $3,300 and $4,500, it sits at the lower end of the industrial machine spectrum, though it is still an order of magnitude more expensive than Safe Dip. Its fundamental limitation is that it is chemically and geometrically constrained: it applies a single fluid type to a single class of headed fasteners, with no capability to handle other part geometries or coating materials. It also has no pick-and-place functionality and cannot interface with a generalized conveyor workflow.

1.3.5 Full Automatic Thread Coater

The full automatic thread coater is a high-capacity, end-to-end automated machine designed for the uninterrupted production-line application of thread-locking compounds or sealants on screws and bolts. It integrates automated part feeding, precision coating application, and a discharge conveyor into a single self-contained unit, eliminating the need for manual intervention during normal operation. The system is optimized for single-fluid, single-geometry workflows and is typically configured for a specific fastener size range at installation. Throughput is very high, and the machine is engineered for continuous multi-shift operation in demanding industrial environments. Priced between $20,000 and $30,000, it represents a significant capital investment best justified in high-volume production contexts. Its automation level, while impressive within its domain, comes at the cost of flexibility: changing the coating fluid or accommodating a different part geometry requires machine reconfiguration

and downtime. It lacks any conveyor-integrated pick-and-place capability and cannot adjust dip depth or dwell time per individual part. These constraints make it an effective solution for its intended use case but a poor match for the adaptive, multi-geometry coating scenario that Safe Dip is designed to address.

1.3.6 Emmeti WAX-ON Machine

The Emmeti WAX-ON is a fully robotic, end-to-end bottle waxing line developed for high-volume industrial winery and spirits production. The system integrates a depalletizer, an infeed conveyor, a robotic dipping arm, a multi-tank wax station with interchangeable color tanks on rails, and a vision-based defect inspection camera, all feeding into an automated case packer. Its robotic arm performs true pick-and-place operations, selecting individual bottles from the conveyor and submerging them into the wax tank with consistent depth and dwell time control. The 3D part identification system allows the machine to adapt to minor variations in bottle positioning without human correction. Coating uniformity is achieved through a combination of precise dip speed, controlled withdrawal rate, and optional rotation of the bottle during extraction. The system is capable of processing hundreds of bottles per hour with minimal operator involvement, making it one of the most capable waxing machines commercially available. However, its price range of $50,000 to $150,000 and its large industrial footprint place it entirely outside the scope of small-scale or research applications. It is also purpose-designed for bottles and is not adaptable to other component geometries without significant engineering modification.

1.3.7 South Shore Controls Robotic Dip Cell

The South Shore Controls robotic dip cell is a custom-engineered industrial automation solution designed for precision dipping, coating, and chemical treatment of components across a range of industries, including food processing, medical device manufacturing, and industrial finishing. The system is built around a 6-degree-of-freedom industrial robot arm that provides full spatial control over the component throughout the dip cycle, enabling complex dip trajectories, variable dip angles, and multi-stage coating sequences that rigid gantry systems cannot replicate. Custom end-effectors, or grippers, are designed to match the specific geometry of the target component, ensuring reliable handling even for irregular or delicate parts. The system is PLC-controlled and fully programmable, with PID-regulated temperature control on the coating tank and real-time process monitoring. Pricing is entirely quote-based and typically falls in the range of $50,000 to $100,000 depending on configuration, making it accessible only to well-capitalized industrial clients. Unlike Safe Dip, it does not incorporate a conveyor-based intake and exit workflow; parts must be presented to the robot

by separate handling systems. Its strength lies in precision and flexibility within a controlled cell environment, rather than inline conveyor integration.

1.3.8 Other Alternatives

Before arriving at the final design, several alternative approaches were evaluated across three key areas of the system: the overall mechanical architecture, the end effector mechanism, and the fail-safe strategy for power loss events. Each alternative was assessed against the project's core priorities of operator safety, cost, mechanical simplicity, and suitability for the hot-dip environment. The following subsections document the options considered and the reasoning behind the decisions made.

1.3.8.1 Mechanical Architecture

From the very beginning of the project, the idea of a gantry competed against the idea of a robotic arm, as the one in Fig. 1.1. While it presented enough dexterity for the task at hand, a couple of issues remained, regardless of the degrees of freedom it presented:

- The bigger the number of degrees of freedom it had, the bigger the electronic and computer complexity the system would carry, as each degree of freedom would require at least an additional actuator.
- Mechanically, the fact that the arm is only anchored at one point at the bottom, offset from the object's position, meant that the motors would have to generate and support the entirety of the weight of the object.
- Given the height of some hot-dipping pots, the arm would either require unusually long sections, which would toll even more the motors to sustain the torque, all this to be able to get deep enough for the pot, and high enough to avoid it while grabbing the object.

In contrast, the mechanics of a cartesian gantry are pretty straightforward: only a motor per axis is required, and the positioning of them makes it such that the structure bears most of the weight in a predictable way.

Finally, a last alternative was considered where the objects being dipped could just move through a conveyor belt and have the liquid poured from the top, but that was not acceptable, since materials like hot wax would harden and break the belt, and a uniform coating would be hard to achieve.

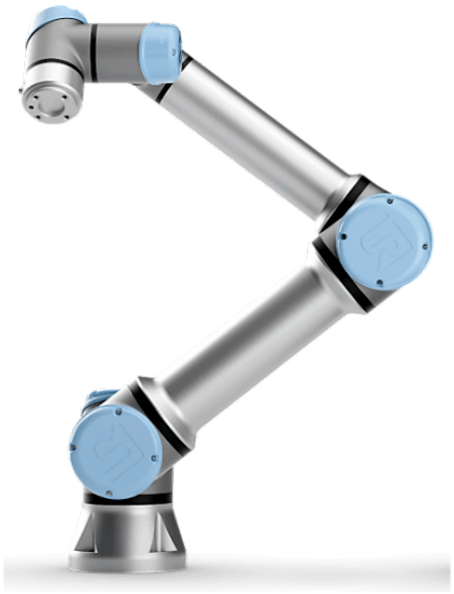


Fig. 1.1: Universal Robots UR5e Collaborative Robot Arm

1.3.8.2 Effector Alternatives

Another aspect worth consideration is the end effector (i.e., the mechanism that grabs the objects). While a claw is usually the first option everyone thinks of, it also comes with its own disadvantages:

- Claws must have a strong enough grip to ensure the object doesn't slip, while being soft enough as to not deform it or break it.
- The grip strength is dependent on the geometry of the object and the positioning of the claw.
- As the weight of the objects to be picked increases, it's been recommended to use a pneumatic claw, which introduces a new design dimension, complications, and considerations.

While an electro-permanent magnet would have been a more suitable effector, more aligned with the safety-conscious objective of this project, its availability is limited, and its price is cost-prohibitive for a prototype.

1.3.8.3 E-Stop Alternatives

Finally, for a project created out of the need for a safe alternative to manual operation, a lot of thought has been put into how the system should behave in the case an emergency stop was required. International standard IEC 60204-1 specifies three categories of emergency stops [3]:

- Category 0: uncontrolled stop by immediate removal of actuator power.
- Category 1: controlled stop, with power retained long enough to stop, followed by power removal.
- Category 2: controlled stop with actuator power remaining available.

Category 1 requires more complexity than category 0, since it requires hardware to disconnect power after a specific time. While stepper motors could benefit from a soft deceleration, given the speed they reach, the complexity outweighed the benefits for this project. Furthermore, the microcontroller cannot be used to trigger a power-off sequence; it is always assumed that it can fail too.

Category 2 simply didn't fit the requirements of this project.

1.4 System Requirements and Success Criteria

Each requirement below describes a specific, observable behavior with a defined pass condition. The system is treated as a black box for these requirements, and emphasis is placed on interactions with system boundaries, the required level of performance, and physical requirements.

Table 1.2
System requirements and verification criteria

ID	Requirement	Success criterion	Test method
R-01	The system shall not initiate a pick-dip-place cycle unless its part-ready trigger input is asserted.	A cycle starts with the trigger asserted; no motion starts with it released or disconnected.	Command a cycle with the trigger asserted by a push button. Release it, then disconnect it, and command a cycle each time.
R-02	The system shall position the end effector at the programmed pickup position within ± 1 cm along both axes before acquiring the workpiece.	Measured error within ± 1 cm along both axes.	Command the system to the programmed pickup position and measure the position of the end effector relative to the target.
R-03	The system shall retain the workpiece throughout the complete pick-dip-place cycle without slipping or dropping it.	Workpiece does not visibly slip or detach at any point.	Execute one complete pick-dip-place cycle using the approved prototype payload.
R-04	The system shall lower the workpiece to the programmed immersion depth within ± 1 cm.	Measured depth within ± 1 cm of programmed depth.	Execute one dip sequence and measure the actual immersion depth relative to the programmed value.

ID	Requirement	Success criterion	Test method
R-05	The system shall hold the workpiece at the programmed immersion depth for the programmed dwell time within ± 1 s.	Measured dwell within ± 1 s of programmed value.	Execute one dip sequence and measure the dwell period with a stopwatch or equivalent timing instrument.
R-06	The system shall complete one full unattended pick-dip-place cycle, beginning with the part-ready trigger and ending with workpiece release at the output station and return to the home position.	Every stage completes in order without operator intervention.	Place a workpiece at the input station and assert the part-ready trigger. Confirm that the system acquires the workpiece, transfers it to the dipping position, lowers it to the programmed immersion depth, holds it for the programmed dwell time, raises it, transfers it to the output station, releases it, and returns to the home position.
R-07	The system shall remove power from both stepper-motor drivers when the emergency stop is actuated.	Driver power removed and all axis motion stops.	Actuate the emergency stop while an axis is moving and measure the power supplied to both stepper-motor drivers.

ID	Requirement	Success criterion	Test method
R-08	The system shall remain inhibited following an emergency-stop event until the emergency-stop button has been reset and the operator performs a deliberate restart action.	No motion occurs until the deliberate restart action is performed.	Actuate the emergency stop during operation, reset the emergency-stop button, and attempt to command motion without performing the restart action. Perform the deliberate restart action and confirm that normal operation can be re-established.
R-09	The system shall transmit its current operating state and any active faults through the selected communications interface during operation.	Each state transition and the active fault are transmitted with correct labels.	Monitor the communications interface during one complete operating cycle and introduce at least one detectable fault condition.
R-10	The system shall complete ten consecutive unattended full pick-dip-place cycles using the approved prototype payload without dropping a workpiece, missing the intended placement position, or requiring operator intervention between cycles.	All ten complete; no drops, no missed placements; depth and dwell within tolerance.	Command the system to perform ten consecutive full pick-dip-place cycles using the approved prototype payload. Record the cycle time, commanded immersion depth, measured immersion depth, and measured dwell time for each cycle.

ID	Requirement	Success criterion	Test method
R-11	The system shall complete the full pick-dip-place cycle while carrying a workpiece with a mass of 1 lb.	Lifts, transports, lowers, retains, and releases without stalling, losing position, slipping, or dropping.	Attach a 1 lb workpiece and command one complete pick-dip-place cycle.

1.5 Operating Limitations

The following constraints define the boundaries within which Safe Dip is designed to operate, which are outside the control of the system. They are not subject to pass/fail testing, but scope the design and inform component selection.

- Safe Dip operates along two axes only: a horizontal axis for transfer between the input station, dipping bath, and output station, and a vertical axis for picking, dipping, and placing the workpiece. The system does not support rotation or movement perpendicular to the horizontal travel path.
- The system is designed to handle cylindrical workpieces within the specified size and payload limits.
- Workpieces must be presented at a known, fixed input position. The system does not perform lateral alignment, orientation correction, or vision-based localization.
- Each workpiece must provide a clean, flat, ferromagnetic attachment surface that remains outside the required coating area. Because the electromagnet remains in contact with this surface throughout the cycle, the attachment area will not be dipped or coated. Workpieces requiring complete surface coverage would require a different end effector or a secondary coating operation.

For the laboratory prototype, the following additional operating limitations apply:

- The laboratory prototype is designed and validated for workpieces with a maximum mass of 1 lb. Operation above this payload is outside the prototype's validated operating range.
- The prototype shall not be operated with a heated dipping bath or exposed to temperatures beyond the rated limits of its selected components. All acceptance testing was performed using an unheated, laboratory-safe medium under room-temperature conditions.

- The prototype does not validate operation near molten wax, heated resin, process fumes, splashing hot material, or other industrial hot-dip conditions.
- Because of budget limitations and component availability, the prototype uses a conventional electromagnet rather than one that holds the workpiece without power. A loss of the effector power supply or of mains power will therefore release the workpiece, although axis motion stops.

This limitation keeps the prototype safe and practical to demonstrate in the laboratory. Using molten wax or another heated material would introduce burn, splash, spill, and fume hazards that would require additional protective equipment and facilities. Heat could also affect the frame, wiring, sensors, motors, lubrication, and end effector.

Although these issues would need to be addressed in an industrial version, detailed thermal and mechanical design is outside the main scope of the ECET prototype. The project therefore focuses on demonstrating the electrical, control, sensing, communication, and safety systems at room temperature.

1.6 System Block Diagram

The Safe Dip system can be divided into 7 modules as shown in Fig. 1.2.

The power distribution module is composed of the power supply and is in charge of converting wall power into the different types of voltage needed.

The E-stop module will contain all the mechanisms needed to perform a category-0 emergency stop, able to cut the power to the movement module. It is also in charge of alerting the processing module about the E-stop so that it can handle it gracefully.

The processing module is the heart of the system, encapsulating the microcontroller, which receives all inputs and translates them into the corresponding actuator responses. It also includes any and all components to ensure that the incoming data is at an acceptable input voltage for the microcontroller. This module receives its power from the power distribution module, and through the microcontroller's on-board power converter, it feeds 3.3 V to the other modules.

The sensing module is composed of the limit switches used to home each axis. Because the conveyor is outside the project scope, the system that supplies the workpieces also signals when one is ready, through a part-ready trigger input on the controller. For the demonstration, this input is driven by a push button.

The movement module is composed of all the stepper motors and controllers required for the horizontal movement of the gantry and the movement of the vertical assembly.

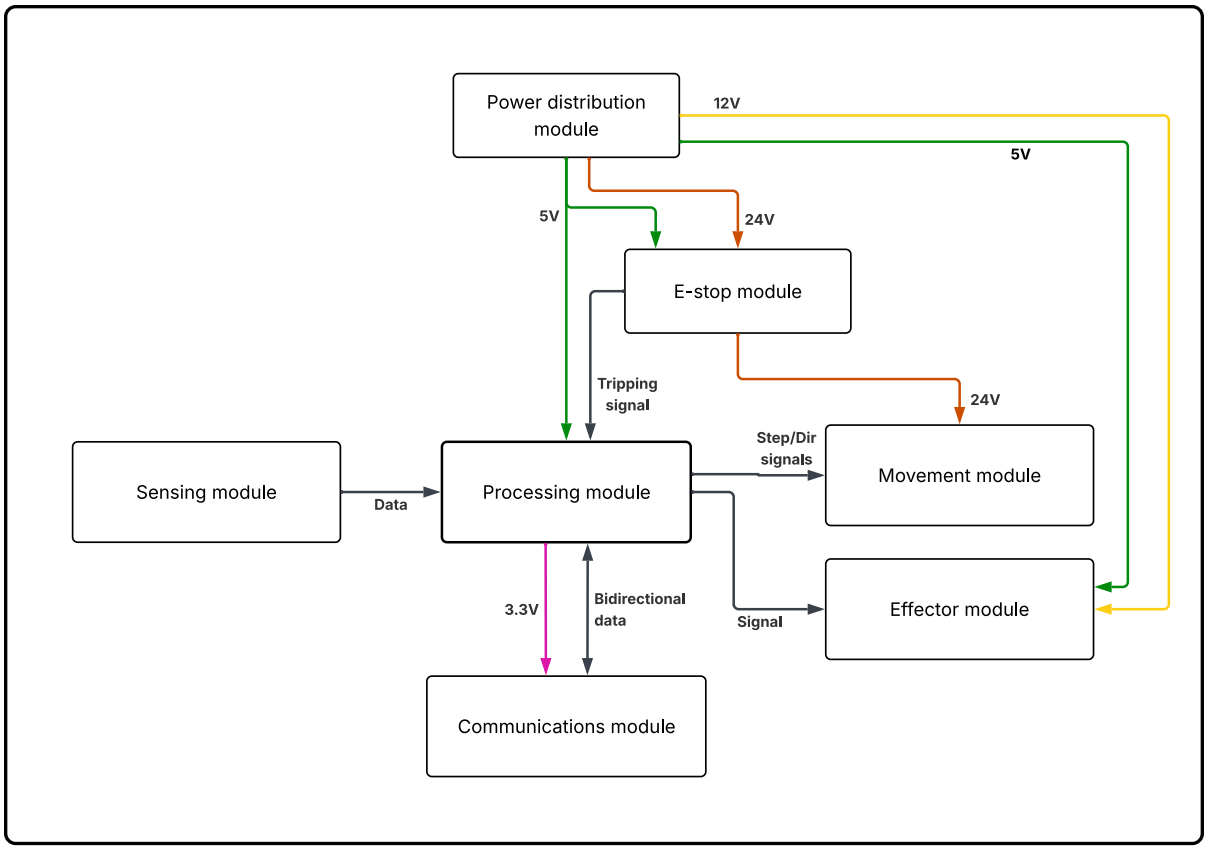


Fig. 1.2: System-wide block diagram

The effector module controls the part of the system that grabs the object and is composed of an electromagnet. It is nonetheless designed to be as modular as possible, so that a change of effector type is not hard to implement.

The communications module is an addition to the processing module in case communications are decided to be done via Ethernet TCP-Modbus. It would include a module capable of Ethernet communications.

The functional modules described above are expanded in Fig. 1.3 to show the electrical implementation of Safe Dip and the interfaces between its major components. The diagram identifies the system's power-distribution paths, control signals, sensor connections, communication interfaces, and actuator drivers. It also distinguishes between the high-power circuitry used by the motion and end-effector subsystems and the low-voltage circuitry used by the processing, sensing, and communications modules. This hardware-level representation provides the basis for selecting components, defining subsystem specifications, preparing the power budget, and verifying that the modules can operate together as an integrated system.

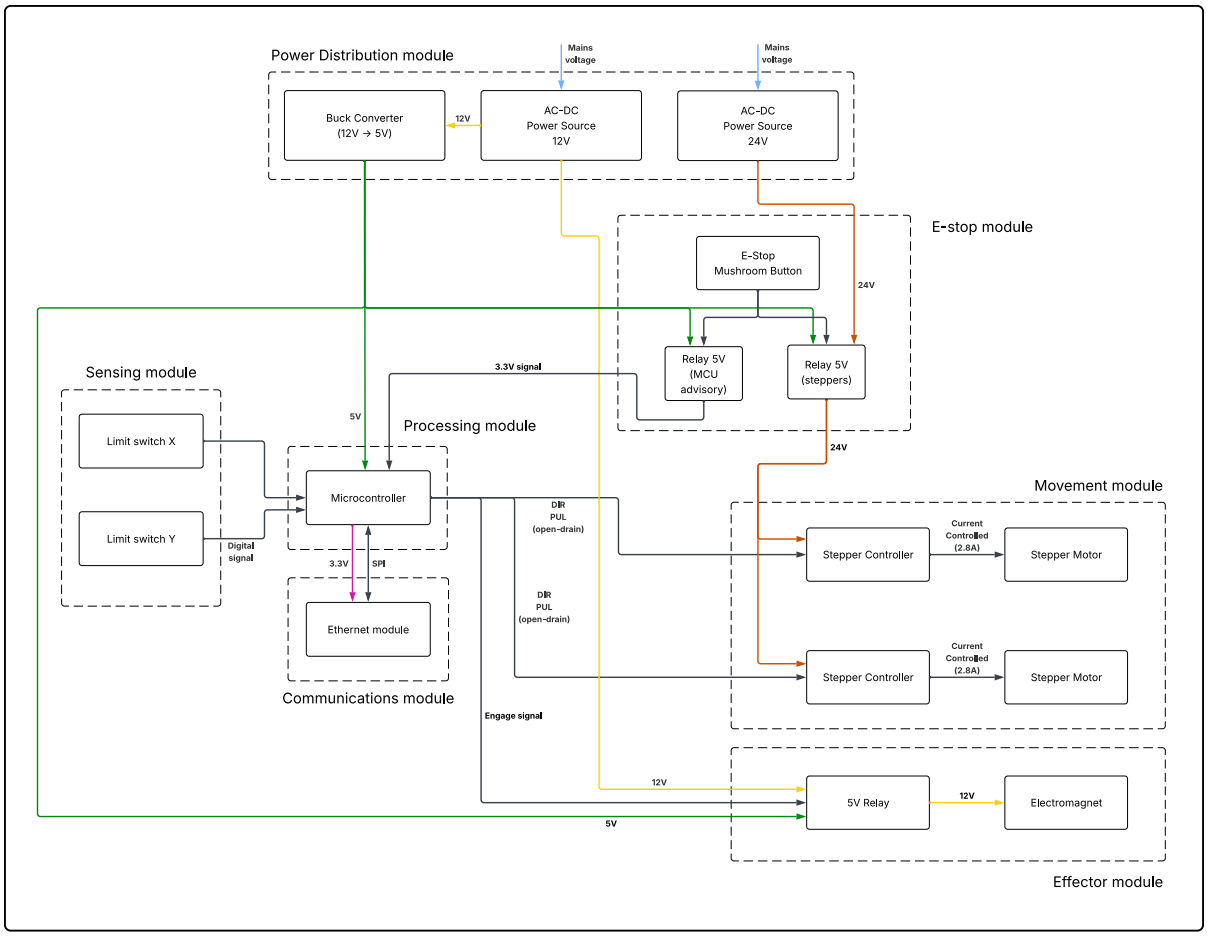


Fig. 1.3: Hardware architecture

1.7 Engineering Specifications

1.7.1 Power Distribution

Table 1.3
Power distribution specifications

Specific Component	Engineering Specifications	Justification & Verification
AC-DC power supply (motors)	• Input: 110 VAC, 60 Hz • Output: 24 VDC • Must supply sufficient current for two stepper motors at peak load	Verified by measuring output voltage under full load with a multimeter.
AC-DC power supply (12 V)	• Input: 110 VAC, 60 Hz • Output: 12 VDC • Ripple and noise within acceptable limits for logic, sensors, and effector	Verified by measuring output voltage and ripple with an oscilloscope under full load.
DC-DC Buck Converter (12 V $\longrightarrow$ 5 V)	• Input: 12 VDC • Output: 5 VDC • Ripple and noise within acceptable limits for logic and sensors	Verified by measuring output voltage and ripple with an oscilloscope under full load.

1.7.2 E-Stop

Table 1.4
E-stop specifications

Specific Component	Engineering Specifications	Justification & Verification
Emergency stop button	• Must offer a Normally Closed channel for cutting the power to the motors • Must be able to handle peak power coming from the AC-DC power supply (motors) • Requires manual reset before power is restored	Verified by actuating the E-stop during operation and confirming output power is cut.

1.7.3 Processing

Table 1.5
Processing specifications

Specific Component	Engineering Specifications	Justification & Verification
Microcontroller	• Operates at 3.3 VDC • Sufficient GPIO for stepper control, sensor inputs, and effector output • Supports USB UART for baseline serial communications • Capable of driving a Pulse Train Output (PTO) at the frequency set by the stepper and stepper controller	Verified using an oscilloscope to check GPIO voltage and PTO frequency.

1.7.4 Sensing

Table 1.6
Sensing specifications

Specific Component	Engineering Specifications	Justification & Verification
Limit switches	• Normally-closed configuration • One per axis, at the home end of travel, used for homing • Rated for the operating environment	Verified by commanding each axis to its respective home and confirming motion stops upon switch actuation.

1.7.5 Movement

Table 1.7
Movement specifications

Specific Component	Engineering Specifications	Justification & Verification
Stepper motors (x2)	• NEMA 23 frame • Holding torque sufficient to lift rated payload on vertical axis and drive belt under load on horizontal axis • Positional accuracy: ±1 cm	Verified by commanding each axis to a known target and measuring offset with a ruler.
Stepper motor drivers (x2)	• Compatible with a 24 VDC supply • Microstepping support • Current output set to motor rating	Verified by confirming smooth motion on both axes across the full travel range.

1.7.6 End Effector

Table 1.8
End effector specifications

Specific Component	Engineering Specifications	Justification & Verification
Electromagnet	• Holding force sufficient to retain rated payload throughout the dip cycle • Controlled by a single digital output from the microcontroller	Verified by attaching a part at rated payload and confirming no drop occurs throughout one full pick-dip-place cycle.
Effector driver	• Active-low control input that the microcontroller can drive from a single open-drain output • Isolates the microcontroller from the 12 V electromagnet circuit • Normally-closed contact that keeps the electromagnet energized whenever the relay coil is off • Contact voltage rating greater than the 12 V electromagnet supply • Contact current rating greater than the maximum electromagnet current	Command the electromagnet on and off from the firmware while monitoring the control input and the electromagnet voltage. The test passes if the driver switches the full electromagnet current reliably, the electromagnet stays engaged when the control input is released or the module loses its 5 V supply, it remains within an acceptable temperature range, and no damaging voltage transient is observed when the electromagnet is de-energized.

1.7.7 Communications

Table 1.9
Communications specifications

Specific Component	**Engineering Specifications**	**Justification & Verification**
Ethernet module	• Operating voltage compatible with the processing module • SPI interface compatible with microcontroller • Reports system state, axis positions, and fault events • Supports Ethernet and TCP socket communication	Verify communication between the STM32 and Ethernet module through SPI. Establish a TCP connection from an external computer and confirm that data can be transmitted and received.

Chapter 2

Background Research

Summary

This chapter describes the theory and the design of Safe Dip in detail. We explain the system's two-axis Cartesian gantry architecture and define its operating limitations. We present the testable system requirements and their verification criteria, describe each functional module through a system block diagram, provide engineering specifications for each subsystem, and outline the project budget.

2.1 Concepts, Algorithms, and Equations 24
2.2 Hardware Components and Mechanical Design (CAD) 26
2.3 Software and Programming Languages 36
2.4 Power Budget 37

2.1 Concepts, Algorithms, and Equations

This section presents the theory behind the design: how the axes convert motor rotation into motion, how the firmware generates and shapes the step pulses, and how the electrical loads were sized.

2.1.1 Axis Motion and Resolution

Both axes run open loop, so the firmware commands each position as a number of STEP pulses, and every distance in millimeters must be converted to pulses. Each pulse moves the motor by one microstep. With N_{full} full steps per motor revolution and the driver DIP switches set to M microsteps per full step, the number of pulses per revolution is

$$N_{\text{rev}} = N_{\text{full}} M \tag{2.1}$$

The 23HS22-2804S motor has a step angle of 1.8°, so $N_{\text{full}} = 200$, and both DM542TE drivers are set to $M = 8$, which gives $N_{\text{rev}} = 1600$ pulses per revolution.

One motor revolution moves an axis by a distance L that depends on its transmission. On the horizontal axis, a pulley with Z teeth advances a belt of pitch p_{belt} by

$$L_x = Z p_{\text{belt}} \tag{2.2}$$

The GT2 belt has a pitch of 2 mm and the pulley has 20 teeth, so $L_x = 40$ mm. On the vertical axis, a leadscrew with n_{starts} thread starts of pitch p_{screw} advances the nut by its lead,

$$L_y = n_{\text{starts}} p_{\text{screw}} \tag{2.3}$$

The Tr8x2 leadscrew has a pitch of 2 mm and a single start, so $L_y = 2$ mm. The scale factor of an axis, in steps per millimeter, is then

$$K = \frac{N_{\text{rev}}}{L} \tag{2.4}$$

A position x in millimeters, measured from the origin set by homing, corresponds to the step count

$$N = \text{round}(Kx) \tag{2.5}$$

and a step count N corresponds to the position

$$x = \frac{N}{K} = \frac{NL}{N_{\text{rev}}} \tag{2.6}$$

The firmware rounds to the nearest step. Positions are programmed in whole millimeters and K is an integer on both axes, so the conversion is currently exact. One step moves the axis by

$$\Delta x = \frac{L}{N_{\text{rev}}} = \frac{1}{K} \tag{2.7}$$

and at a linear speed v in mm/s the step rate is

$$f_{\text{step}} = Kv = \frac{v}{\Delta x} \tag{2.8}$$

The same factor converts an acceleration in mm/s² to steps/s². Table 2.1 lists the resulting values for each axis. For example, a 100 mm move takes 4000 pulses on the horizontal axis and 80,000 pulses on the vertical axis.

Table 2.1

Conversion between stepper driver pulses and axis travel

Quantity	**Symbol**	**Horizontal axis (X)**	**Vertical axis (Y)**
Full steps per revolution	N_{full}	200	200
Microsteps per full step	M	8	8
Pulses per revolution	N_{rev}	1600	1600
Travel per revolution	L	40 mm	2 mm
Scale factor	K	40 steps/mm	800 steps/mm
Distance per step	Δx	0.025 mm	0.00125 mm

2.1.2 Leadscrew Payload

The load that the vertical axis can lift follows from the kinematics of the leadscrew and the torque–speed curve of the motor. For a leadscrew with lead L, defined as the axial distance traveled by the nut during one revolution [4], the rotational speed n that gives a linear speed v is

$$v = nL \tag{2.9}$$

The vertical axis cruises at $v = 20$ mm/s, and with $L = L_y = 2$ mm this is $n = 10$ rev/s, or 600 rpm. At this speed the motor delivers the torque $T(\omega)$ read from its torque–speed curve, where $\omega = 2\pi n$ is the angular speed. The leadscrew converts this torque into an axial force, which carries a mass $m_{\max}$ against gravity, and a safety factor S gives the maximum safe payload:

$$F_{\text{axial}} = \frac{2\pi T(\omega)\eta}{L} \tag{2.10}$$

$$m_{\max} = \frac{F_{\text{axial}}}{g} = \frac{2\pi T(\omega)\eta}{Lg} \tag{2.11}$$

$$m_{\text{safe}} = \frac{m_{\max}}{S} = \frac{2\pi T(\omega)\eta}{SLg} \tag{2.12}$$

Here η is the mechanical efficiency of the leadscrew, which accounts for frictional losses, and g is the gravitational acceleration. A safety factor of $S = 2$ is applied to account for

uncertainties in the loading, the acceleration, drivetrain losses that the nominal efficiency does not capture, and differences between the theoretical and the available motor torque.

The torque must be read from the torque–speed curve at the operating speed. The static holding torque must not be used for a moving axis, because the available torque of a stepper motor decreases as its speed increases. The 150 rpm point quoted for the motor in the movement module is well below the 600 rpm of the vertical axis.

2.1.3 Step Pulse Generation

Each axis uses a hardware timer channel in output-compare toggle mode, so the pin changes level every half period. With a timer clock f_{timer}, the compare register is advanced by

$$N_{\text{half}} = \frac{f_{\text{timer}}}{2 f_{\text{step}}} \tag{2.13}$$

timer ticks on every event.

2.1.4 Motion Profile

The motion planner accelerates at a constant rate a, cruises at the maximum speed, and brakes down to a low landing speed v_{land} so that the axis stops exactly on the target step. Braking starts when the remaining distance equals

$$d_{\text{brake}} = \frac{v^2 - v_{\text{land}}^2}{2a} \tag{2.14}$$

For a move of D steps too short to reach the maximum speed, the peak speed is

$$v_{\text{peak}} = \sqrt{aD} \tag{2.15}$$

2.1.5 Electrical Loads

The power drawn by each load is $P = VI$, and the total is compared with the rated power of each supply. The magnet current follows from its measured resistance R,

$$I = \frac{V}{R} \tag{2.16}$$

2.2 Hardware Components and Mechanical Design (CAD)

The components selected for Safe Dip were chosen to satisfy the engineering specifications defined in Section 1.7 while remaining within the project's budget constraint and accessible through standard commercial channels. The hardware is divided into two categories: the electronics responsible for control, motion, sensing, and communications, and the structural components that form the physical frame of the gantry.

2.2.1 Mechanical structure

The structural frame and motion transmission of Safe Dip are built from standard commercial extrusion and linear-motion components, selected so the dimensions listed in Table 2.2 can be met without custom machining.

The frame is assembled from 4040 T-slot aluminum extrusion, which forms the horizontal beam, the vertical column, and the supporting legs shown in Fig. 2.1. The T-slot profile allows components to be mounted at arbitrary positions along each beam and secured with T-nuts and M5 hardware, which simplifies alignment during assembly and permits adjustment of the input, dipping, and output station positions.

The horizontal axis is guided by an MGN12H linear rail and carriage and driven by a GT2 timing belt, converting the rotation of the horizontal NEMA 23 into the linear travel between stations described in Table 2.2. The vertical axis uses a Tr8x2 leadscrew (2 mm lead) supported by pillow-block bearings and coupled to the vertical NEMA 23 through a flexible shaft coupler; the leadscrew converts motor rotation into the controlled lifting and immersion motion, and its selection is supported by the leadscrew speed and payload calculations in Section 2.1.2. A brass nut on the leadscrew carries the magnet holder, so that the holder rises and lowers as the screw turns.

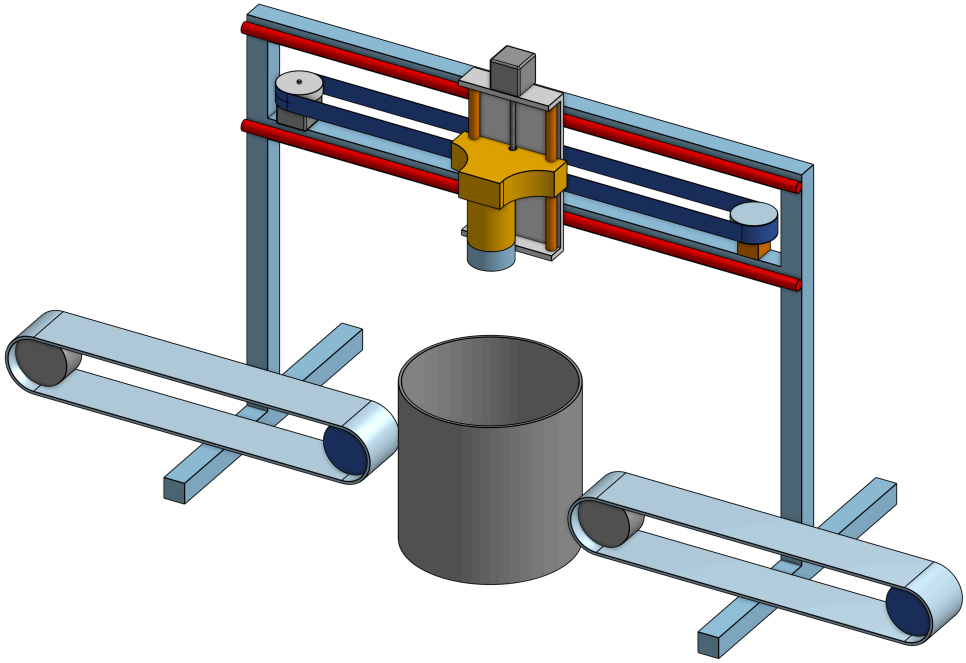


Fig. 2.1: CAD model of the Safe Dip mechanical arrangement - Perspective view

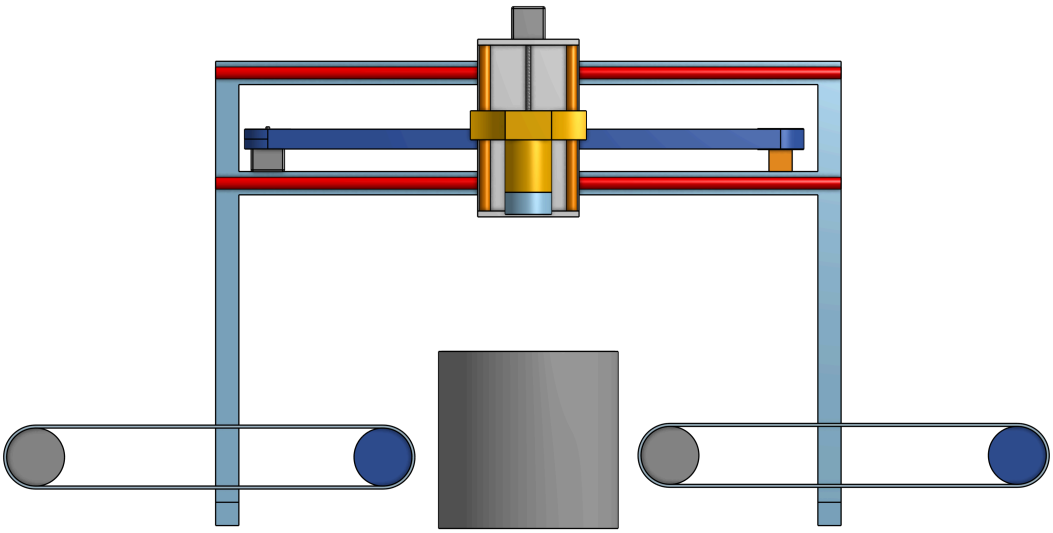


Fig. 2.2: CAD model of the Safe Dip mechanical arrangement - Front view

The CAD model in Fig. 2.1 shows the mechanical arrangement of the Safe Dip prototype. The model shows the relative placement of the horizontal axis, vertical assembly, input station, dipping station, output station, and end effector. The dimensions were checked against the selected mechanical components and the required axis travel and clearances.

The values in Table 2.2 describe the operating envelope of the prototype. The usable travels are the ones set in the firmware. Homing sets the origin of each axis a fixed back-off past its limit switch, 40 mm on the horizontal axis and 20 mm on the vertical axis, and a travel is measured from that origin. The distances are taken to the surface of the carriage that is closest to the limit switch: the left side of the horizontal carriage and the top of the vertical carriage.

Table 2.2
Principal dimensions of the Safe Dip prototype.

Dimension	**Value**	**Design Purpose**
Overall prototype dimensions	108 cm × 60 cm × 80 cm	Defines the approximate laboratory floor space and vertical clearance required by the prototype.
Usable horizontal travel	55 cm	Allows movement between the input, dipping, and output stations.
Usable vertical travel	20 cm	Provides sufficient lifting clearance and programmed immersion depth.
Bath envelope	30 cm × 30 cm × 30 cm	Defines the approximate size of the dipping container accommodated by the prototype.
Clearance above dipping bath	≈ 5 cm	Allows the workpiece and end effector to pass above the bath during horizontal transfer.
Maximum workpiece envelope	10 cm × 10 cm	Defines the approximate maximum workpiece size supported by the prototype geometry.

2.2.2 E-Stop and Protection module

The emergency-stop and protection hardware satisfies the requirements in Table 1.4. Its purpose is to remove power from the motor drivers independently of the STM32 while

treating the electromagnet as a load-holding device. This arrangement stops the hazardous motion without causing the suspended workpiece to fall into the coating bath.

A normally closed, mechanically latching mushroom emergency-stop button controls relay K1, which switches the 24 V supply of the stepper drivers. When the button is actuated, its contact opens and K1 releases, immediately removing power from the DM542TE drivers, causing the axes to coast to a standstill and implementing a Category 0 stop. Because the stop path is hardwired and normally closed, pressing the button, breaking its wire, or losing the relay's 5 V supply all remove driver power without requiring any action from the microcontroller. A second relay, K2, driven by the same contact, signals the emergency stop to the microcontroller, so that the firmware also stops the step pulses. The button remains latched until it is manually reset, satisfying the "requires manual reset" requirement in Table 1.4.

This stop path is intentionally independent of firmware and programmable logic, as IEC 60204-1 requires for a Category 0 emergency stop [5]. A welded K1 contact is the remaining single point of failure; a production version would use a safety relay with monitored, forcibly guided contacts.

A separate effector driver switches the electromagnet coil supply and is not included in the Category 0 cutoff. The electromagnet is treated as a load-holding device rather than as a hazardous actuator that must be immediately de-energized. Removing power from it during an emergency stop could create a secondary hazard by releasing the suspended ferromagnetic part into the coating bath. IEC 60204-1 requires the emergency-stop function to avoid creating such additional hazards [5]. The electromagnet therefore remains energized during the stop, and the part is released in a controlled manner only after motion has ceased.

An inline fuse holder provides overcurrent protection for the switched supply rail, and the resistor kit provides the resistors used in the circuit.

Fig. 2.3: E-stop mushroom button

2.2.3 Sensing Module

The sensing hardware provides the axis-homing feedback required by the pick-dip-place sequence and satisfies the requirements in Table 1.6. Because the stepper motors run open

loop, the controller knows each axis position only by counting the steps it has commanded, and that count is lost whenever the controller restarts or the drivers lose power during an emergency stop. Each axis therefore has a limit switch at the home end of its travel, which gives it a fixed physical reference: the axis is driven slowly toward its switch, and the point where the switch trips becomes position zero.

The HiLetgo KW12-3 micro limit switch shown in Fig. 2.4 was selected for this purpose. It is a compact snap-action switch, 20 × 10.5 × 6.5 mm, with a roller lever that lets the moving carriage actuate it without a precise contact point. Its single-pole double-throw contacts provide common, normally-open, and normally-closed terminals, so the normally-closed contact can be used: a broken wire then reads the same as a tripped switch and stops the axis instead of letting it drive past home. The contacts are rated for 5 A at 125 VAC, far above the low-voltage, low-current logic signal they switch in Safe Dip. The switches are sold in packs of ten, which leaves spares beyond the two that the axes require.

Fig. 2.4: HiLetgo KW12-3 micro limit switch with roller lever

2.2.4 Power Distribution Module

This system requires three main voltage lines, one at 24 V for the stepper motor controllers, another at 12 V for the effector, and finally one of 5 V for the processing module. To that end, two different power supplies provide the 24 V and 12 V lines, while a DC-DC buck converter generates the 5 V line.

The Mean Well LRS-350-24 is an AC-DC power supply that converts 110 or 220 Vrms into 24 V. It has four parallel terminals for the output, and it can supply a maximum current of 14.6 A.

Fig. 2.5: Mean Well LRS-350-24 AC-DC power supply

The less stringent requirements of the 12 V line means that it can be provided by an off-brand transformer, as long as it complies with the specifications. In this case, an ALITOVE DC 12V 5A Power Supply was chosen.

Fig. 2.6: ALITOVE DC 12V 5A Power Supply

Finally, for the 5 V line, as shown in Fig. 1.3, a HiLetgo buck converter based on the XL4016 chip was chosen.

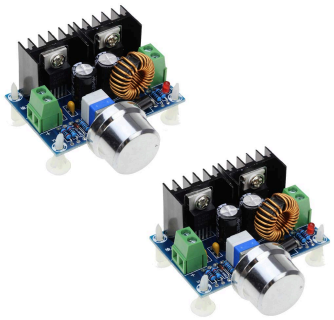


Fig. 2.7: HiLetgo buck converter

2.2.5 Processing Module

The STM32 NUCLEO-F446RE serves as the microcontroller responsible for coordinating the efforts for the sense-pick-dip-place sequence. It features an ARM Cortex-M4, 32-bit, 3.3 V microcontroller operating at a maximum frequency of 180 MHz. The STM32 family is widely adopted in embedded industrial designs due to its peripheral density, which allows for tight deterministic loops.

This board provides the necessary computing bandwidth to manage the real-time state machine while simultaneously running communication tasks. Crucially, its advanced hardware timers can be utilized in Output Compare mode to generate precise, jitter-free frequency pulses for step generation without taxing the CPU. Furthermore, its configurable interrupts can be used to immediately respond to external signals or to timer completion. The ability to delegate tasks to specialized hardware enables the Nucleo board to perform communication tasks with other peripherals.

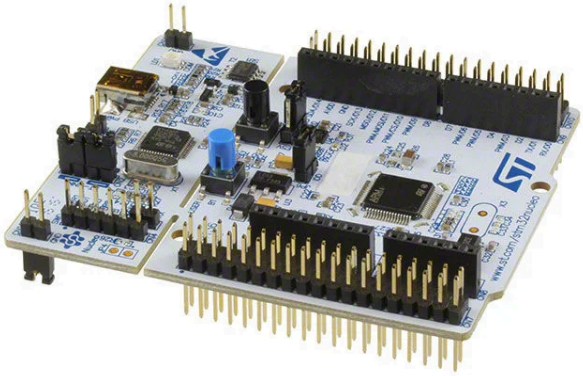


Fig. 2.8: STM32 Nucleo-F446RE microcontroller board

The STEP and DIR signals of the Nucleo board are connected directly to the optocoupler inputs of the stepper controllers, so no level-shifting buffer is used. The pins are configured as open-drain outputs that sink the optocoupler current, as described in Section 3.1.6.

2.2.6 Movement Module

For both the horizontal and vertical movement, a NEMA 23 stepper motor drives each axis. For this project, StepperOnline's 23HS22-2804S NEMA 23 stepper motor was chosen due to its static torque of 1.24 Nm and dynamic torque of almost 0.9 Nm at 150 RPM.

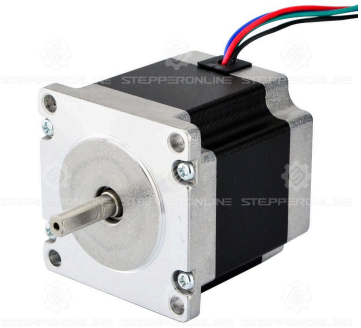


Fig. 2.9: NEMA 23 23HS22-2804S stepper motor

To drive the NEMAs using a microcontroller, a stepper motor controller is required to set their speed and direction. The DM542TE accepts a supply of up to 50 V, and in this project it is powered at 24 V and accepting 5 V signals.

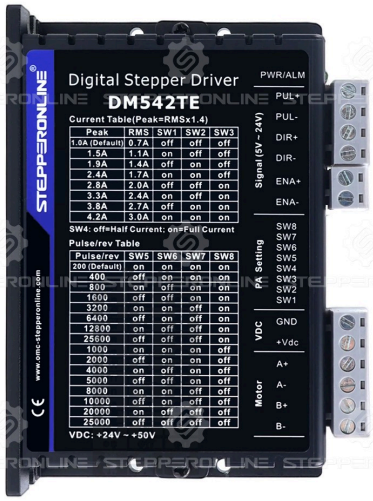


Fig. 2.10: DM542TE stepper controller

2.2.7 End Effector Module

The end-effector hardware holds the ferromagnetic workpiece throughout the pick-dip-place cycle and switches it under microcontroller control, satisfying the requirements in Table 1.8.

A Baomain BM-P 80-38 electromagnet is used as the gripping element. It holds the ferromagnetic workpiece by magnetic attraction rather than a mechanical grip, which avoids moving jaws near the coating bath and lets the part be released simply by removing coil

current. Its holding force is sized to retain the rated payload for the full duration of the dip, as required in Table 1.8, and it is driven from a single microcontroller output.

A 5 V single-channel relay module, built around the Songle SRD-05VDC-SL-C relay [14], switches the electromagnet supply. The module has an active-low input that it pulls up to its own 5 V supply through an opto-coupler LED and resistor, so the microcontroller drives it from a single open-drain output on a 5 V-tolerant pin, sinking about 4 mA, as measured on the bench. The 12 V supply is wired through the relay's normally-closed contact, so the electromagnet is energized whenever the relay coil is off and is released only while the microcontroller actively pulls the input low. The module was chosen because it isolates the microcontroller from the 12 V electromagnet circuit, because the electromagnet stays engaged if the microcontroller resets or loses power or the input wire breaks, and because its contact ratings exceed the 12 V electromagnet supply and coil current, matching the effector-driver requirements in Table 1.8.

A 1N4007 flyback diode is placed across the electromagnet coil to protect the relay contacts and the rest of the circuit from the inductive back-EMF generated when the coil is de-energized, addressing the "no damaging voltage transient" condition in the Table 1.8 verification.

Fig. 2.11: Baomain BM-P 80-38 electromagnet

2.2.8 Communications Module

Safe Dip communicates with the user over Modbus TCP, which reports the state of the machine and its faults and lets the operator change the parameters of the cycle. The NUCLEO-F446RE's UART peripheral, along with the ST-LINK connector on the board, is used for debug messages.

For Modbus TCP, a W5500 was chosen. The W5500 is a standalone Ethernet module with a full TCP/IP stack implemented in the hardware/firmware on the chip itself. This means that the microcontroller doesn't have to run a TCP/IP stack, but instead it only has to communicate with the module via SPI.

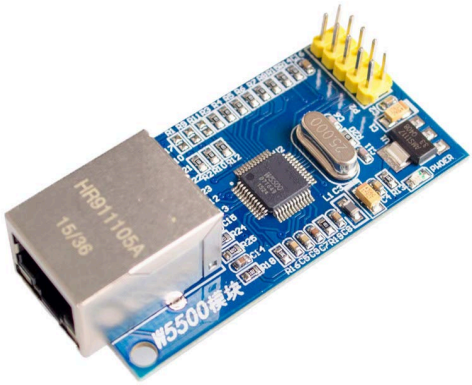


Fig. 2.12: W5500 Ethernet module

2.3 Software and Programming Languages

The firmware and the electrical and mechanical design of the project were made with a small set of programming languages and software tools. They were chosen so that timing and memory use stay predictable on a small microcontroller, so that, wherever possible, the tools come from the component manufacturer or are free, and so that both team members can work on the same files.

The Modbus TCP library is written in C99. C is ubiquitous in embedded systems because it is simple and close to assembly code, which makes its behavior predictable and its memory use efficient. The library has no dependency on the STM32, so the same code can be built and tested on a computer, and it uses no dynamic memory. The application firmware is written in C++20 and built with the compiler set to the C++23 standard. C++ makes abstraction easier: classes and a common interface let the sequencer run any operation without knowing how it works, and features such as `constexpr` and static assertions make it easier to express constraints in the code, so that mistakes are found when the program is compiled and not while the machine is running. Strongly typed enumerations and variants also make invalid states difficult to write. The code is compiled without exceptions, run-time type information, or dynamic memory, so the language adds no unpredictable timing or memory use.

The microcontroller is configured with STM32CubeMX and the STM32Cube HAL, and the firmware is built with the STM32CubeCLT toolchain (ST Arm Clang, CMake, and Ninja), which gives the team one reproducible command-line build. clangd and clang-format keep the editor feedback and the code style consistent, the on-board ST-LINK of the Nucleo board programs and debugs the microcontroller and provides serial output, and Git and GitHub keep the history of the firmware.

Testing uses a Modbus TCP client, `madbus-cli`, to read and write coils and registers through the whole path from the computer to the STM32, an oscilloscope to check the step pulses, which come from a hardware timer and can only be confirmed by measuring the signal, and a multimeter to check the supply voltages and signal levels.

The electrical schematic is drawn in KiCad, which is free and open source and checks the wiring rules before anything is built. The mechanical parts are modeled in Onshape, which runs in a web browser so that both team members can work on the same model, and the 3D-printed parts are prepared in Bambu Studio, which previews the supports and the print time before a part is printed.

Together, these tools keep the code generated by the vendor separate from the hand-written code, rely on hardware timers and static memory for what is critical for timing, and can be shared through a file, a repository, or an online editor, so that both team members can reproduce each other's work.

2.4 Power Budget

The power budget of the Safe Dip prototype is given in Table 2.3, which lists the voltage, current, and power of each load, and Table 2.4 compares the resulting demand with the rating of each supply. The two stepper motors and their drivers are powered from the 24 V rail, the electromagnet from the 12 V rail, and the emergency-stop, communications, and processing modules from the 5 V and 3.3 V rails derived from them. The figures for the motors and drivers are worst-case values, computed as the supply voltage multiplied by the rated current.

Table 2.3
Power budget for the Safe Dip prototype.

	Component	Voltage (V)	Current (A)	Unit Power (W)	Qty	Total Power (W)
Movement	Stepper motor (max usage per phase)	24	2.8	67.2	2	134.4
	Stepper motor driver	24	0.5	12	2	24
E-Stop	E-stop relay	5	≈0.07–0.10	≈0.5	2	≈1
End Effector	Electromagnet	12	1.5	18	1	18
	Electromagnet driver	5	≈0.07–0.10	≈0.5	1	≈0.5
Communications	Ethernet module	3.3	0.132	≈0.436	1	≈0.436
Processing	STM32 (at max rate)	3.3/5	≈0.1	≈0.33	1	≈0.33
				Total operational power		**178.7 W**

Table 2.4

Maximum power available from the Safe Dip AC-DC power adapters.

Power Rail	Power Adapter	Voltage (V)	Maximum Current (A)	Maximum Power (W)
24 V	Mean Well LRS-350-24	24	14.6	350.4
12 V	12 V AC-DC power adapter	12	5	60
			Total available power	**410.4 W**

The Safe Dip prototype uses two independent AC-DC power adapters. The 24 V supply provides a maximum of 14.6 A, corresponding to approximately 350.4 W, and powers the movement subsystem. The 12 V adapter provides a maximum of 5 A, corresponding to 60 W, and powers the electromagnet and low-voltage circuitry. Together, the two adapters provide a total rated output capacity of approximately 410.4 W. Table 2.5 compares the load on each rail with the rating of its adapter.

Table 2.5

Load on each power rail compared with the rating of its AC-DC power adapter.

Power Rail	Loads	Load (W)	Adapter Rating (W)	Share of Rating
24 V	Two stepper motors and two stepper motor drivers	158.4	350.4	45%
12 V	Electromagnet, and, through the 12 V to 5 V buck converter, the e-stop relays, electromagnet driver, Ethernet module, and STM32 (conversion losses neglected)	20.3	60	34%
	Total	**178.7**	**410.4**	**44%**

Chapter 3

Project Contribution

Summary

This chapter presents the work carried out to turn the Safe Dip design into a working prototype. It explains how the hardware and firmware modules connect and operate together, describes how each module was built, wired, and programmed, and records the significant problems found along the way with their causes and solutions. It then tests each system requirement against its success criterion, summarizes the results, and shows the prototype as built.

3.1 Design Integration and Flow Charts 41
3.2 Implementation 53
3.3 Troubleshooting 63
3.4 Requirement Testing and Results 68
3.5 Verification Summary 73
3.6 Final Design 75

3.1 Design Integration and Flow Charts

Safe Dip is organized around the NUCLEO-F446RE controller, which connects every module shown in Fig. 1.3 and wired as shown in the schematic in Appendix D. The controller sends STEP and DIR signals to the two DM542TE drivers, which power the NEMA 23 motors from the 24 V supply through relay K1. It reads the emergency-stop signal, the Start button, and the limit switches, drives the relay that switches the electromagnet from the 12 V supply, and communicates with the W5500 Ethernet module over SPI to publish the machine state through Modbus TCP. The emergency-stop button acts on the motor supply through the relays independently of the controller, so the firmware adds to that protection but is not part of it.

The control software uses a cooperative, nonblocking architecture. Rather than executing the complete pick-dip-place cycle as one blocking function, the software is divided into modules that are processed repeatedly from a single main loop: the cycle sequencer, one motion controller per axis, and the communications interface. Each call into a module performs a bounded amount of work and returns immediately. This keeps the controller responsive to the emergency stop, the limit switches, the Start button, and communication requests while a movement, effector operation, or dwell is still in progress.

The firmware is divided into layers, and the build system rejects any dependency that points the wrong way. The driver layer handles timers and pins; the control layer turns a position request into a speed over time; the sequencer runs an ordered list of steps; the operations layer defines the kinds of step that exist; a debounce layer filters the raw switch and button inputs; and the application layer is the only one that knows the whole machine, including its axes, cycle, and Modbus registers.

Fig. 3.1 shows the main loop in outline. The software first initializes the Ethernet controller, the Modbus TCP task, and the motion controllers, disengages the electromagnet, and then enters a continuous main loop. Each pass reads all inputs and the time once, so that everything in that pass acts on the same values. The limit-switch inputs pass through a 3 ms debounce first, described in Section 3.3.2. Two safety checks then run before the sequencer: any axis moving into its limit switch is stopped, and a pressed emergency stop raises a fault. A press of the Start button with the emergency stop released either clears the latched faults or, if there are none and the cycle is stopped, restarts the cycle.

If no fault is latched, the sequencer is stepped, and the current operation may itself raise a fault. A second check follows immediately, so a fault acts on the same pass in which it is raised: the machine enters its safe state instead of running the motion controllers. The communications interface is then serviced on every pass, including during a fault, so that

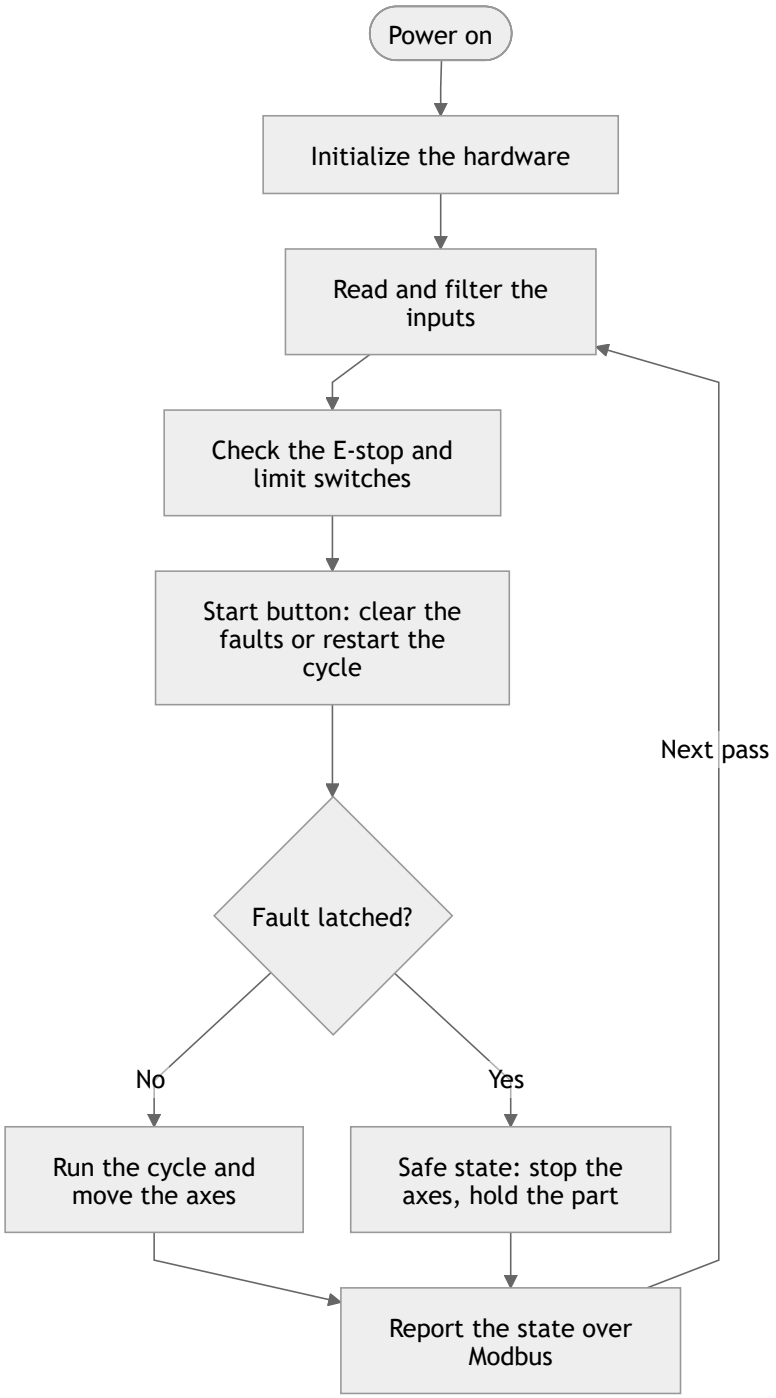


Fig. 3.1: The firmware's main loop

the system state remains observable. Last, the duration of the whole pass, Modbus included, is recorded with the processor's cycle counter, which shows how quickly the firmware can respond.

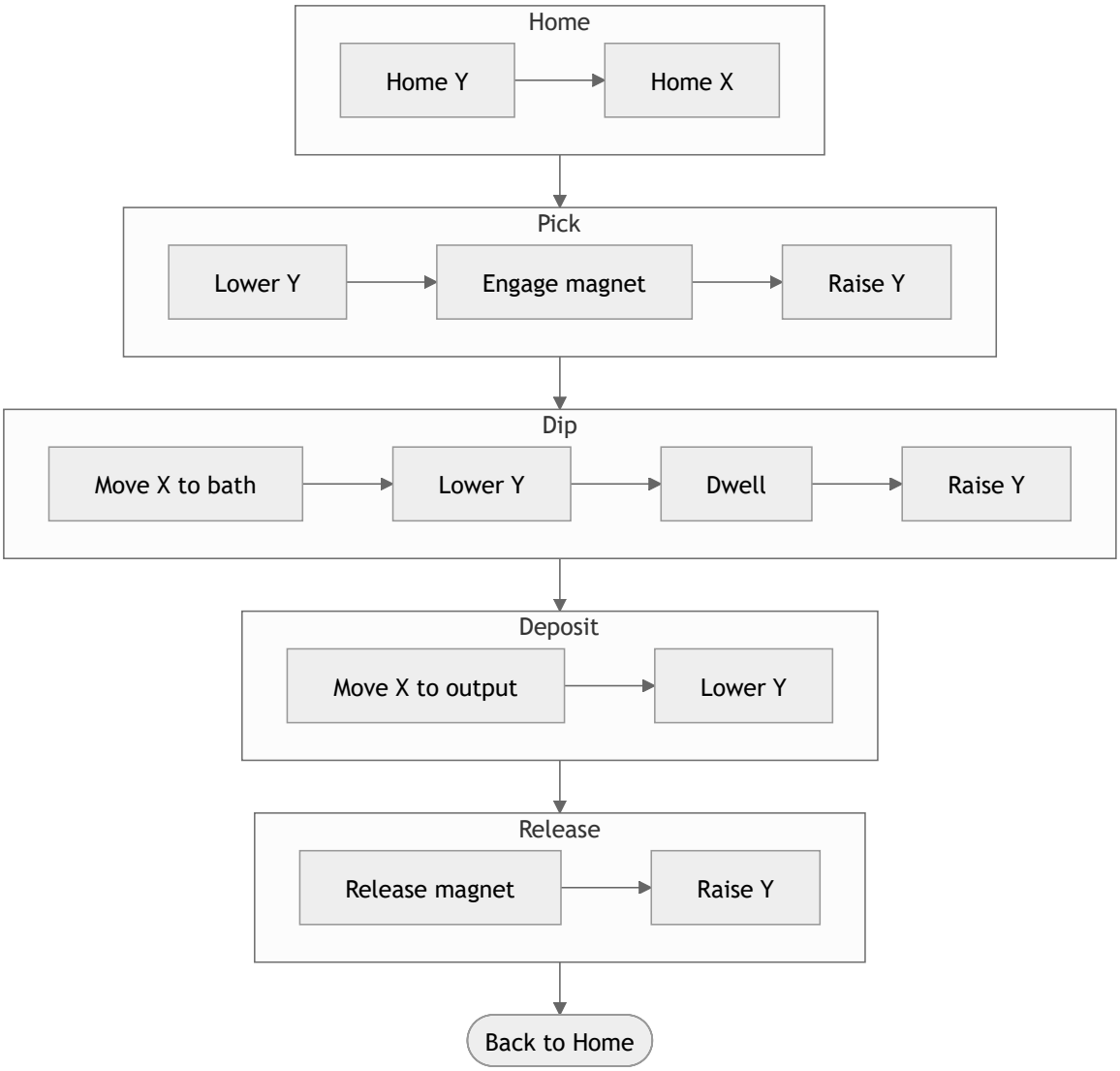


Fig. 3.2: Normal pick-dip-place operation sequence

3.1.1 Normal Cycle Sequence

The normal operating cycle is represented as an ordered list of thirteen nonblocking operations, shown in Fig. 3.2 and grouped into five process stages: home, pick, dip, deposit, and release. After the last operation completes, the sequence wraps around to the first, so the gantry repeats the cycle, homing both axes again at the start of each one.

Each operation represents an absolute objective rather than a blocking procedure. For example, *Raise Y* requests that the vertical axis move to its configured upper position. The operation hands the target to the motion controller and returns immediately; the sequencer then keeps checking the operation until the axis reports that the target has been reached.

Positions, the dwell time, and the effector settle time are held in a recipe, in millimeters and milliseconds, that can be changed over Modbus.

The home stage runs the vertical axis first, so that the end effector is raised before the gantry travels horizontally. Homing leaves the horizontal axis at its origin, which is where the pick stage takes place.

The part-ready trigger is not in the firmware yet, and the rest of this section describes its design. The trigger is a pair of Dupont pins on the controller. One pin is connected to ground, and the other to a free input, pin PA10 (D2), which tolerates 5 V and has its internal pull-up enabled. Closing the pair, with a jumper or with the dry contact of the external system, pulls the input low, which means that a part is ready. An open pair, and an unplugged or broken wire, leave the input high, which means that no part is ready, so a disconnected trigger can never start a cycle. The input is read once per pass of the main loop with the other inputs, and it goes through the same filter as the limit switches, so that a spike of noise cannot start a cycle: the trigger counts as asserted only after the input has read low for 3 ms, and it counts as released at once.

The trigger is used by a new operation, *Wait for trigger*, which is the first operation of the cycle, before the homing operations. While the trigger is not asserted, the operation keeps running and nothing moves, so no motion starts without it. When the trigger is asserted, the operation finishes, and the cycle homes both axes and runs as before. The Start button still restarts the sequence, which brings the machine back to the wait, so the trigger is needed at the start of every cycle. It is not checked while a cycle is running, so opening it during a cycle does not stop the machine; the emergency stop and the faults do that. The state of the trigger is also reported over Modbus as a discrete input, and the operation appears in the stage list of the monitor, so that the wait can be seen. Adding the operation shifts the operation index by one, so the Modbus map and the monitor change with it.

3.1.2 Operation Interface

Every entry in the sequence is an object implementing the same two-function interface:

- `start()` begins the operation, for example by giving a target position to a motion controller, and returns immediately.
- `poll()` performs any work the operation needs and reports whether the operation is still `running`, has `finished`, or was `aborted`.

The sequencer only ever calls these two functions. The interface is listed in Listing A.6. The correct implementation is selected at runtime through C++ virtual functions, so the sequencer needs no knowledge of which module is responsible for an operation or how that module works.

Each operation copies every configurable value it depends on when it starts. A value changed while an operation is running, for example through Modbus, therefore takes effect the next time that operation starts and never alters a run already in progress. An operation that cannot complete raises the fault that explains why before it reports `aborted`. Table 3.1 lists the operations.

Table 3.1
Operations available to the sequencer

Operation	**`start()`**	**Finished when…**	**Aborted when…**
Move axis	Converts the position to steps and sets the target	The axis is on the target step	Its limit switch is pressed, or something else stopped the move
Home axis	Moves toward the limit switch at homing speed	The axis has backed off the switch and the switch has released	The switch stays pressed after backing off, or the time limit passes
Effector	Engages or releases the electromagnet	The settle time has elapsed	Never
Dwell	Records the start time and the dwell duration	The dwell duration has elapsed	Never

This produces a clear abstraction boundary. The sequencer does not need to know which hardware timer is assigned to an axis, which GPIO pins control the stepper drivers, how many steps remain, or how the acceleration profile is calculated. Conversely, the modules do not need to know the structure of the pick-dip-place sequence, so each one can be developed and tested independently.

3.1.3 Cycle Sequencer

The cycle sequencer keeps an index to the current operation, a flag indicating whether that operation has been started, and a flag indicating whether the system is running. Fig. 3.3 shows one sequencer step.

The step is listed in Listing A.7. If the system is not running, the step returns without doing anything. If the current operation has not been started, the sequencer starts it and

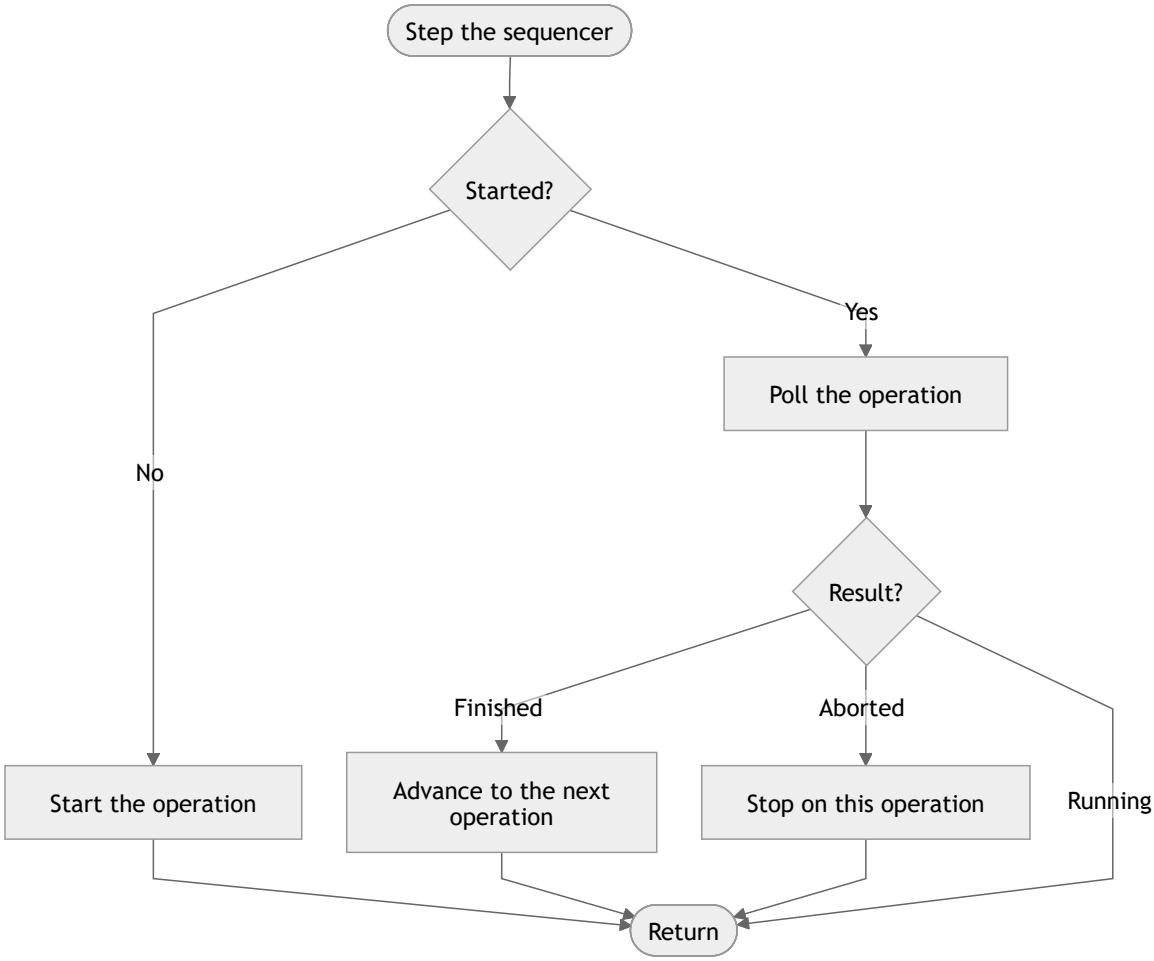


Fig. 3.3: One step of the cooperative cycle sequencer

returns; polling begins on the next iteration of the main loop. Otherwise, the operation is polled. While it reports `running`, the sequencer performs no further work. When it reports `finished`, the sequencer clears the started flag and advances to the next operation, wrapping to the first after the last. When it reports `aborted`, the sequencer stops and keeps its index on that operation, so the failed step can be read over Modbus.

The sequencer can be stopped, which halts it in place, or restarted, which resets the index to the first operation and sets it running. The Start button always uses the restart path, so the cycle never resumes from the middle of an operation whose physical state may no longer be known.

3.1.4 Move and Homing Operations

Fig. 3.4 shows a move from start to finish. Its checks repeat on every main-loop pass until the move finishes or aborts. A move never expects its limit switch to be pressed, whichever way

it is traveling: heading into the switch is a collision risk, and heading away from it means a stuck or miswired switch, lost steps, or an axis that was never homed.

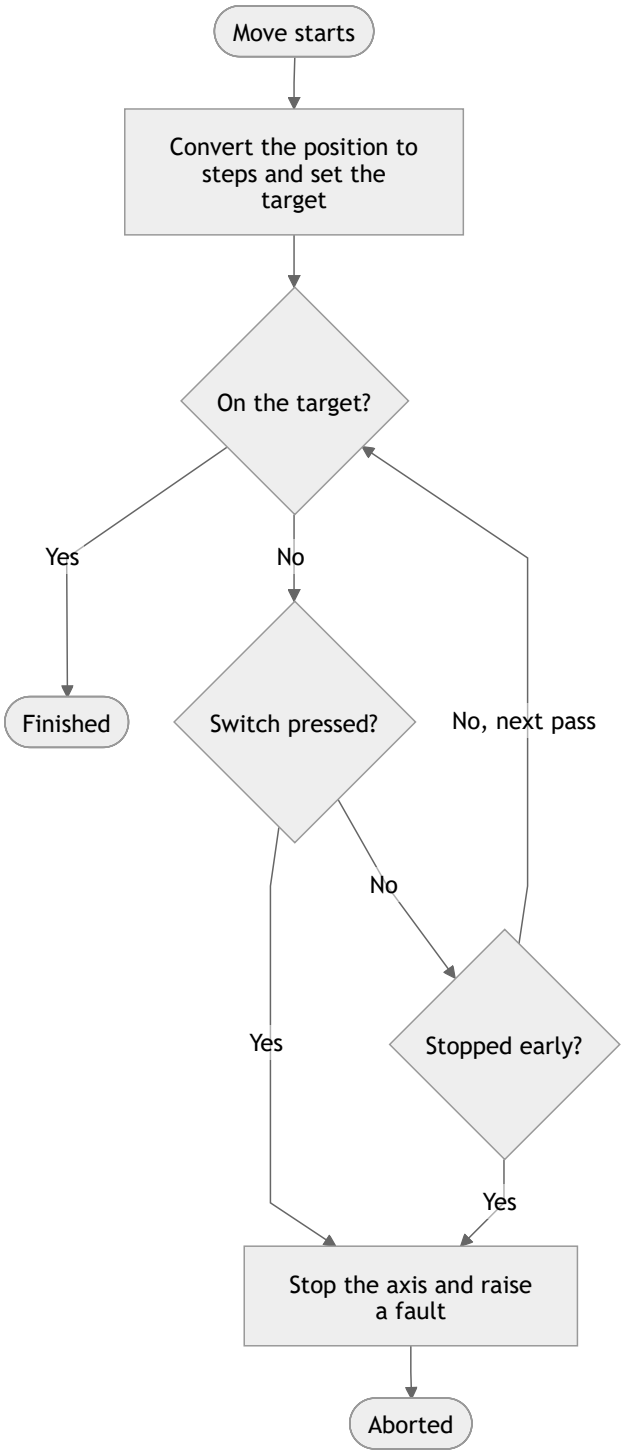


Fig. 3.4: A move operation

Homing, shown in Fig. 3.5, drives the axis toward its switch at homing speed. When the switch trips, the origin is set a fixed back-off away from the switch, 40 mm on the horizontal axis and 20 mm on the vertical axis, and the axis moves to that origin. If the switch is still pressed after backing off, it cannot be trusted and the operation fails. A time limit, derived from the axis travel, the back-off, and the homing speed, catches an axis that never reaches its switch. The poll function of the homing operation is listed in Listing A.8.

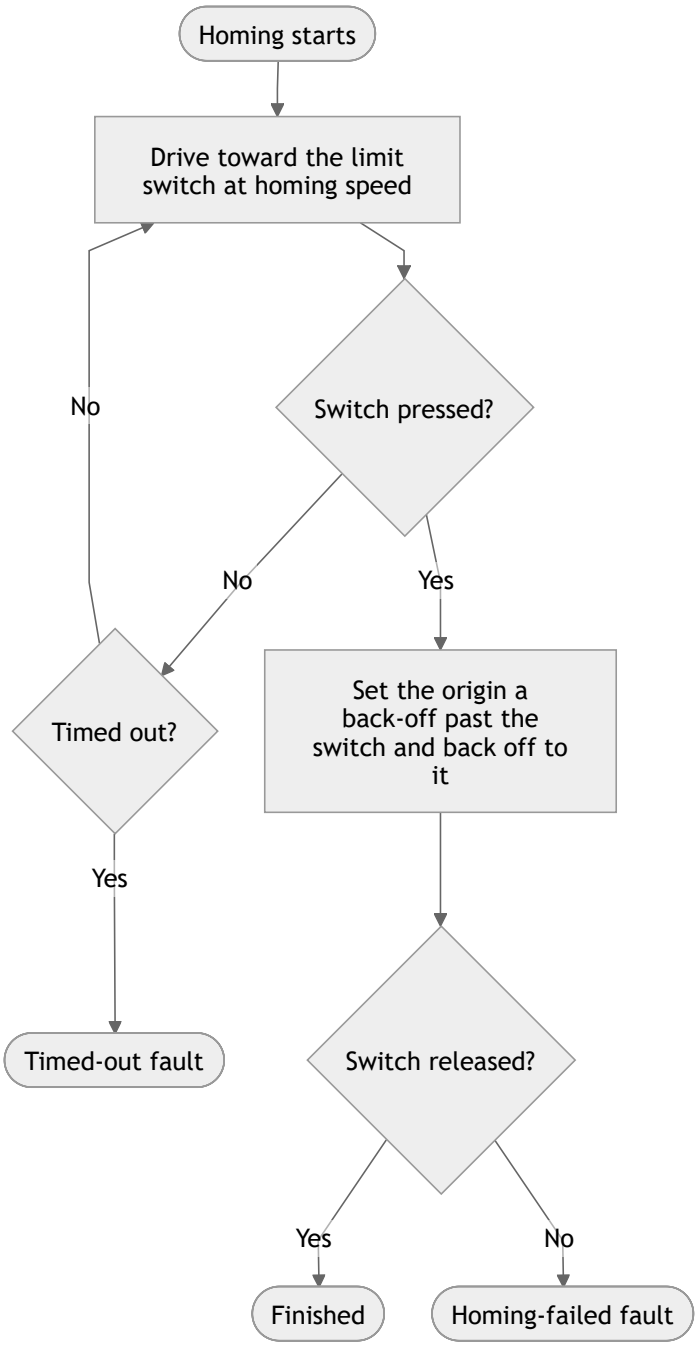


Fig. 3.5: Homing one axis

3.1.5 Motion Controller

Each axis has its own motion controller, which converts a goal into a continuously updated step rate. The controller supports two control modes. In *position mode*, used for every move in the cycle, it drives the axis to a target step count. In *velocity mode*, it holds a commanded speed; the homing operations use this mode to drive each axis toward its limit switch.

Fig. 3.6 shows how the controller updates an axis's speed. The update runs for each axis on every iteration of the main loop in which no fault is latched.

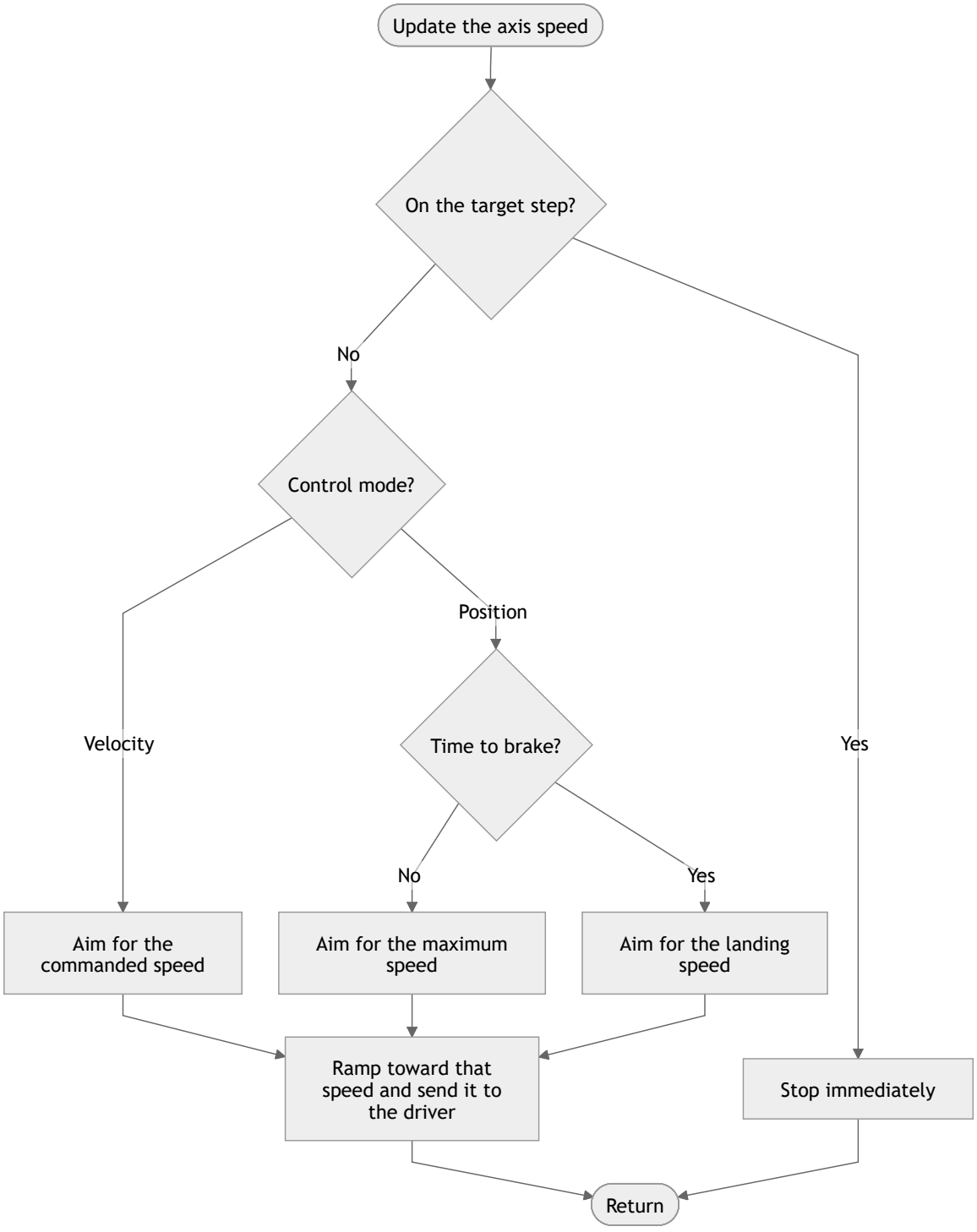


Fig. 3.6: One speed update of the motion controller

In position mode, the controller first checks whether the axis is exactly on its target step. This check runs on every call, not only once per millisecond, so that a fast-moving axis cannot step past the target unnoticed; when it succeeds, the axis is stopped immediately. Because the system tick advances once per millisecond and the main loop runs much faster, most of the remaining calls find no elapsed time and return early.

Otherwise, the controller decides whether it is time to brake. The distance needed to slow from the current speed v to the landing speed v_{land} at acceleration a is given by Equation (2.14). Once the remaining distance falls within that value, a landing flag is set and the desired speed becomes v_{land} instead of $v_{\max}$. If the axis is moving away from its target, the desired speed is zero, so the axis first brakes and then reverses. In velocity mode, the desired speed is simply the commanded speed. In both modes, the current speed is then ramped toward the desired speed by at most $a \cdot \Delta t$ and sent to the stepper driver.

The braking test and the speed ramp are listed in Listing A.2. The resulting motion is a trapezoidal speed profile, accelerating, cruising at $v_{\max}$, and braking, followed by a short approach at the landing speed, which lets the axis stop exactly on the target step instead of overshooting or stalling short of it. With the current tuning, the horizontal axis uses $v_{\max} = 200$ mm/s, $a = 100$ mm/s², and $v_{\text{land}} = 2.5$ mm/s, which the scale factor in Equation (2.4) converts to 8000 steps/s, 4000 steps/s², and 100 steps/s. The vertical axis uses $v_{\max} = 20$ mm/s, $a = 10$ mm/s², and $v_{\text{land}} = 0.5$ mm/s, or 16,000 steps/s, 8000 steps/s², and 400 steps/s. The usable travel is 550 mm on the horizontal axis and 200 mm on the vertical axis. Each travel is measured from the origin that homing sets, and to the surface of the carriage that is closest to the limit switch, as explained for Table 2.2.

3.1.6 Step Pulse Generation

The motion controller only decides *how fast* each axis should turn; the individual STEP pulses are produced by hardware. TIM2 runs at 1 MHz, and each axis uses one of its channels in output-compare toggle mode. On every compare event, the timer toggles the STEP pin and an interrupt schedules the next toggle half a period later, where the half period in timer ticks is given by Equation (2.13).

The interrupt code is shown in Listing A.1. The interrupt also counts one step on each active edge, which gives the controller the axis position, and loads any new speed requested by the controller on each inactive edge, so that speed changes never shorten a pulse. When the requested speed is zero, the interrupt stops rescheduling itself and the channel is released.

The STEP and DIR pins are configured as open-drain outputs that sink current from the stepper drivers' optocoupler inputs, whose anodes are tied to 5 V. The microcontroller

therefore never drives these lines to 3.3 V, and the 5 V-tolerant pins interface with the drivers directly, without a level-shifting buffer.

3.1.7 Fault Handling and Emergency Stop

Every abnormal condition is represented by one of seven latched faults: emergency stop, interrupted operation, timed-out operation, homing failure on either axis, and limit-switch hit on either axis. Whichever part of the firmware detects the condition raises the fault, but only the main loop reacts to faults and clears them; the latch and the safe state are listed in Listing A.5. While any fault is latched, the machine stays in its safe state: the sequencer and both axes are stopped, and the electromagnet is kept engaged so that a suspended workpiece is not dropped into the bath.

The emergency stop is enforced first by hardware: the normally closed mushroom button releases relay K1, which removes the 24 V supply from the stepper drivers independently of the firmware. A second relay signals the stop to the controller. The input is read at the start of every main-loop pass and is wired so that a broken wire reads the same as a pressed button. Because every module returns within a bounded time, the loop period is short enough that polling detects a press without the need for an external interrupt. The loop timer of the monitor (Fig. 3.14) showed that one whole pass, Modbus included, took about 0.29 ms on average, and between 0.27 ms and 0.31 ms over a 60 s window. The longest pass since the controller was reset was 3.2 ms.

Releasing the emergency stop does not resume the interrupted operation. A first press of the Start button clears the latched faults, and a separate second press restarts the cycle from its first operation, so one press can never do both. Because the drivers lose power during the stop, the axes may coast and the step count may no longer reflect the true gantry position. Because homing is the first stage of the cycle, each restart re-establishes a known position from the limit switches before any other motion.

3.1.8 Communications

The Modbus TCP server runs on the W5500 at port 502 and advances one step per main-loop pass without ever blocking. It serves a single connection, so only one Modbus client can be connected to the controller at a time. Read-only registers report the emergency-stop and Start inputs, whether the sequencer is running, whether each axis is on target, the commanded effector state, the current operation index, both axis positions in steps, every latched fault, the fault that came first, and the last and longest main-loop times. Writable registers hold the recipe; a value outside its allowed range is rejected, and a multi-register write is checked in full before anything is applied. A request whose header is malformed, or whose length

exceeds the largest the protocol allows, closes the connection, because the start of the next request can no longer be found.

Overall, this architecture combines cooperative scheduling with asynchronous peripheral operation. Hardware timers and their interrupts perform the timing-critical work, while the main loop coordinates higher-level behavior through the sequencer. This keeps the production sequence easy to understand while isolating the detailed control of the motors, end effector, dwell timing, and communications inside dedicated software modules.

3.2 Implementation

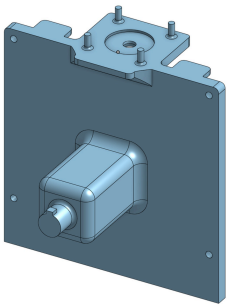
This section describes how each module was built, wired, and programmed, and how it was first tested. The complete electrical schematic is shown in Appendix D, and the wiring of each module is described below with the controller pins it used. Problems found during this work are covered in Section 3.3.

3.2.1 X Axis and Frame

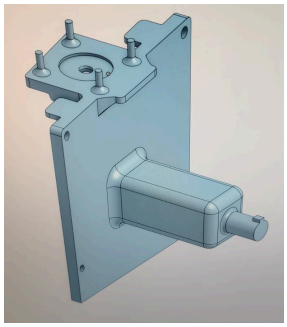
The horizontal axis was designed around an aluminum extrusion frame with two linear rails and four bearing blocks, driven by a stepper motor through a GT2 timing belt. The frame extrusions were cut to the required dimensions, and the rails were measured and installed on the base frame. The mechanism was then modeled at full scale in Onshape from the real dimensions of the frame and rails, as shown in Fig. 3.7, including the plate that connects the bearing blocks to the rest of the moving assembly.

Fig. 3.7: Full-scale CAD model of the frame with the linear rails and bearing blocks

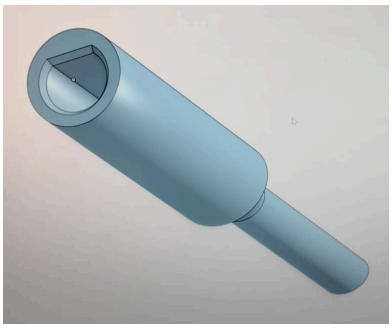
The parts that carry the belt drive were designed in CAD and 3D printed, as shown in Fig. 3.8. The base plate combines the motor mount with a pocket for the transmission. At the far end of the belt path, a ball bearing and gear were joined through a printed adapter and secured with adhesive; this assembly sits in a holder fixed to the frame, and a printed cap fitted into the holder's groove keeps it from coming out when the belt pulls. Video 3.2 shows this fit. The team compared an oblong belt path with an elongated rounded triangle while working out the belt geometry.



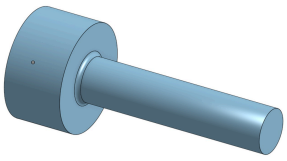
(a) Base plate with motor mount



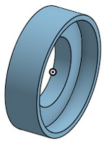
(b) X-axis holder with motor mount



(c) Motor and belt adapter



(d) Bearing and gear adapter



(e) Retaining cap

Fig. 3.8: X-axis parts in CAD

The X-axis holder went through five versions, the last printed 90% solid for strength. Braces were added so that the frame no longer wobbled. In the final adjustments, the rear motor mount and the belt were moved toward the end of the frame so that the mechanism reaches both ends of the axis, as homing requires, and grease was applied to the moving parts. The horizontal NEMA 23 motor was wired to a DM542TE driver powered from the 24 V supply, and its movement was adjusted until it looked right. The driver's STEP and DIR inputs were connected to controller pins PA0 and PB0, and its DIP switches were set to eight microsteps per full step, which gives the 1600 pulses per revolution used in Table 2.1. On the first full group test, shown in Video 3.1, both axes moved successfully with the live axis positions displayed on a laptop.

Video 3.1: First full test of both axes with the monitor interface
https://safe-dip.vercel.app/progress/week-8-first-full-test.mp4

Video 3.2: Fitting the ball bearing, adapter, and gear
https://safe-dip.vercel.app/progress/week-7-bearing-gear-fit.mp4

3.2.2 Y Axis

The vertical axis was designed with two bearing holders supporting the leadscrew and connecting it to the main base. A brass threaded nut on the leadscrew carries the central section of the magnet holder, so that the holder rises and lowers as the screw turns, while side sliders guide it and keep the magnet aligned. The plate shown in Fig. 3.9 joins this assembly to the X-axis carriage.

(a) Isometric view (b) Side view

Fig. 3.9: Vertical assembly plate in CAD

The vertical motor was wired in the same way to a second DM542TE driver, with its STEP and DIR inputs on controller pins PA1 and PC1 and the same microstep setting. The X-axis base holder and the Y-axis holder were then integrated into a single assembly, and the rails were aligned and secured. The vertical movement was first tested without the magnet by holding the nut so that it could not rotate, and the assembly was later checked by hand and with a drill driving the leadscrew.

3.2.3 End Effector

The electromagnet was characterized before it was installed. Its resistance was measured and gave an estimated current of 1.2 A, and a later test connected directly to the 12 V supply with

its flyback diode drew about 1.5 A, which is the value used in the power budget. A qualitative test with a hammer showed strong holding force even with little contact area.

The holder in Fig. 3.10 is a cylindrical cage that seats the magnet at its lower end. Its third version was printed and fitted the magnet smoothly, with a 5/16-inch screw added as a secondary retention. The planned IRLZ44N MOSFET driver was replaced with a 5 V relay module for its better safety behavior: the 12 V supply was wired through the normally closed contact, so the magnet stays powered whenever the relay coil is off, including if the controller resets or loses power. The relay module was powered from the 5 V rail, its input was connected to controller pin PB10, which drives it through an open-drain output, and a flyback diode was placed across the electromagnet to absorb the energy of the coil when the contact opens. Because there is no feedback from the magnet, the cycle waits a settle time after each switch, 300 ms by default and adjustable over Modbus, so that the magnet has time to grip or release the part.

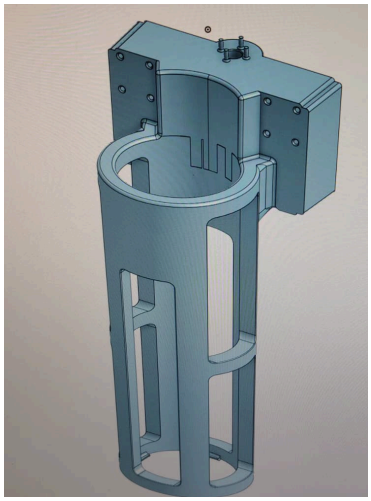
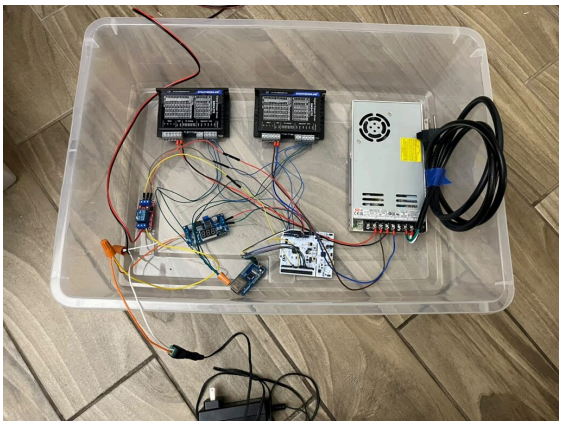


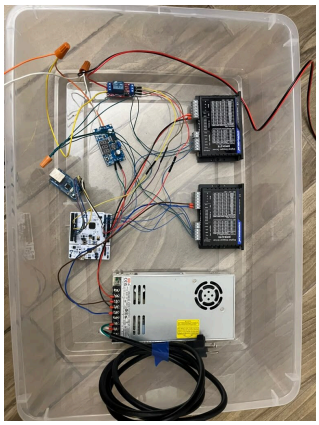
Fig. 3.10: Y-axis magnet holder in CAD

3.2.4 Power Distribution and Enclosure

The 24 V supply was assembled and its output adjusted to 24.1 V. The electronics were then placed in a plastic container, as shown in Fig. 3.11, with the stepper drivers and the 24 V supply given fixed positions. The installation was tested with a multimeter, confirming that the stepper drivers could be driven, that the relay worked, and that both supplies behaved correctly. The 24 V supply fed the two stepper drivers through the emergency-stop relay, the 12 V supply fed the electromagnet circuit and the buck converter, and the buck converter provided the 5 V rail for the Nucleo board and the relay modules. During the first full installation of the electronics and mechanics, the team redid much of the wiring and replaced the wire nuts with Wago connectors for cleaner and more reliable connections.



(a) One side



(b) The other side

Fig. 3.11: Electronics in the container during installation

3.2.5 Emergency Stop and Relays

After researching machinery safety and consulting the advisor on interrupting a DC load, the team revised the schematic so that relay modules cut the motor supply on an emergency stop and signal the controller. The normally closed contact of the mushroom button was wired to the coil of relay K1, which switches the 24 V supply of the drivers, and a second relay, K2, signaled the stop to controller pin PB5. In firmware, that input uses an internal pull-down, so a broken wire reads the same as a pressed button. The installed enclosure is shown in Section 3.6. An informal test confirmed that the emergency stop cut power to the stepper drivers and that the system restarted only after a deliberate press of the Start button. The relay later failed, as described in Section 3.3.6.

3.2.6 Sensing

Each axis was given a normally closed limit switch at the home end of its travel. Each switch was wired between its controller pin, PA8 for X and PA9 for Y, and ground, with the internal pull-up enabled, so an untouched switch reads low, while a pressed switch or a broken wire reads high and stops the axis. The Start button was wired the same way on pin PB4, but with its other contact to ground, so that it reads low when pressed. The switches could not be reliably soldered, so a way of mounting them without soldering was developed, along with a printed holder. In the first test, the X switch was wired and held by hand facing the carriage, and the firmware stopped the motors fast enough that the carriage did not crash into it. The firmware was then extended to read both switches, through a software debounce, and to home both axes.

Fig. 3.12 shows the two switches as mounted, in a frame from a video of the prototype running. The X switch is at the end of the upper rail, on the left in the picture, and the Y

switch is on the carriage plate beside the leadscrew. Each switch was glued in place, and the X switch was also wrapped in tape, for extra strength and to hide its wires. With both fitted, the cycle begins with the two homing operations, and Fig. 3.14 shows the machine in its homing stage with no fault latched.

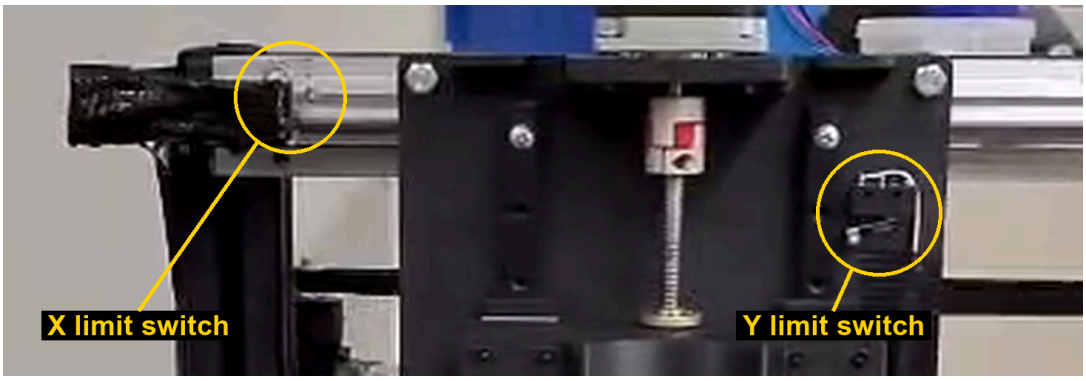


Fig. 3.12: The limit switches as mounted, in a frame from the video of the prototype running. The X switch is at the end of the upper rail, and the Y switch is on the carriage plate

Part detection changed during the project. An ultrasonic sensor was first selected on the professor's recommendation, but reviewers questioned the need for a sensor when the conveyor is outside the project scope, since the sensor's type and position depend on the conveyor used. With the advisor's approval, the sensor was replaced with a part-ready trigger input, leaving part detection to the external system. The design of the trigger is described in Section 3.1.1.

When both switches were first connected they proved very noisy; the fix, a software debounce and braided wires, is described in Section 3.3.2.

3.2.7 Motion Control Firmware

The firmware began with two-axis motion control, and the first hardware work was validating the STEP pulse generation, which was confirmed to produce the correct frequency and timing. A common-anode, open-drain connection was devised so that the 3.3 V controller could drive the 5 V driver inputs through its 5 V-tolerant pins. The first complete controller-to-motor test turned the motor ten times and returned it to the same place. Fig. 3.13 shows the bench setup used for the open-loop test in Video 3.3, and Video 3.4 shows a later simulation of both axes with an LED standing in for the electromagnet.

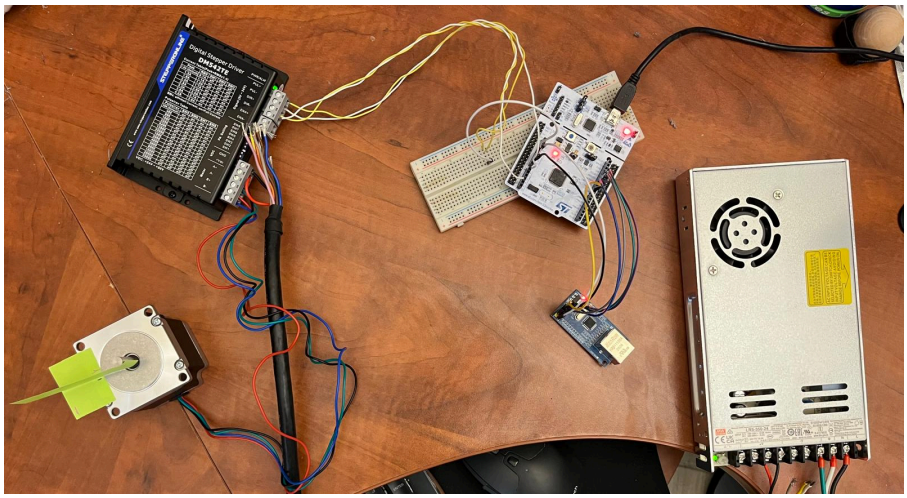


Fig. 3.13: Bench setup for the open-loop test: 24 V supply, DM542TE driver, NEMA 23 motor with a paper flag to show its position, and the Nucleo board with the W5500 module

> **Video 3.3:** Open-loop positioning test of one stepper motor
> https://safe-dip.vercel.app/progress/week-5-stepper-test.mp4

> **Video 3.4:** Bench simulation of both axes and the effector signal
> https://safe-dip.vercel.app/progress/week-6-stepper-simulation.mp4

Positions, speeds, and homing distances were later converted to millimeters using the scale factors in Table 2.1, and the conversion was tested. The conversion is listed in Listing A.3.

The scale, the travel of both axes, and the positions of the cycle were then tuned to the machine. First, rough values were found by measuring the axes with a long ruler. They were then fine tuned through the Modbus interface, by writing a position to the controller and measuring where the axis actually stopped. The first measurements were made from the center of the carriage, which gave wrong values. The point that matters is the surface of the carriage closest to the limit switch, because that is the part that meets the switch, so every distance was measured from there: from the left side of the horizontal carriage and from the top of the vertical carriage. Correcting this was the fine tuning.

3.2.8 Cycle and Fault Firmware

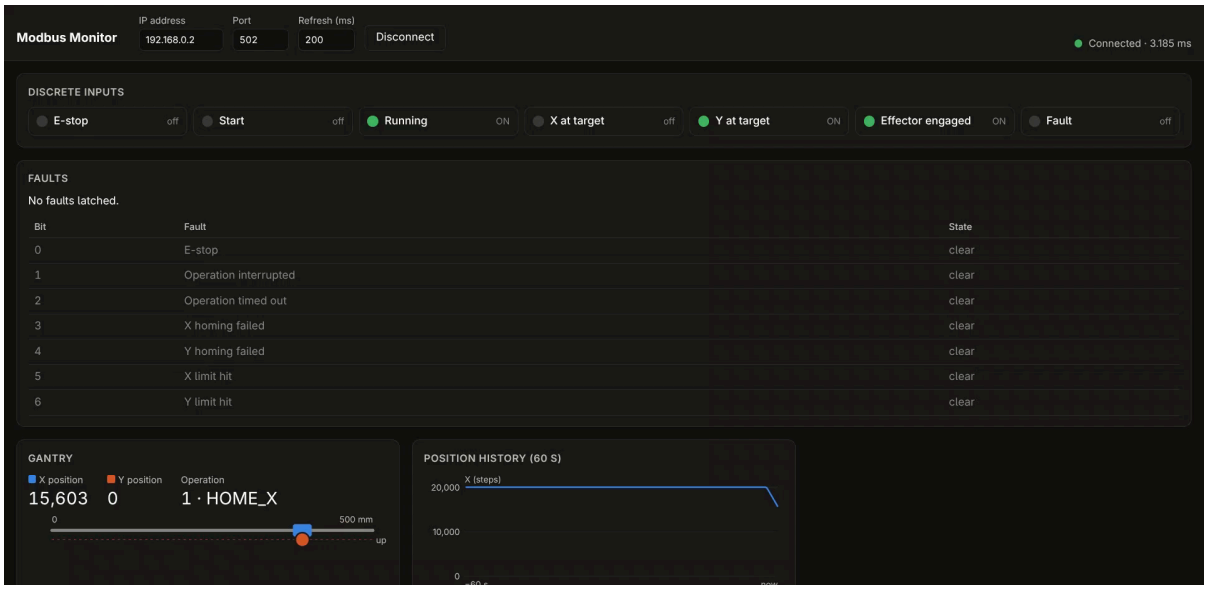
All thirteen operations of the production cycle were implemented in order, with a Start button that restarts the cycle and an emergency stop that halts both axes while keeping the electromagnet engaged. The homing operations were then written, followed by fault handling: homing gained a time limit, a stuck switch became detectable, and a limit-switch press during

a move stops the machine. Faults were made to stay latched until a Start press clears them. A full unattended cycle was run, started with the Start button.

A bug found while connecting the X limit switch prompted a review of the firmware's structure, which was reorganized into the layers described in the previous section, around a machine layer that owns the state and lets operations raise their own faults. Static analysis was configured for the firmware at the same stage. Homing was then enabled in the sequence for both axes, with a separate back-off for each. A lost-step check that compared each switch hit with the previous origin was removed at this stage. The main-loop time was then instrumented so that the responsiveness of the cooperative design could be measured. Finally, a code review of the whole firmware found and fixed several defects, described in Section 3.3.9.

3.2.9 Communications and Monitoring

The W5500 module was connected to the controller over SPI, with the clock, data-in and data-out lines on pins PA5, PA7 and PA6, its chip select on PB6, and its reset on PC7. The project's memory was then mapped to Modbus registers. The registers were first tested with the `madbus-cli` client, confirming that the code worked correctly. A browser-based monitor was then built, with the assistance of Claude Code, to display the Modbus output, and it served as a live view of the commanded axis positions during testing. The monitor runs on a computer connected to the controller's Ethernet port, and because the controller serves one connection, only one copy of it, or of any other Modbus client, can run at a time. The electromagnet state, the active faults, and the main-loop time were later added to both the Modbus map and the monitor, which graphs the loop time over the last 60 s. The monitor was generated with Claude Code, and parts of its code are shown in Appendix A.9. Fig. 3.14 shows it during a test. It lists the discrete inputs, the latched faults with the first one marked, the axis positions and their history, the loop time, the stage of the cycle that is running, and the parameters that can be written.



(a) The connection settings, the discrete inputs, the faults table with no fault latched, and the position of the gantry

(b) The axis positions and their history over the last 60 s, and the loop time

Fig. 3.14: The Modbus monitor while the cycle ran

Fig. 3.15 shows the same monitor after an emergency stop. The emergency-stop fault is latched and marked as the first fault, the Fault input is on, and the monitor shows that a Start press clears the fault and that a second press restarts the cycle.

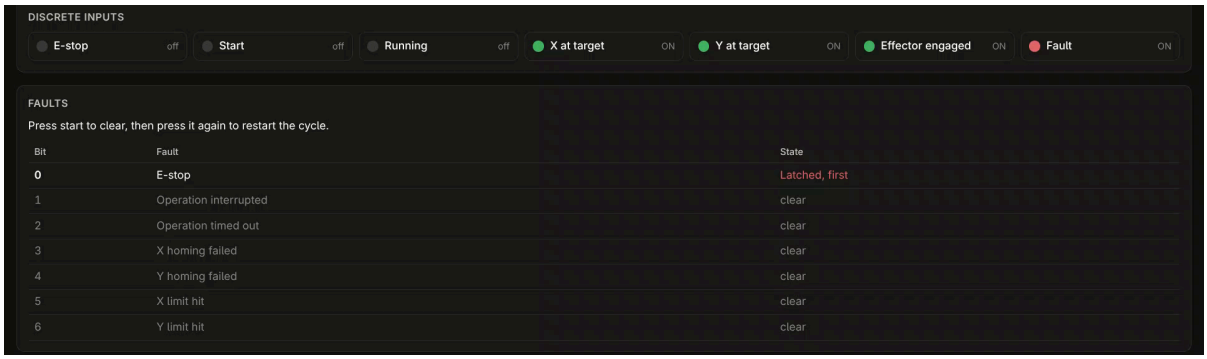


Fig. 3.15: The Modbus monitor after an emergency stop: the emergency-stop fault is latched and marked as the first fault

3.2.10 Firmware Size Over Time

Fig. 3.16 shows how the amount of firmware grew during the project. It counts the non-blank lines of source that the team wrote, in two groups: the application and the Modbus library. The vendor code (the HAL, CMSIS, and the WIZnet library) and the files that STM32CubeMX generates are not counted. The count was made for every commit of the firmware repository, following the main line of its history. It includes the code written during the summer break, before the build semester, such as the Modbus library.

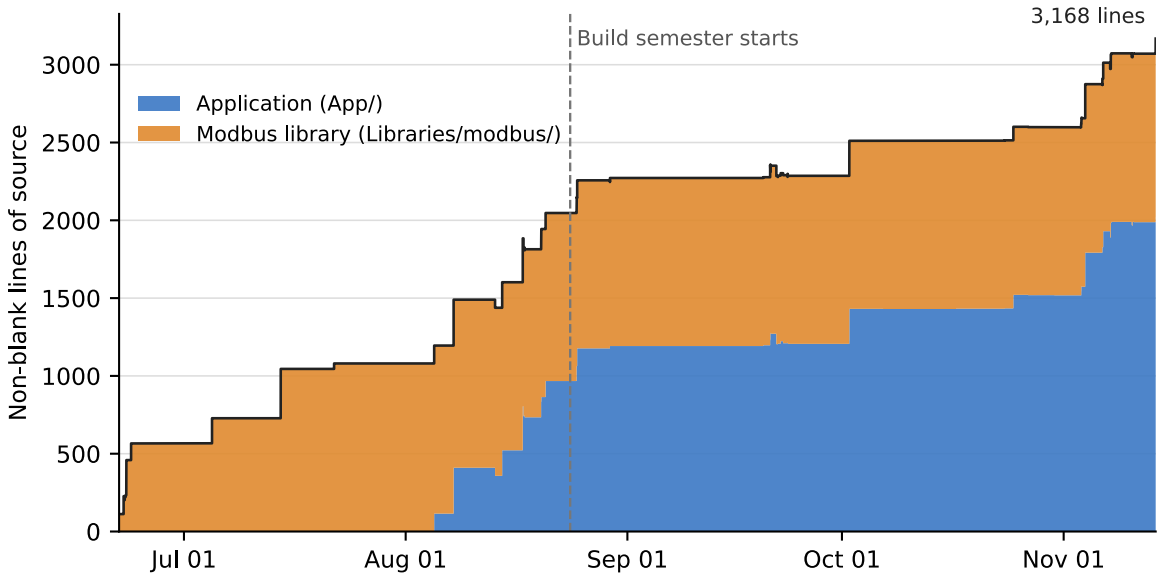


Fig. 3.16: Lines of firmware source over time, counted at every commit

The Modbus library came first. It was started on June 21 and had about 1,100 lines by early August, when the work on the application began. On August 24, the start of the build semester, the firmware had 2,047 lines in 40 files, 967 of them in the application. The application then

grew in steps, to about 1,200 lines by the time the full production cycle ran at the end of September, and to 1,517 lines by November 3. Most of the growth in November came from the fault handling, the homing of both axes, the machine layer, the limit-switch debounce, and the loop timer. At the time of writing, the firmware has 3,168 lines in 57 files: 2,067 in the application and 1,101 in the Modbus library.

3.3 Troubleshooting

This section records each significant problem found during the build, its cause, the solution applied, and the result.

3.3.1 Limit Switch Triggering a Constant Fault

Symptom. When the X limit switch was connected for the first time, it triggered a fault constantly and the machine could not run.

Cause. A firmware bug in the handling of the switch. It was most likely the lost-step ("drift") check, which compared each switch hit with the position that the step count expected. The check was badly set up, in a codebase where the axes had not been calibrated yet, so the expected position was wrong and the check faulted constantly. It was hard to find at first because the firmware showed little of its own state.

Solution. The problem led to a re-architecture of the firmware. The code was reorganized into layers around a machine layer that owns the state and lets operations raise their own faults, and the faults and the state were made visible over Modbus and in the monitor. The lost-step ("drift") check, which compared each switch hit with the previous origin, was removed. Homing was then enabled on both axes.

Result. The limit switches no longer trigger the constant fault. Both are read correctly, the two axes home at the start of each cycle, and the monitor shows no limit-switch fault (Fig. 3.14).

3.3.2 Noisy Limit Switches

Symptom. When both limit switches were connected for the first time and the system was run, an axis moved only a little during homing and then stopped, as if it had already reached its limit switch. The switch had not been touched. Soon afterward the limit-switch faults appeared, and the machine stopped. The switch inputs were reading as pressed at random moments.

Cause. Electrical noise picked up on the switch wires, which carry a pulled-up signal that is easily disturbed. The wires run close to the stepper motors and drivers.

Solution. A software debounce, listed in Listing A.4, was added to the switch inputs: a press counts only once the input has read pressed for 3 ms without a break, while a release counts at once. This improved the readings a great deal, at the price of letting the horizontal axis run about 0.6 mm further past the switch at its maximum speed. Some false triggers still appeared from time to time, so the ground wire and the pulled-up signal wire of each switch cable were braided together, and the cables were routed away from the stepper wiring. Twisting the pair makes both wires pick up the same noise, so it cancels out at the input.

Result. With the debounce and the braided wires together, the false triggers stopped.

3.3.3 Broken Ball-Bearing Housing

Symptom. Parts of the belt transmission of the X axis broke, on both sides of the axis. The stepper motor adapter was the main failure. The housing of the other ball bearing was damaged, but it was only twisted, not broken.

Cause. The belt needs side tension to work properly, and both parts were cantilevered: they were held only from the bottom, on a small surface. The side load of the belt broke them easily.

Solution. The stepper motor adapter was given a second hold, at the top as well as at the bottom, and the belt is no longer overtightened. The ridges of the belt transfer the motion, so it does not need much tension. The bearings for the upper part of the X axis were also reworked to stop its pieces from oscillating.

Result. Because both axes are homed from their limit switches at the start of each cycle, the occasional lost step does not matter.

3.3.4 Belt Tension and Skipped Steps

Symptom. During the first group test, the X axis skipped a few steps.

Cause. The skipped steps were largely attributed to the way the belt had been temporarily tensioned.

Solution. The motor and belt supports were completed, and the rear motor mount and the belt were moved toward the end of the frame. The belt is no longer overtightened, because its ridges transfer the motion without much tension.

Result. The later ruler measurement of the X axis showed drift under 0.5 cm (Section 3.4.2).

3.3.5 Electromagnet Overheating

Symptom. The electromagnet overheated while the machine was idle.

Cause. The firmware engaged the magnet at startup as its safe default, keeping it energized with no workpiece to hold. The output pin also came up in the state that energizes the magnet until the startup code ran.

Solution. The firmware was changed to start with the magnet disengaged, and the pin's initial state was changed to match.

Result. The magnet is now energized only when the cycle or a fault requires it.

3.3.6 Failed Motor-Supply Relay

Symptom. The stepper drivers remained powered even when the emergency stop removed power from the relay's coil.

Cause. The normally open contact of the relay connecting the 24 V supply to the drivers stayed closed. The team attributed this to a defective component, since the rest of the circuit is within its specifications.

Solution. The relay is to be replaced next week, probably by switching the AC side of the power supply instead. Until then, the emergency stop still stops the machine, because the second relay signals the controller, which stops the step pulses.

Result. Pending. Requirements R-07 and R-08 must be tested after the replacement.

3.3.7 Modbus Connections Dropping

Symptom. The Modbus connection dropped during operation.

Cause. A TCP keep-alive had been enabled on the socket so that a client that vanished without closing would not hold the only socket open. With it enabled, Modbus communication broke.

Solution. The keep-alive was removed.

Result. Modbus communication worked again. The original limitation returns: a client that disappears without closing can hold the socket until the controller is reset.

3.3.8 Modbus Stalling the Main Loop

Symptom. Network activity could stall the main loop and delay the step pulses, and after moving to nonblocking sockets, every Modbus connection was aborted.

Cause. The socket calls blocked, and interrupts were disabled during each access to the W5500. The vendor library also reported a busy socket before checking whether data had arrived, so a nonblocking socket never read anything.

Solution. The socket was made nonblocking, the unnecessary interrupt disabling was removed, and the library was patched to return received data first.

Result. The registers were read and written correctly with `madbus-cli`.

3.3.9 Defects Found in Code Review

Symptom. No failure had been observed. Before the final tests, the firmware was reviewed for defects that testing might not reveal.

Cause. The review found that the Modbus library was told its response buffer was larger than it was, accepted request lengths beyond the largest the protocol allows, and unpacked multi-coil writes into the wrong place. The network gateway address was outside the board's subnet, and a speed profile with zero acceleration would have caused a division by zero.

Solution. The buffer size and the request length are now checked, a malformed request header closes the connection, the coil unpacking was corrected, the gateway was moved into the subnet, and the speed limits are kept above zero.

Result. The fixes are on a development branch that had not been merged at the time of writing.

3.3.10 No Step Pulses From the Timer

Symptom. The timer channel produced no STEP signal.

Cause. Repeated speed requests before the first interrupt re-armed the channel from scratch each time, so its counter never reached a compare match. The compare value could also sit far behind the free-running timer.

Solution. The driver was changed to arm the channel only once per transition from stopped to moving, and to schedule the first pulse relative to the live counter.

Result. The pulses were confirmed on the hardware at the correct frequency and timing.

3.3.11 Stepper Overshoot

Symptom. The motor slightly overshot its target position.

Cause. The axis arrived at the target while still moving too fast to stop exactly on it.

Solution. A landing speed was added, so that the axis approaches its last steps slowly, similar to the creep speed used in elevators.

Result. The motor arrived smoothly and returned to the same place after ten turns.

3.3.12 Inverted Button Inputs

Symptom. The emergency-stop and Start buttons behaved opposite to their actual state.

Cause. Both inputs were read with an inverted comparison.

Solution. The polarity was corrected and the duplicated button read was removed.

Result. Both buttons were read correctly in the production cycle.

3.3.13 3D Printing and Part Fit

The mechanical parts required learning both Onshape and the slicer software, and several problems came from that learning process.

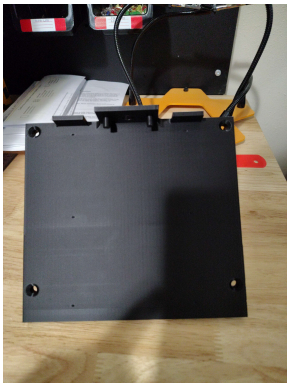
Symptom. Early prints had rough surfaces and uneven layers, as shown in Fig. 3.17. In assembly, holes and supports repeatedly missed their positions by a few millimeters, and some parts did not fit or meet their requirements.

Cause. The printer configuration, the material, and a lack of print supports caused the surface problems. The fit problems came from offsets that were not considered during design.

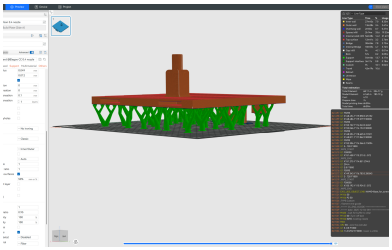
Solution. The printer settings were corrected and tree supports were added. Holes were enlarged, with 1 mm of added clearance giving a good fit on the frame supports. Several parts were redesigned: an anti-rotation section was added to the holder between the transmission and the belt, with its locking hole enlarged from 2 mm to 6 mm, and the top holder for the stepper was remodeled when the separate leadscrew holder was removed. Sections under stress, such as the connection between the two parts of the X-axis holder, were printed at higher density.

Result. The corrected prints came out smooth, as shown in Fig. 3.17, and the parts assembled. Denser parts are much stronger but take significantly longer to print.

(a) Early print



(b) Print after the corrections



(c) Slicer preview, with the tree supports in green

Fig. 3.17: 3D printing of the vertical assembly part

3.3.14 Damaged Leadscrew Holder

Symptom. The lower section of the leadscrew holder was damaged.

Cause. An accidental impact.

Solution. Two aluminum L-brackets were installed to repair and reinforce it, as shown in Fig. 3.18.

Result. The assembly passed the hand and drill tests that followed.

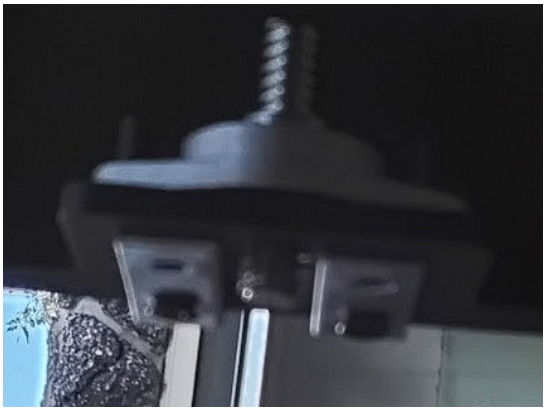


Fig. 3.18: Lower section of the holder reinforced with two L-brackets

3.3.15 Frame Wobble

Symptom. When the team first saw the full frame together, it wobbled slightly.

Cause. The frame had not yet been braced.

Solution. Braces were added, using metal L-supports like the one in Fig. 3.19.

Result. The frame no longer wobbled.

Fig. 3.19: Metal L-support used to brace the frame

3.4 Requirement Testing and Results

Each requirement in Table 1.2 was tested against its success criterion. All testing used an unheated, laboratory-safe medium at room temperature and the approved 1 lb prototype payload; the prototype does not validate operation at the intended industrial temperature or with a production coating medium. The first results were recorded in Video 3.5.

> **Video 3.5:** Checkpoint 2 viability demonstration of the integrated prototype
> https://safe-dip.vercel.app/progress/viability-video.mp4

3.4.1 R-01: Part-Ready Trigger

Requirement. The system shall not initiate a pick-dip-place cycle unless its part-ready trigger input is asserted. Success criterion: a cycle starts with the trigger asserted; no motion starts with it released or disconnected.

Test setup. A push button connected to the trigger input.

Procedure. Command a cycle with the trigger asserted, then release it, then disconnect it, and command a cycle each time.

Results. Not yet tested.

Discussion. The trigger input has not been added to the firmware yet, and the cycle currently starts from the Start button. Its design is described in Section 3.1.1: a pair of pins that reads as "part ready" only while it is closed, so the released and disconnected cases are the same state. The test setup is a jumper on the pair. Status: not yet tested.

Evidence. None yet.

3.4.2 R-02: Pickup Position Accuracy

Requirement. The system shall position the end effector at the programmed pickup position within ±1 cm along both axes before acquiring the workpiece. Success criterion: measured error within ±1 cm along both axes.

Test setup. A ruler placed along each axis.

Procedure. The system was commanded to the programmed pickup position, and the position of the end effector was measured relative to the target.

Results. The X axis showed drift under 0.5 cm. The Y axis was harder to measure because the platform moves, and it was judged to be within 1 cm without a precise measurement.

Discussion. The X result is a measurement and meets the criterion. The Y result is an estimate, and both axes will be measured again with a recording. A later check of the X axis, with the whole system running, also found it correct, but it was not recorded. Status: met.

Evidence. Video 3.5.

3.4.3 R-03: Workpiece Retention

Requirement. The system shall retain the workpiece throughout the complete pick-dip-place cycle without slipping or dropping it. Success criterion: the workpiece does not visibly slip or detach at any point.

Test setup. The electromagnet end effector with the approved payload.

Procedure. One complete pick-dip-place cycle was executed while observing the workpiece.

Results. The workpiece did not slip or fall in the test shown in the video. A weight of 2.5 lb or more with a minimal contact area was also held through a full cycle in an informal test.

Discussion. Retention exceeded the requirement in the informal test. Status: met.

Evidence. Video 3.5 (“Screw not slipping test”).

3.4.4 R-04: Immersion Depth

Requirement. The system shall lower the workpiece to the programmed immersion depth within ±1 cm. Success criterion: measured depth within ±1 cm of programmed depth.

Test setup. A ruler at the dipping station.

Procedure. Execute one dip sequence and measure the actual immersion depth relative to the programmed value.

Results. Not yet tested.

Discussion. Both axes have since been calibrated, but the immersion depth has not been measured yet. Status: not yet tested.

Evidence. None yet.

3.4.5 R-05: Dwell Time

Requirement. The system shall hold the workpiece at the programmed immersion depth for the programmed dwell time within ±1 s. Success criterion: measured dwell within ±1 s of the programmed value.

Test setup. A stopwatch or equivalent timing instrument.

Procedure. Execute one dip sequence and measure the dwell period.

Results. Not yet documented.

Discussion. Cycles with a part were run to check the dwell time, and the dwell looked correct, but there is no recording of it. Status: not yet tested.

Evidence. None yet.

3.4.6 R-06: Full Unattended Cycle

Requirement. The system shall complete one full unattended pick-dip-place cycle, beginning with the part-ready trigger and ending with workpiece release at the output station and return to the home position. Success criterion: every stage completes in order without operator intervention.

Test setup. The complete prototype with a workpiece at the input station.

Procedure. A cycle was started, and each stage was observed in order.

Results. Every stage completed in order without operator intervention, except the start from the part-ready trigger, because the cycle was started with the Start button.

Discussion. The start from the part-ready trigger is tested under R-01, which is not implemented yet. Status: met.

Evidence. Video 3.5.

3.4.7 R-07: Emergency-Stop Power Removal

Requirement. The system shall remove power from both stepper-motor drivers when the emergency stop is actuated. Success criterion: driver power removed and all axis motion stops.

Test setup. A multimeter at the supply terminals of both drivers.

Procedure. Actuate the emergency stop while an axis is moving and measure the power supplied to both drivers.

Results. Not yet tested.

Discussion. An informal test confirmed that the emergency stop cut driver power. The motor-supply relay then failed with its contact closed (Section 3.3.6), so the emergency stop does not currently remove driver power. It still stops the machine, because the second relay signals the controller, which stops the step pulses. The relay will be replaced next week, probably by switching the AC side of the power supply. The test must be run after the relay is replaced. Status: not yet tested.

Evidence. None yet.

3.4.8 R-08: Restart Inhibit

Requirement. The system shall remain inhibited following an emergency-stop event until the emergency-stop button has been reset and the operator performs a deliberate restart action. Success criterion: no motion occurs until the deliberate restart action is performed.

Test setup. The complete prototype with the emergency stop installed.

Procedure. Actuate the emergency stop during operation, reset the button, attempt to command motion without the restart action, then perform the restart action and confirm that normal operation resumes.

Results. Not yet tested.

Discussion. An informal test confirmed that the system restarted only after a deliberate press of the Start button. The test should be documented after the relay is replaced. Status: not yet tested.

Evidence. None yet.

3.4.9 R-09: State and Fault Reporting

Requirement. The system shall transmit its current operating state and any active faults through the selected communications interface during operation. Success criterion: each state transition and the active fault are transmitted with correct labels.

Test setup. A laptop running the Modbus monitor, connected to the controller over Ethernet.

Procedure. Monitor the interface during one complete cycle and introduce at least one detectable fault.

Results. State reporting was shown working in the video. State and faults were then checked informally over Modbus during a cycle. A screen recording of the monitor (Fig. 3.14) shows the operation index, the axis positions, the inputs that show whether each axis is on target, and the highlighted stage of the cycle changing while the cycle ran. It also shows the emergency-stop fault latched and marked as the first fault, with the instruction to press Start to clear it (Fig. 3.15).

Discussion. Fault reporting was implemented after the viability video was recorded. The screen recording of the monitor covers the emergency-stop fault, but the other faults have not been shown on it. Status: partially met.

Evidence. Video 3.5 for state reporting, and Fig. 3.15 for the fault.

3.4.10 R-10: Ten Consecutive Cycles

Requirement. The system shall complete ten consecutive unattended full pick-dip-place cycles using the approved prototype payload without dropping a workpiece, missing the intended placement position, or requiring operator intervention between cycles. Success criterion: all ten complete with no drops and no missed placements, and depth and dwell within tolerance.

Test setup. The complete prototype, a ruler, and a stopwatch.

Procedure. Command ten consecutive full cycles, recording the cycle time, commanded and measured immersion depth, and measured dwell time for each.

Results. Not yet tested.

Discussion. The test needs the depth to be measured in each cycle. Status: not yet tested.

Evidence. None yet.

3.4.11 R-11: Rated Payload

Requirement. The system shall complete the full pick-dip-place cycle while carrying a workpiece with a mass of 1 lb. Success criterion: the system lifts, transports, lowers, retains, and releases without stalling, losing position, slipping, or dropping.

Test setup. The complete prototype with a test weight.

Procedure. A weight was attached and one complete cycle was commanded.

Results. A weight of 2.5 lb or more, above the required 1 lb, was held through a full cycle in an informal test.

Discussion. The informal test exceeded the rated payload. Status: met.

Evidence. The informal test recorded on the Final Testing and Results page of the website.

3.5 Verification Summary

Table 3.2 summarizes the results of Section 3.4, in the order of Table 1.2. Trial counts are marked "–" where they have not been recorded.

Table 3.2

Verification summary of the system requirements

Req.	Requirement	Success criterion	Test method	Measured result	Trials passed	Pass/Fail	Evidence
R-01	No cycle without the part-ready trigger	Starts only when asserted	Push button: asserted, released, disconnected	Trigger input not added yet	–	Not yet tested	–
R-02	Pickup position	Within ±1 cm on both axes	Ruler	X: drift under 0.5 cm. Y: judged within 1 cm	–	Met	Video 3.5
R-03	Workpiece retention	No visible slip or detachment	One observed cycle	No slip or drop	–	Met	Video 3.5
R-04	Immersion depth	Within ±1 cm	Ruler	Depth not yet measured	–	Not yet tested	–
R-05	Dwell time	Within ±1 s	Stopwatch	Observed, not documented	–	Not yet tested	–
R-06	Full unattended cycle	All stages in order	One observed cycle	All stages, started with the Start button	–	Met	Video 3.5
R-07	E-stop removes driver power	Power removed, motion stops	Multimeter at drivers	Observed earlier; relay has since failed	–	Not yet tested	–
R-08	Restart inhibit	No motion before restart	Reset, attempt motion, restart	Observed, not documented	–	Not yet tested	–
R-09	State and fault reporting	Correct labels transmitted	Modbus monitor, induced fault	State shown; faults checked informally	–	Partially met	Video 3.5
R-10	Ten consecutive cycles	All complete, within tolerance	Ten recorded cycles	Depth not yet measured	–	Not yet tested	–
R-11	1 lb payload	Full cycle without failure	One cycle with weight	2.5 lb or more held through a cycle	–	Met	–

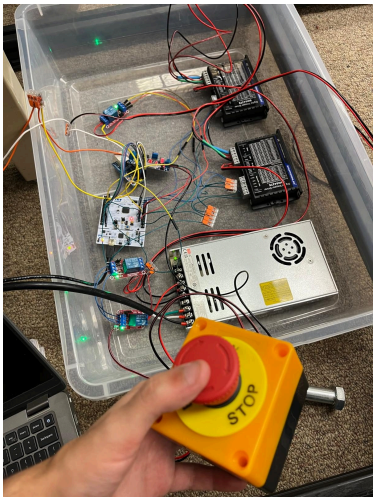
3.6 Final Design

Fig. 3.20 shows the current build of the gantry, and Fig. 3.21 shows the moving assembly, the end effector, and the electronics. Fig. 3.22 is a frame from a video of the assembled prototype running, with the limit switches fitted.

Fig. 3.20: Safe Dip gantry: aluminum frame, two horizontal rails, and the Y-axis assembly with its stepper motor

(a) Y-axis assembly on the X-axis rails

(b) Magnet holder with the electromagnet and the screw used as secondary retention



(c) Emergency-stop button with the enclosure holding the relay modules, stepper drivers, 24 V supply, and Nucleo board

Fig. 3.21: Details of the assembled prototype

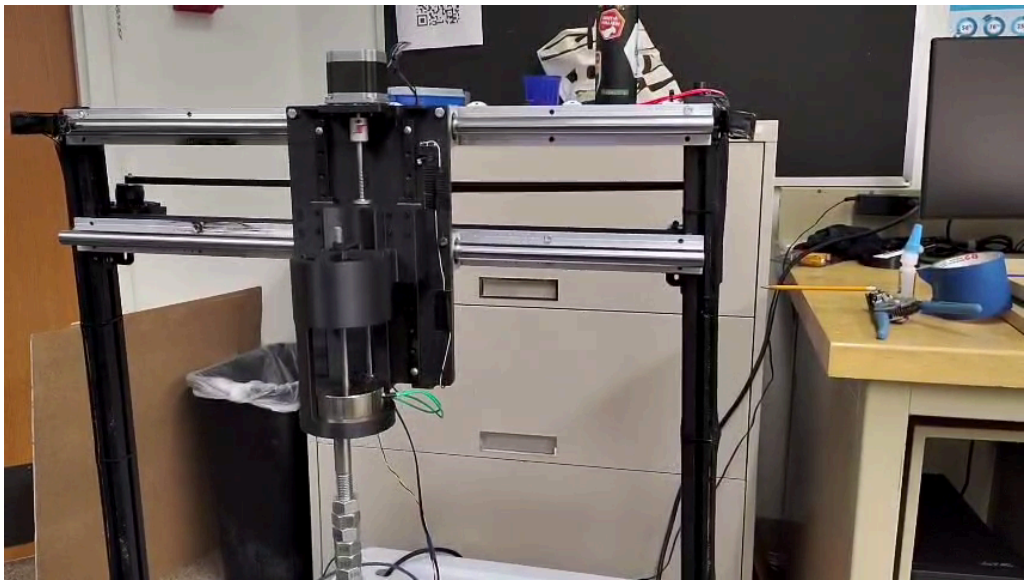


Fig. 3.22: Frame from a video of the assembled prototype running, with the limit switches fitted

Chapter 4

Non-Technical Considerations

Summary

This chapter presents the project planning and non-technical considerations associated with Safe Dip. It provides a monetary budget, a fifteen-week implementation timeline, and the distribution of responsibilities between the team members. The chapter then evaluates the environmental, health and safety, ethical, social, and sustainability aspects of the system. Particular attention is given to the hazards created by moving machinery, electrical power, and the handling of an approximately 10 kg suspended load for the fully-featured system, along with the prototype controls used to reduce those risks. Finally, the chapter discusses the project's potential to reduce worker exposure to hazardous and repetitive dipping operations while emphasizing honest reporting, maintainability, and responsible automation.

4.1 Project Timeline 78
4.2 Project Expenses 81
4.3 Distribution of Tasks 85
4.4 Health and Safety 88
4.5 Ethical Aspects 89
4.6 Environmental Aspects 90
4.7 Social Aspects 90
4.8 Sustainability 91

4.1 Project Timeline

The proposed implementation schedule spans sixteen weeks. Mechanical construction and firmware development proceed in parallel so that each subsystem can be tested soon after installation. The schedule intentionally reserves the final weeks for integrated testing, debugging, refinement, and presentation preparation.

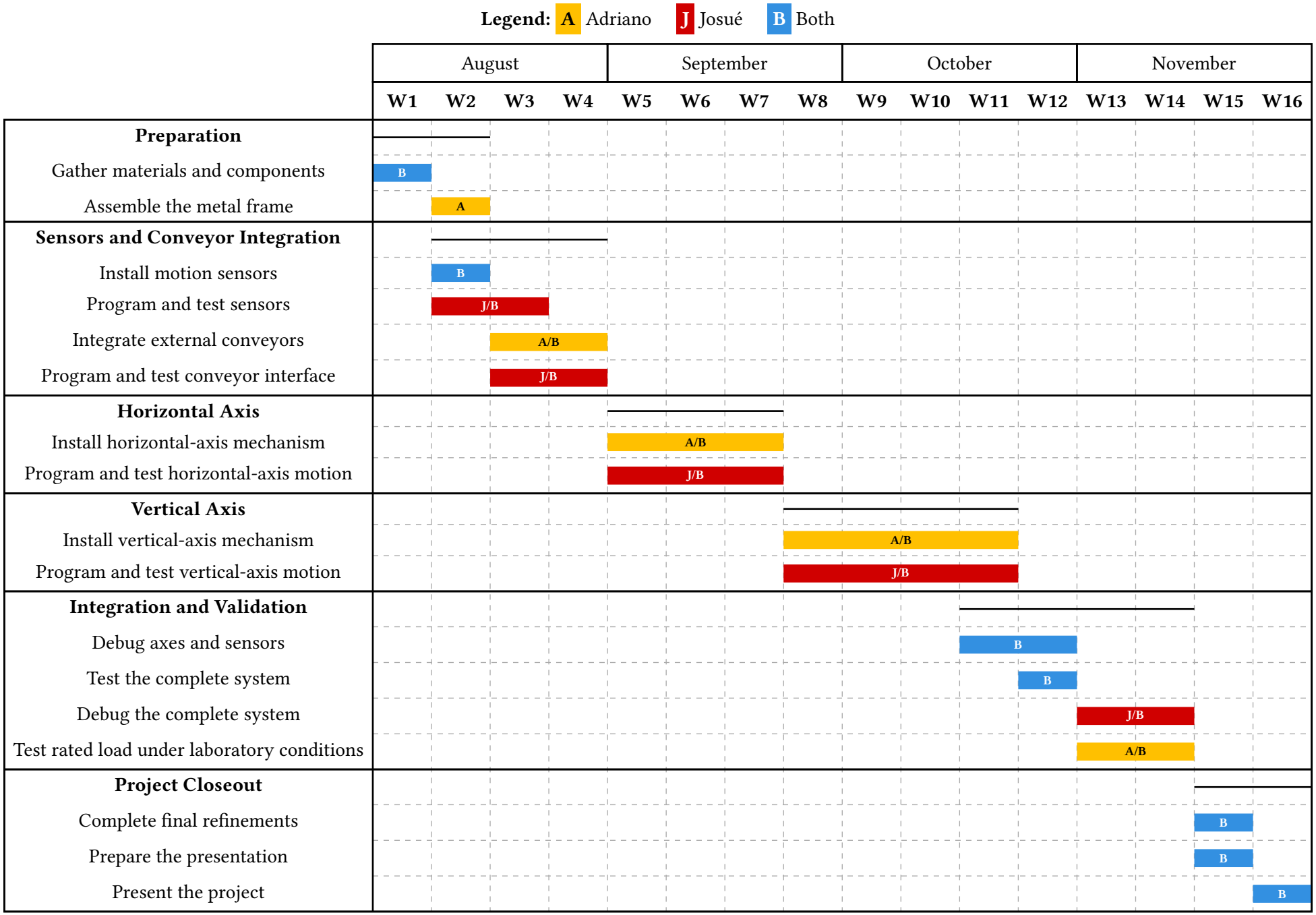


Fig. 4.1: Sixteen-week Safe Dip implementation schedule.

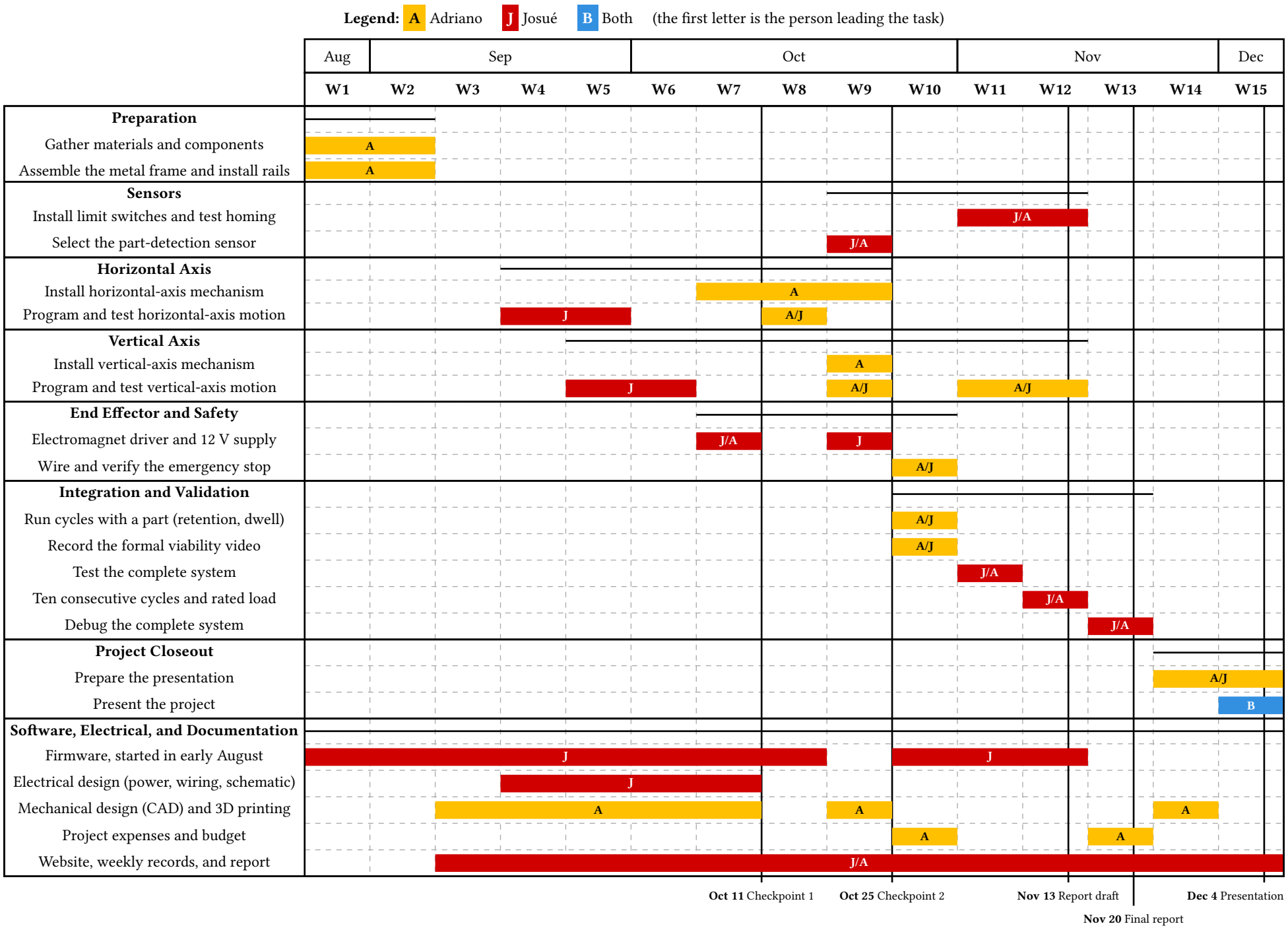


Fig. 4.2: Modified Safe Dip implementation schedule.

4.2 Project Expenses

This section compares three budgets: the budget proposed in the Senior Design Proposal, the total spent on the prototype to date, and the cost of the parts in the final design. Prices are from the commercial listings the parts were bought from and exclude taxes and shipping.

4.2.1 Proposed Budget

Table 4.1 is the budget submitted with the Senior Design Proposal.

Table 4.1

Proposed component budget, from the Senior Design Proposal.

Item	Component	Qty.	Price total
Single Output Switchable Power	Mean Well LRS-350-48	1	$ 35.06
Microcontroller	STM32 NUCLEO-F446RE	1	$ 31.00
4-channel digital buffer/line driver	SN74AHCT125N Quadruple Buffer	1	$ 0.80
Stepper Motor	23HS22-2804S NEMA 23	2	$ 26.74
Stepper Controller	DM542TE Stepper Controller	2	$ 42.00
Ethernet Module	W5500 Ethernet LAN Module	1	$ 6.50
E-Stop	1NC 1NO Red Mushroom Emergency Stop	1	$ 9.99
Transformer	ALITOVE Power Supply Adapter Converter Transformer	1	$ 14.00
Buck Converter	LM2596 DC-DC Buck Converter	1	$ 1.60
Electromagnet	Electromagnet Solenoid BM-P 80-38	1	$ 44.98
Proximity Laser Sensor	VL53L0X ToF Sensor	1	$ 2.98
Shaft Coupler	Shaft Coupler (6.35 x 8 mm)	2	$ 11.99
Limit Switch	HiLetgo KW12-3 Micro Limit Switch	1	$ 0.60
Rail Guide	MGN12H 300 mm Linear Rail Guide	2	$ 35.99
Lead Screw	TR8x2 Lead Screw (300 mm)	1	$ 11.99
Bearings	Pillow Block Bearing, 5/16 in (8 mm)	4	$ 8.49
Guide Rails	Linear Rail Slide SBR20-800 mm	2	$ 44.90

Item	Component	Qty.	Price total
Belt	GT2 Timing Belt	1	$ 16.88
Aluminum Beams	4040 T-Slot Extruded Aluminum Profile, 1000 mm	4	$ 79.99
Relay	One-Channel Relay Module	1	$ 3.70
Buzzer	Gikfun Passive Electronic Buzzer	2	$ 1.80
MOSFET Transistor	IRLZ44N MOSFET Transistors	4	$ 4.00
Terminal Block	Glarks Terminal Block Set	2	$ 0.26
Flyback Diode	BOJACK 1N4007 Rectifier Diode	1	$ 0.05
Wires	22 AWG Stranded Wire Spool	1	$ 12.29
Cable Wrap Insulation	Eventronic Heat-Shrink Tubing 2:1	1	$ 6.99
Fuse Holder	FEBRYTOLD 0-Gauge Inline Fuse Holder	1	$ 4.25
Resistor Kit	Upvivi Resistor Assortment Kit	1	$ 6.99
T-Nuts	4040 Series T-Sliding Nuts	15	$ 9.99
Bolt and Nut Kit	M5 Button-Head Socket Cap Screw and Nut Kit	1	$ 16.99
Motor Mounting	NEMA 23 Steel Mounting Bracket	4	$ 11.99
Nut Block	Lysee Lead-Screw Nut Block	2	$ 20.19
Conveyor Belt	12 V DC Electric Desktop Conveyor Belt	1	$ 99.51
	Total without conveyor belt (before tax)		**$ 525.94**
	Total with conveyor belt (before tax)		**$ 625.45**

4.2.2 Total Expenses to Date

Table 4.2 lists what has been spent on the prototype so far. Parts sold in packs are counted at the price of the whole pack, and parts that were bought but later dropped from the design are included.

Table 4.2

Total expenses to date.

Item	Component	Qty. bought	Amount spent
Power Supply (24 V)	Mean Well LRS-350-24	1	$ 31.47

Item	Component	Qty. bought	Amount spent
Microcontroller	STM32 NUCLEO-F446RE	1	$ 31.00
Stepper Motor	23HS22-2804S NEMA 23	2	$ 26.74
Stepper Controller	DM542TE Stepper Controller	2	$ 42.00
Ethernet Module	W5500 Ethernet LAN Module	2	$ 12.99
E-Stop	Red Mushroom Emergency Stop	1	$ 9.99
Power Supply (12 V)	ALITOVE Power Supply	1	$ 14.00
Buck Converter	LM2596 DC-DC Buck Converter	3	$ 13.99
Electromagnet	Electromagnet Solenoid BM-P 80-38	1	$ 44.98
Shaft Coupler	Shaft Coupler (6.35 x 8 mm)	2	$ 11.99
Limit Switch	HiLetgo KW12-3 Micro Limit Switch	10	$ 5.99
Rail Guide	MGN12H 300 mm Linear Rail Guide	2	$ 35.99
Lead Screw	TR8x2 Lead Screw (300 mm)	2	$ 11.99
Bearings	Pillow Block Bearing, 5/16 in (8 mm)	4	$ 8.49
Guide Rails	Linear Rail Slide SBR20-800 mm	2	$ 44.90
Belt	GT2 Timing Belt	1	$ 16.88
Aluminum Beams	4040 T-Slot Aluminum Profile 1000 mm	4	$ 79.99
Buzzer	Gikfun Passive Electronic Buzzer	10	$ 8.98
Flyback Diode	BOJACK 1N4007 Rectifier Diode	125	$ 5.99
Wires	18 AWG 2-Conductor Wire, 100 ft	1	$ 12.29
Heat Shrink	Eventronic Heat-Shrink Tubing 2:1	560	$ 6.99
Fuse Holder	FEBRYTOLD 0-Gauge Inline Fuse Holder	2	$ 8.49
Resistor Kit	Upvivi Resistor Assortment Kit	1	$ 6.99
T-Nuts	4040 Series T-Sliding Nuts	15	$ 9.99
Bolt and Nut Kit	M5 Screw and Nut Kit	1	$ 16.99
Relay	DC 5V One-Channel Relay Module	10	$ 11.99
Motor Mounting	NEMA 23 Steel Mounting Bracket	4	$ 11.99
Wago Connectors	WAGO 221 Connector Assortment	1	$ 30.00
Nut Block	Lysee Lead-Screw Nut Block	2	$ 20.19
M3 Screws	M3 40 mm Screw Set	1	$ 12.00

Item	Component	Qty. bought	Amount spent
M5 Screws	M5 30 mm Screw Set	1	$ 17.00
Screw	5/16 in Screw, 1 ft	1	$ 6.00
Nuts and Washers	5/16 in Nuts and Washers	1	$ 5.00
Small L-Brackets	Small L Metal Bracket	2	$ 3.00
L-Brackets	L Metal Bracket	16	$ 20.00
PLA Filament	PLA 3D Printer Filament Spool	4	$ 53.96
Aluminum Beams	4040 T-Slot Aluminum Profile 1000 mm	4	$ 80.00
	Total spent (before tax)		**$ 791.22**

4.2.3 Cost of the Final Design

Table 4.3 lists the parts used in the final design, each at the cost of the quantity used.

Table 4.3

Cost of the final design.

Item	Component	Qty.	Price total
Power Supply (24 V)	Mean Well LRS-350-24	1	$ 31.47
Microcontroller	STM32 NUCLEO-F446RE	1	$ 31.00
Stepper Motor	23HS22-2804S NEMA 23	2	$ 26.74
Stepper Controller	DM542TE Stepper Controller	2	$ 42.00
Ethernet Module	W5500 Ethernet LAN Module	1	$ 6.50
E-Stop	Red Mushroom Emergency Stop	1	$ 9.99
Power Supply (12 V)	ALITOVE Power Supply	1	$ 14.00
Buck Converter	LM2596 DC-DC Buck Converter	1	$ 4.67
Electromagnet	Electromagnet Solenoid BM-P 80-38	1	$ 44.98
Shaft Coupler	Shaft Coupler (6.35 x 8 mm)	2	$ 11.99
Limit Switch	HiLetgo KW12-3 Micro Limit Switch	2	$ 1.20
Rail Guide	MGN12H 300 mm Linear Rail Guide	2	$ 35.99
Lead Screw	TR8x2 Lead Screw (300 mm)	1	$ 11.99
Bearings	Pillow Block Bearing, 5/16 in (8 mm)	4	$ 8.49
Guide Rails	Linear Rail Slide SBR20-800 mm	2	$ 44.90
Belt	GT2 Timing Belt	1	$ 16.88

Item	Component	Qty.	Price total
Aluminum Beams	4040 T-Slot Aluminum Profile 1000 mm	4	$ 79.99
Flyback Diode	BOJACK 1N4007 Rectifier Diode	1	$ 0.05
Wires	18 AWG 2-Conductor Wire, 100 ft	1	$ 12.29
Heat Shrink	Eventronic Heat-Shrink Tubing 2:1	1	$ 6.99
Fuse Holder	FEBRYTOLD 0-Gauge Inline Fuse Holder	1	$ 4.25
Resistor Kit	Upvivi Resistor Assortment Kit	1	$ 6.99
Bolt and Nut Kit	M5 Screw and Nut Kit	1	$ 16.99
Relay	DC 5V One-Channel Relay Module	3	$ 3.60
Motor Mounting	NEMA 23 Steel Mounting Bracket	4	$ 11.99
Wago Connectors	WAGO 221 Connector Assortment	1	$ 30.00
M3 Screws	M3 40 mm Screw Set	1	$ 12.00
M5 Screws	M5 30 mm Screw Set	1	$ 17.00
Screw	5/16 in Screw, 1 ft	1	$ 6.00
Nuts and Washers	5/16 in Nuts and Washers	1	$ 5.00
Small L-Brackets	Small L Metal Bracket	2	$ 3.00
L-Brackets	L Metal Bracket	16	$ 20.00
PLA Filament	PLA 3D Printer Filament Spool	4	$ 53.96
Aluminum Beams	4040 T-Slot Aluminum Profile 1000 mm	4	$ 80.00
	Total (before tax)		**$ 712.87**

4.3 Distribution of Tasks

The task assignments shown in Fig. 4.1 identify a primary owner for each activity, but they do not prevent either team member from assisting with work outside that assignment. Mechanical and software tasks are deliberately paired during integration so that installation, programming, and validation decisions are made jointly. Table 4.4 gives the preliminary distribution, and Table 4.5 gives the current distribution, after the changes described below.

Table 4.4

Preliminary distribution of project responsibilities.

Team member	**Primary responsibilities**
Josué Bouchard	Embedded firmware; part-detection sensor and limit-switch interfacing; horizontal- and vertical-axis motion control; supervisory state machine; Modbus TCP integration; system debugging; and software-oriented verification.
Adriano Duque Mena	Mechanical frame assembly; installation of rails, transmissions, motors, sensors, and external conveyor interfaces; mechanical troubleshooting; fabrication support; and preparation of rated-load testing.
Both	Component procurement; electrical and mechanical integration; emergency-stop validation; complete-system testing; final refinement; report preparation; and presentation.

As the build has progressed, responsibilities have shifted in several areas. Table 4.5 shows the distribution as it stands now.

Table 4.5
Current distribution of project responsibilities.

Team member	**Primary responsibilities**
Josué Bouchard	Embedded firmware; interfacing of the part-ready trigger input and the limit switches; horizontal- and vertical-axis motion control; supervisory state machine; Modbus TCP integration; electrical design, including the power distribution and wiring; system debugging; software-oriented verification; project documentation; and preparation of most of the report.
Adriano Duque Mena	Mechanical design in CAD, including the vertical assembly; mechanical frame assembly; installation of rails, transmissions, motors, and sensors; fabrication of custom parts; procurement of the mechanical components; mechanical troubleshooting; preparation of rated-load testing; and preparation of the mechanical parts of the report.
Both	Procurement of the remaining components; electrical and mechanical integration; emergency-stop validation; complete-system testing; final refinement; and presentation.

The main changes are:

- The external conveyor interfaces were removed from the project scope, so the conveyor activities in Fig. 4.1 no longer apply.
- The part-detection sensor was replaced by a part-ready trigger input, on the advisor's recommendation, since the sensor's type and position depend on the conveyor, which is outside the project scope.
- Electrical design and project documentation became part of Josué's responsibilities.
- Mechanical design in CAD and the procurement of the mechanical components became part of Adriano's responsibilities.
- Report preparation, previously shared, is now split: Josué writes most of the report, and Adriano contributes the mechanical parts.

Two factors drove these changes. The first is the dependencies between tasks: the electrical design depends on the firmware, which defines the controller's pin assignments and the driver interfaces, and procurement depends on the mechanical design, which determines which parts are needed. The second is availability and timing: when a task became ready, it

was taken by the team member who could start it first, rather than waiting for the originally assigned owner.

4.4 Health and Safety

Safety is the central motivation for Safe Dip. The system is intended to remove the operator from direct interaction with a repetitive dipping task and, in the full application, from proximity to hot and viscous coating material. The prototype was evaluated under laboratory conditions without process heat, handling a workpiece of approximately 1 lb. However, the system was designed in preparation for handling 10 kg workpieces, which will present meaningful mechanical, electrical, and suspended-load hazards.

The movement emergency stop is implemented as a Category 0 stop of the movement subsystem. Actuating the normally-closed mushroom button removes power from both stepper-motor drivers immediately. The button requires a twist action to reset, and motion must remain inhibited until the button has been reset and the operator deliberately restarts the system. The electromagnet is powered from a separate supply that is not interrupted by the motion emergency stop so that an attached workpiece is not intentionally released when axis power is removed.

Table 4.6

Primary prototype hazards, controls, and residual limitations.

Hazard	**Prototype control**	**Residual limitation**
Unexpected axis motion	Normally-closed, twist-reset mushroom emergency stop removes power from both motor drivers. Firmware prevents automatic restart after a stop event.	The emergency stop does not replace safe setup procedures or physical separation from the moving gantry.
Pinch and crush points	Motion is limited to controlled laboratory tests, and personnel must remain clear of the rails, leadscrew, belt transmission, and carriage while the system is enabled.	The current prototype does not claim the protection provided by a fully guarded industrial cell.

Hazard	Prototype control	Residual limitation
Suspended workpiece	The magnet remains powered during a motion emergency stop. Retention is tested before complete-cycle operation, and no person may stand beneath the workpiece.	A conventional electromagnet does not retain the workpiece after a total loss of magnet power. This is a prototype limitation and must not be described as fail-safe load retention.
Electrical fault or short circuit	Fuses are installed in the prototype power branches, and exposed connections are insulated and secured before operation.	Fuse ratings and conductor sizes must be finalized from the actual branch currents and selected hardware.
Loss of position reference	Normally-closed limit switches provide homing and boundary references, while the external part-ready trigger must be asserted before a cycle begins.	The trigger does not make entry into the machine workspace safe while motion is enabled.
Thermal or coating exposure	No heated coating material is used during prototype acceptance testing.	The prototype does not validate operation at the intended industrial temperature or with a production coating medium.

The conventional electromagnet creates an important distinction between an emergency stop and a complete power failure. The separate supply preserves holding force when only the motion subsystem is stopped, but a failure of the magnet supply may still release the load. This limitation must remain visible in the operating instructions, test procedure, and final evaluation of the prototype.

4.5 Ethical Aspects

The ethical responsibility of the project team extends beyond demonstrating that the mechanism can complete a pick-dip-place cycle. Because Safe Dip is presented as a safety-oriented system, performance and safety claims must be based on documented tests rather than design intent alone. In particular, the report must clearly distinguish between the intended 10 kg,

elevated-temperature product concept and the laboratory prototype, which is tested without heat and with several safeguards that remain incomplete relative to an industrial installation.

The team will report failed tests, unresolved hazards, and prototype limitations honestly; will not describe the conventional electromagnet as retaining the load after complete power loss; and will not claim compliance with an industrial safety standard unless the relevant architecture and validation have actually been completed. Libraries, technical references, component data, figures, and external designs used during development will be credited appropriately. Safety decisions will take priority over cycle speed, appearance, or demonstration convenience.

Safe Dip does not require the collection of personal information. Operational data communicated through the serial or Modbus TCP interface is limited to machine state, sensor values, commands, and fault information. Any future logging system should collect only the information needed for diagnostics and process traceability.

4.6 Environmental Aspects

The laboratory prototype is not intended to use a heated bath or process chemicals. Its direct environmental impact is therefore primarily associated with electricity consumption, packaging waste, metal and plastic fabrication waste, and the eventual disposal of electronic components. Scrap aluminum, steel, wire, and electronic assemblies should be separated whenever practical so they can be reused or sent through an appropriate recycling stream rather than discarded as mixed waste.

The intended industrial application may use wax, resin, or another coating medium at elevated temperature. Because that operating condition is outside the current prototype test envelope, this report does not claim validation of spill control, fume generation, coating disposal, or thermal energy consumption. Those subjects must be evaluated using the selected coating material and production process before the system is deployed outside laboratory conditions.

4.7 Social Aspects

Safe Dip is intended to reduce worker exposure to a repetitive and potentially hazardous manual process. Removing the operator from the immediate dipping area may reduce physical strain and, in the full-temperature application, reduce the risk of burns and contact with hot coating material. A lower-cost gantry system may also make basic automation more accessible to small and medium manufacturers that cannot justify a full industrial robotic cell.

Automation can also create concern about the replacement of manual labor. The system is better understood as transferring the operator away from direct handling and toward setup, supervision, inspection, maintenance, and process control. Its value should therefore be evaluated not only through throughput, but also through changes in worker exposure, training needs, system reliability, and the quality of the resulting work environment.

4.8 Sustainability

The design supports long-term maintenance by using a modular frame and commercially available motors, drivers, sensors, power supplies, and communication hardware. A failed subsystem can therefore be repaired or replaced without discarding the complete machine. The aluminum structure, rails, and mechanical transmission can also remain in service when the controller, sensors, or end effector are upgraded.

The conventional electromagnet is a practical prototype choice, but it consumes power continuously while holding a workpiece. The control system should therefore energize it only when retention is required and should avoid unnecessary idle holding periods. This energy-use tradeoff should be considered when comparing the prototype with a future electro-permanent or mechanical holding solution.

Repeatable control of immersion depth and dwell time may reduce coating inconsistency and material waste in a future process implementation. That benefit is not demonstrated by the present non-heated prototype and should remain a design objective until it can be measured using the selected coating medium.

Chapter 5

Conclusion

Summary

This chapter concludes the Safe Dip report by reviewing the purpose and architecture of the system, identifying the expected contribution of the laboratory prototype, and outlining improvements that may be considered in future development.

5.1 Summary and Conclusion 93
5.2 Suggestions for Future Work 93

5.1 Summary and Conclusion

Safe Dip is a safety-oriented pick-dip-place system designed to reduce worker exposure to the hazards associated with manual hot-melt coating operations. The design uses a two-axis Cartesian gantry to detect, acquire, transport, dip, and release a workpiece without requiring the operator to remain near the dipping area.

The system is controlled by an STM32 microcontroller that coordinates the stepper-driven horizontal and vertical axes, the part-ready trigger input, limit switches, electromagnet, emergency-stop input, and communication interfaces. Serial communication provides the baseline operator interface, while Modbus TCP through the W5500 module allows system states, sensor information, and fault conditions to be observed by external equipment.

Safety is a central consideration in the design. A normally-closed, twist-reset emergency-stop button immediately removes power from the movement subsystem, while the conventional electromagnet remains powered from a separate supply so that actuating the movement emergency stop does not intentionally release the suspended workpiece. Fuses are also included in the prototype power branches to reduce the consequences of electrical faults.

The laboratory prototype was tested without a heated coating bath and handled a workpiece of approximately 1 lb under controlled conditions. The tests in Chapter 3 cover the control, sensing, communication, movement, and emergency-stop architecture, but they do not validate operation at the intended industrial temperature. The conventional electromagnet also cannot provide load retention during a complete loss of magnet power, which remains an important prototype limitation.

Overall, Safe Dip proposes a lower-cost and modular alternative to manual dipping and more expensive industrial robotic systems. The project provides a foundation for determining whether the design can later be developed into a guarded, temperature-rated, and industrially deployable machine.

5.2 Suggestions for Future Work

Future development should first address the safeguards required for operation near a heated coating bath. These additions may include a transparent physical enclosure, interlocked access points, independent overtemperature protection, spill containment, and visual and audible status indicators.

The main focus of future improvents would be to harden the system by providing thermal protection to the mechanism. The conventional electromagnet may also be replaced by an electro-permanent magnet, mechanically locking gripper, or another end effector

capable of retaining the workpiece during a complete power failure. Additional sensing could verify that the workpiece has been successfully acquired before movement begins.

Further improvements may include closed-loop position feedback, enhanced fault monitoring, a dedicated operator interface, and standardized communication with upstream and downstream conveyor equipment. After these safeguards are implemented, the system could be evaluated using the intended coating material and operating temperature to measure retention, positioning accuracy, dip consistency, and long-term reliability.

Also, because the electromagnet is deliberately held during an emergency stop, it depends on its 12 V supply remaining available. A planned future improvement is to add a small backup battery on the electromagnet rail, sized to hold the coil energized for a few minutes in the event of a total power loss, so that a suspended part is not dropped when mains power is lost rather than merely when the E-stop is actuated

References

[1] "What is Hot Melt Wax Coating for Tools?," Jul. 13, 2022, *Dip Seal Plastics, Inc.* Accessed: Jun. 25, 2026. [Online]. Available: https://www.dipseal.com/plastic-dipped-parts-tools-and-corner/what-is-hot-melt-wax-coating-for-tools

[2] D. Dominessy, "End Mill Hot-Dip Wax Coating." Accessed: Jun. 25, 2026. [Online]. Available: https://www.youtube.com/watch?v=gGHxNu6oPSY

[3] "What is the difference between Stop Category 0, Stop Category 1 and Stop Category 2?," Feb. 28, 2014, *Schneider Electric*. Accessed: Jul. 05, 2026. [Online]. Available: https://www.se.com/eg/en/faqs/FA225420/

[4] "Thrust generation principle of a linear lead screw motor," *MOONS' Industries*. Accessed: Jun. 25, 2026. [Online]. Available: https://www.moonsindustries.com/article/thrust-generation-principle-linear-lead-screw-motor

[5] "Safety of machinery — Electrical equipment of machines — Part 1: General requirements," International Electrotechnical Commission, IEC 60204-1:2016+A1:2021, Sep. 2021. Accessed: Jul. 20, 2026. [Online]. Available: https://webstore.iec.ch/en/publication/71256

[6] IQS Directory Editorial Team, "Lead Screws: What Is It?, How It Works, Types, and Advantages," Jun. 16, 2026. Accessed: Jun. 26, 2026. [Online]. Available: https://www.iqsdirectory.com/articles/ball-screw/lead-screws.html

[7] RipInPease, "madbus-cli: A modbus tui using the madbus crate," Mar. 18, 2026. Accessed: Jul. 04, 2026. [Online]. Available: https://lib.rs/crates/madbus-cli

[8] "EFFCO DS-250 Dip-Spin Coating Machine." Accessed: Jul. 20, 2026. [Online]. Available: https://www.aramfix.com/product/effco-ds-250-dip-spin-coating-machine-250mm-basketxlxwxh-1500x1500x1890mm-steel-alloy-blue-1-pce

[9] "MS800 Automatic Tiltable Dip-Spin Coating Machine." Accessed: Jul. 20, 2026. [Online]. Available: https://www.shishengst.com/shisheng-group-sale-ms800-automatic-tiltable-dip-spin-coating-machine.html

[10] "TH-2004L3 Thread Coating Machine." Accessed: Jul. 20, 2026. [Online]. Available: https://www.tianhaodispensing.com/products/coating-machine/thread-coating-machin-TH-2004L3-D.html

[11] "Automatic Thread Coating Machine." Accessed: Jul. 20, 2026. [Online]. Available: https://www.tianhaodispensing.com/products/coating-machine/thread-coating-machine-automatic.html

[12] “Custom Automation and Specialty Equipment.” Accessed: Jul. 20, 2026. [Online]. Available: https://southshorecontrols.com/specialty-equipment/

[13] “UR5e Collaborative Industrial Robot.” Accessed: Jul. 20, 2026. [Online]. Available: https://www.universal-robots.com/manuals/EN/HTML/SW10_12/Content/Landingpages/WebPolyX/LandingarmUR5e.htm

Appendix A - Programming Code

This appendix shows the pieces of the firmware that carry the main ideas of Chapter 3. They are excerpts, not the whole program, and some comments were shortened. The firmware is written in C++ for the STM32 controller. Each listing is introduced by a short explanation and points to the section of Chapter 3 that describes the same idea in words.

A.1 Step pulse generation

The STEP signal is produced by a hardware timer, not by the main loop. Listing A.1 shows the interrupt that runs on every compare event of the timer (Section 3.1.6). Each event toggles the STEP pin. On the active edge the interrupt counts one step, in the direction of travel, which gives the controller the axis position. On the inactive edge it loads any new speed requested by the motion controller, so that a speed change never shortens a pulse. A requested speed of zero stops the rescheduling. Finally, the next toggle is scheduled half a period later, where the half period in timer ticks is the value of Equation (2.13).

```cpp
void StepperDriver::on_compare_elapsed() noexcept
{
    is_step_output_active_ = !is_step_output_active_;

    if (is_step_output_active_)
    {
        // Count one step, in the direction of travel
        pulse_count_.fetch_add(static_cast<int8_t>(direction_current_),
                               std::memory_order_relaxed);
    }
    else
    {
        // Load the speed the motion controller asked for
        const auto speed_requested =
            speed_requested_.load(std::memory_order_relaxed);

        if (speed_requested == 0)
        {
            return; // Stop: the main loop cancels the interrupt
        }

        apply_speed(speed_requested);
    }

    // Schedule the next toggle half a period later
    const std::uint32_t previous_compare = config_.timer_channel.compare();
    config_.timer_channel.set_compare(previous_compare + half_period_ticks_);
}
```

Listing A.1: The timer interrupt that generates the STEP pulses

A.2 Braking and speed ramp

The motion controller shapes the speed of each axis into a trapezoid (Section 3.1.5). Listing A.2 shows its two core calculations. The first function computes the distance needed to slow from the current speed to the landing speed, which is Equation (2.14), and reports whether the remaining distance has fallen within it. The second function moves the current speed toward the desired speed by at most the acceleration times the elapsed time, and snaps to the desired speed when it is closer than one step of that size, so that the speed does not oscillate around it.

```cpp
bool StepperController::should_decelerate(
    const PositionMode &position) const noexcept
{
    const std::int64_t distance = distance_to_target(position);

    const float speed_magnitude = std::abs(speed_current_signed_);
    const float landing_speed = speed_profile_.landing_speed;

    // Distance to slow down to the landing speed, not to a standstill
    const float braking_distance =
        std::max(0.0F, (speed_magnitude * speed_magnitude) -
                           (landing_speed * landing_speed)) /
        (2.0F * speed_profile_.acceleration);

    return std::abs(static_cast<float>(distance)) <= braking_distance;
}

float StepperController::ramp_speed_toward(float speed_desired,
                                           float delta_time_s) const noexcept
{
    const float speed_error = speed_desired - speed_current_signed_;
    const float speed_step = speed_profile_.acceleration * delta_time_s;

    // Snap to the setpoint instead of oscillating around it
    if (std::abs(speed_error) <= speed_step)
    {
        return speed_desired;
    }

    return speed_current_signed_ + std::copysign(speed_step, speed_error);
}
```

Listing A.2: Braking distance and speed ramp of the motion controller

A.3 Millimeters to steps

All positions, speeds, and accelerations are set in millimeters and converted to steps by the scale factor of the axis (Table 2.1). Listing A.3 shows the conversion. The position is multiplied by the pulses per revolution and divided by the travel per revolution, as in Equation (2.5), and

the result is rounded to the nearest step. The multiplication is done in 64 bits so that a long travel at a high microstep setting cannot overflow. The second function returns the number of steps per millimeter, Equation (2.4), which converts speeds and accelerations.

```cpp
// Rounded to the nearest step
constexpr std::int32_t steps_from_mm(std::int32_t mm) const noexcept
{
    // 64-bit, so a long travel at a high microstep setting can't overflow
    return static_cast<std::int32_t>(
        divide_rounded(std::int64_t{mm} * pulses_per_rev_, mm_per_rev_));
}

// For speeds and accelerations: multiply mm/s or mm/s^2 by this
constexpr float steps_per_mm() const noexcept
{
    return static_cast<float>(pulses_per_rev_) / static_cast<float>(mm_per_rev_);
}

// Half away from zero. `denominator` is positive
static constexpr std::int64_t divide_rounded(std::int64_t numerator,
                                             std::int64_t denominator) noexcept
{
    const std::int64_t half = denominator / 2;
    return numerator >= 0 ? (numerator + half) / denominator
                          : -((-numerator + half) / denominator);
}
```

Listing A.3: Conversion between millimeters and steps

A.4 Limit-switch debounce

The limit switches were very noisy when first connected (Section 3.3.2). Listing A.4 shows the filter that was added. A press counts only after the input has read pressed for the full debounce time without a break, so a short spike from the stepper drivers cannot fault a move. A release counts at once, so that the axis is not kept stopped longer than necessary. The price of the delay is that the axis runs on past the switch by the debounce time multiplied by its speed.

```cpp
class LimitFilter
{
  public:
    explicit constexpr LimitFilter(std::uint32_t press_ms) noexcept
        : press_ms_(press_ms) {}

    bool update(bool pressed, std::uint32_t now_ms) noexcept
    {
        if (pressed != last_pressed_)
        {
            last_pressed_ = pressed;
            changed_at_ms_ = now_ms; // Restart the timer on every change
        }

        // A press counts only after press_ms without a break
        return pressed && now_ms - changed_at_ms_ >= press_ms_;
    }

  private:
    std::uint32_t press_ms_;
    bool last_pressed_ = false;
    std::uint32_t changed_at_ms_ = 0;
};
```

Listing A.4: Debounce filter for the limit switches

A.5 Fault latch and safe state

Every abnormal condition is recorded as a latched fault (Section 3.1.7). Listing A.5 shows the latch and the way the main loop reacts to it. Each fault is one bit of a 16-bit word, so any number of faults can be active at once, and the first fault to be raised is remembered separately, because it is the cause. Faults are only cleared on a Start press. In the main loop, if any fault is latched, the machine enters its safe state: the sequencer and both axes stop and the electromagnet is kept engaged. Otherwise, the motion controllers run.

```cpp
void Faults::raise(Fault fault) noexcept
{
    const auto bit =
        static_cast<std::uint16_t>(1U << static_cast<std::size_t>(fault));

    if (active_ == 0)
    {
        first_ = bit; // The first fault is the cause
    }

    active_ |= bit;
}

static void enter_safe_state() noexcept
{
    machine.sequencer.stop();
    machine.controller_x.stop_immediately();
    machine.controller_y.stop_immediately();
    hardware::effector.engage(); // Hold the part
}

// In loop(), after the sequencer step:
if (machine.faults.any())
{
    enter_safe_state();
}
else
{
    machine.controller_x.process(now);
    machine.controller_y.process(now);
}
```

Listing A.5: The fault latch and the safe state

A.6 Operation interface

Every step of the cycle is an operation, and all operations implement the same interface (Section 3.1.2). Listing A.6 shows it. An operation has two functions: `start()` begins it and returns at once, and `poll()` is called on every later pass of the main loop and reports whether

the operation is still running, has finished, or was aborted. The comments summarize the rules that every operation follows: it never blocks, it takes the time from the context passed to it and never reads a clock, it reads recipe values only when it starts, and it raises the fault that explains a failure before it reports `aborted`. Because the sequencer only knows this interface, a new kind of step can be added without changing it.

```cpp
enum class PollResult
{
    running,
    finished,
    aborted, // The sequence must not continue past it
};

// One step of the cycle. `Context` is what the sequencer passes on every pass.
// An implementation must:
// - read recipe values only in start(), keeping a copy if poll() needs them
// - reset all per-run state in start(): the same object runs again and again
// - never block in start() or poll()
// - take the time from the context, and never read a clock
// - raise the fault that explains a failure before returning aborted
template <class Context>
class Operation
{
  public:
    virtual void start(Context &ctx) noexcept = 0;
    virtual PollResult poll(Context &ctx) noexcept = 0;

    virtual ~Operation() = default;
};
```

Listing A.6: The interface that every operation of the cycle implements

A.7 Sequencer step

The cycle is a list of nonblocking operations, and the sequencer runs one step of it on every pass of the main loop (Fig. 3.3). Listing A.7 shows that step. An operation that has not started is started, and one that has started is polled. A finished operation advances the sequence to the next one. An aborted operation stops the sequencer but leaves its index on that operation, so the failed step can be read over Modbus.

```cpp
void Sequencer<Context>::step(Context &ctx) noexcept
{
    if (!is_sequencer_running_)
    {
        return;
    }

    if (!is_current_operation_started_)
    {
        is_current_operation_started_ = true;
        current_operation().start(ctx);
        return;
    }

    switch (current_operation().poll(ctx))
    {
    case PollResult::running:
        break; // Nothing to do: the axes move on their own

    case PollResult::finished:
        is_current_operation_started_ = false;
        advance_operation();
        break;

    case PollResult::aborted:
        stop(); // The index stays on this operation
        break;
    }
}
```

Listing A.7: One step of the cycle sequencer

A.8 Homing

Homing drives an axis toward its limit switch, sets the origin just past the switch, and backs off to it (Fig. 3.5). Listing A.8 shows the `poll()` function of the homing operation. While searching, a pressed switch sets the origin a fixed back-off behind the switch and starts the move to it. After the back-off, a switch that still reads pressed cannot be trusted, so

the operation raises a homing-failed fault. A time limit catches an axis that never reaches its switch.

```cpp
PollResult HomeStepperOperation::poll(Tick &ctx) noexcept
{
    StepperController &controller = axis_.controller;
    const bool is_switch_pressed = ctx.inputs.*axis_.limit_pressed;

    if (!is_backing_off_)
    {
        if (is_switch_pressed)
        {
            // The switch is back_off_steps_ behind the new origin
            controller.reset_position(-back_off_steps_);
            controller.move_to_steps(0);
            is_backing_off_ = true;
        }
    }
    else if (controller.is_at_target())
    {
        if (is_switch_pressed)
        {
            // Still pressed after backing off: the origin can't be trusted
            ctx.faults.raise(axis_.homing_failed);
            return PollResult::aborted;
        }

        return PollResult::finished;
    }

    if (ctx.now_ms - start_time_ >= timeout_ms_)
    {
        controller.stop_immediately();
        ctx.faults.raise(Fault::operation_timed_out);
        return PollResult::aborted;
    }

    return PollResult::running;
}
```

Listing A.8: One poll of the homing operation

A.9 Modbus monitor

The Modbus monitor is a small support tool that shows the state of the machine on a laptop during tests (Section 3.2.9). It is not part of the machine's control. Its code was generated with the assistance of Claude Code, an AI coding assistant, and its Modbus decoding is covered by unit tests. A web browser cannot open a raw TCP connection, so the tool is a small program in Go that serves the monitor's web page and passes the page's requests on to the board over Modbus TCP. The page shows the axis positions, the stage of the cycle, the latched faults, and a graph of the main-loop time. The three listings below are excerpts from the Go program.

Listing A.9 shows how one request is made and checked. The request is the Modbus function with an MBAP header in front of it, which holds a transaction number, the protocol identifier, the length, and the unit identifier. The reply must carry the same transaction number, and a reply with the error bit set in its function code is turned into an error. The board serves a single socket, so every request goes through one connection, under a lock.

```go
func (c *Client) roundTrip(target string, pdu []byte) ([]byte, error) {
  // MBAP header: transaction id, protocol id (0), length, unit id
  c.tid++
  frame := make([]byte, 7, 7+len(pdu))
  binary.BigEndian.PutUint16(frame[0:], c.tid)
  binary.BigEndian.PutUint16(frame[4:], uint16(len(pdu)+1))
  frame[6] = 1
  if _, err := c.conn.Write(append(frame, pdu...)); err != nil {
    return nil, err
  }
  header := make([]byte, 7)
  if _, err := io.ReadFull(c.conn, header); err != nil {
    return nil, err
  }
  tid := binary.BigEndian.Uint16(header[0:])
  length := binary.BigEndian.Uint16(header[4:])
  if tid != c.tid || length < 2 {
    return nil, fmt.Errorf("bad MBAP header (tid %d, expected %d)", tid, c.tid)
  }
  body := make([]byte, length-1)
  if _, err := io.ReadFull(c.conn, body); err != nil {
    return nil, err
  }
  return checkResponse(pdu[0], body)
}

// An exception reply has the function code with its top bit set
func checkResponse(fc byte, body []byte) ([]byte, error) {
  switch {
  case body[0] == fc|0x80 && len(body) >= 2:
    return nil, &ExceptionError{Function: fc, Code: body[1]}
  case body[0] != fc:
    return nil, fmt.Errorf("function code mismatch: got %d, sent %d", body[0], fc)
  }
  return body[1:], nil
}
```

Listing A.9: Sending a Modbus TCP request and checking the reply in the monitor

Listing A.10 shows how the data in a reply is read. A register is two bytes, high byte first, and the first byte of the reply holds the number of data bytes, which is checked against the number requested. The bits of the discrete inputs are packed eight to a byte, with the first bit in the lowest position. A position does not fit in one 16-bit register, so the firmware sends it as two registers, high word first, and the last function joins them into a signed 32-bit number. These functions are the ones covered by the unit tests.

```go
func decodeBits(resp []byte, count uint16) ([]bool, error) {
  if len(resp) < 1 || int(resp[0]) != (int(count)+7)/8 || len(resp) != 1+int(resp[0]) {
    return nil, errors.New("malformed bit response")
  }
  bits := make([]bool, count)
  for i := range bits {
    bits[i] = resp[1+i/8]>>(i%8)&1 == 1
  }
  return bits, nil
}

func decodeRegisters(resp []byte, count uint16) ([]uint16, error) {
  if len(resp) < 1 || int(resp[0]) != 2*int(count) || len(resp) != 1+int(resp[0]) {
    return nil, errors.New("malformed register response")
  }
  regs := make([]uint16, count)
  for i := range regs {
    regs[i] = binary.BigEndian.Uint16(resp[1+2*i:])
  }
  return regs, nil
}

// Joins a high-word-first register pair into a signed 32-bit value
func signed32(high, low uint16) int32 {
  return int32(uint32(high)<<16 | uint32(low))
}
```

Listing A.10: Decoding the bits and registers of a Modbus reply in the monitor

Finally, Listing A.11 shows how one poll of the board is turned into the machine state. Three requests read the discrete inputs, the input registers, and the holding registers. The input registers are read in one request, so that both words of a position, and the fault registers, all

come from the same moment. The raw tables are then given their meaning, following the Modbus map of the firmware.

```go
func poll(c *Client, target string) (*State, error) {
  di, err := c.ReadBits(target, fcReadDiscreteInputs, diCount)
  if err != nil {
    return nil, err
  }
  // One request, so both position words and the fault registers each come
  // from the same moment
  ir, err := c.ReadRegisters(target, fcReadInputRegisters, irCount)
  if err != nil {
    return nil, err
  }
  hr, err := c.ReadRegisters(target, fcReadHoldingRegisters, hrCount)
  if err != nil {
    return nil, err
  }
  return newState(di, ir, hr), nil
}

// Maps the raw tables to their meaning (some fields are left out here)
func newState(di []bool, ir, hr []uint16) *State {
  return &State{
    EStop:   di[0],
    Running: di[2],
    Fault:   di[6],
    OpIndex: ir[0],
    X:       signed32(ir[1], ir[2]),
    Y:       signed32(ir[3], ir[4]),
    Params:  hr,

    FaultsActive: ir[irFaultsActive],
    FaultFirst:   ir[irFaultFirst],
    LoopTimeUs:   ir[irLoopTime],
  }
}
```

Listing A.11: Reading the state of the machine in the monitor

Appendix B - Engineering Codes and Standards

The codes and standards in Table B.1 were identified as relevant references for the design and development of Safe Dip. Their inclusion does not represent a claim of certification or complete compliance. Formal compliance would require evaluation of the finished system against the complete requirements of each applicable document.

Table B.1
Engineering codes and standards applicable to the Safe Dip project.

Code or Standard	**Organization**	**Explanation**	**Relevance to Safe Dip**
IEEE Code of Ethics	Institute of Electrical and Electronics Engineers	Defines ethical duties related to safety, honesty, competence, and public welfare.	Supports safe design, truthful reporting, and clear disclosure of prototype limitations.
NSPE Code of Ethics for Engineers	National Society of Professional Engineers	Requires engineers to prioritize public safety and communicate technical information honestly.	Applies to safety decisions, testing, risk disclosure, and performance claims.
ISO 13850:2015	International Organization for Standardization	Defines requirements and design principles for machinery emergency-stop functions.	Guides the latching E-stop, immediate motion cutoff, manual reset, and restart inhibition.
ISO 14118:2017	International Organization for Standardization	Addresses prevention of unexpected machine start-up.	Requires a deliberate restart action after the E-stop is reset.

Code or Standard	Organization	Explanation	Relevance to Safe Dip
NFPA 79	National Fire Protection Association	Covers electrical equipment, wiring, protection, and controls for industrial machinery.	Guides internal wiring, branch protection, grounding, control circuits, and E-stop wiring.
NFPA 70 National Electrical Code	National Fire Protection Association	Defines requirements for safe electrical installations and supply connections.	Guides power connections, wiring, fuses, grounding, and electrical fault protection.
IEC 60204-1	International Electrotechnical Commission	Covers the safety of electrical equipment of machines and defines the emergency-stop categories.	Defines the Category 0 stop architecture of the movement subsystem.

Appendix C - Datasheets

[14] "SRD-05VDC-SL-C Power Relay," Ningbo Songle Relay Co., Ltd., Datasheet. Accessed: Oct. 25, 2026. [Online]. Available: https://www.lcsc.com/datasheet/lcsc_datasheet_2304140030_Ningbo-Songle-Relay-SRD-05VDC-SL-C_C35449.pdf

[15] "STM32F446xC/E: Arm Cortex-M4 32-bit MCU+FPU, up to 512 KB Flash, 128 KB SRAM, USB OTG, timers, ADCs, and communication interfaces," STMicroelectronics, DS10693. Accessed: Jul. 27, 2026. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f446re.pdf

[16] "STM32 Nucleo-64 Boards (MB1136)," STMicroelectronics, UM1724, Sep. 2025. Accessed: Jul. 27, 2026. [Online]. Available: https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

[17] "W5500 Datasheet," WIZnet Co., Ltd. Accessed: Jul. 27, 2026. [Online]. Available: https://docs.wiznet.io/img/products/w5500/W5500_ds_v110e.pdf

[18] "350W Single Output Switching Power Supply LRS-350 Series," MEAN WELL Enterprises Co., Ltd., Datasheet LRS-350-SPEC, Aug. 2021. Accessed: Oct. 11, 2026. [Online]. Available: https://files.omc-stepperonline.com/LRS-350.pdf

[19] "23HS22-2804S NEMA 23 Bipolar Stepper Motor Full Datasheet," StepperOnline. Accessed: Jul. 27, 2026. [Online]. Available: https://www.omc-stepperonline.com/index.php?file=128%2F23HS22-2804S_Full_Datasheet.pdf&route=product%2Fproduct%2Fget_file

[20] "23HS22-2804S Torque Curve," StepperOnline. Accessed: Jul. 27, 2026. [Online]. Available: https://www.omc-stepperonline.com/index.php?file=128%2F23HS22-2804S_Torque_Curve.pdf&route=product%2Fproduct%2Fget_file

[21] "TE Series Two-Phase Digital Stepper Drive User Manual," StepperOnline, DM542TE / DM556TE / DM860TE. Accessed: Jul. 27, 2026. [Online]. Available: https://www.omc-stepperonline.com/download/TE%20_series_user_manual.pdf

[22] "LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator," Texas Instruments, Datasheet SNVS124G, Mar. 2023. Accessed: Nov. 02, 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf

[23] "BM-P 80-38 Electromagnet Solenoid, 1000 N Holding Force." Accessed: Jul. 27, 2026. [Online]. Available: https://baomain.com/products/bm-p-80-38

[24] "1N4001 through 1N4007 General-Purpose Plastic Rectifiers," Vishay Intertechnology, Inc., Datasheet. Accessed: Nov. 02, 2026. [Online]. Available: https://www.vishay.com/en/product/88503/

[25] “MODBUS Application Protocol Specification V1.1b3,” Modbus Organization, Inc. Accessed: Jul. 27, 2026. [Online]. Available: https://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf

[26] “MODBUS Messaging on TCP/IP Implementation Guide V1.0b,” Modbus Organization, Inc. Accessed: Jul. 27, 2026. [Online]. Available: https://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf

Appendix D - Electrical Schematic

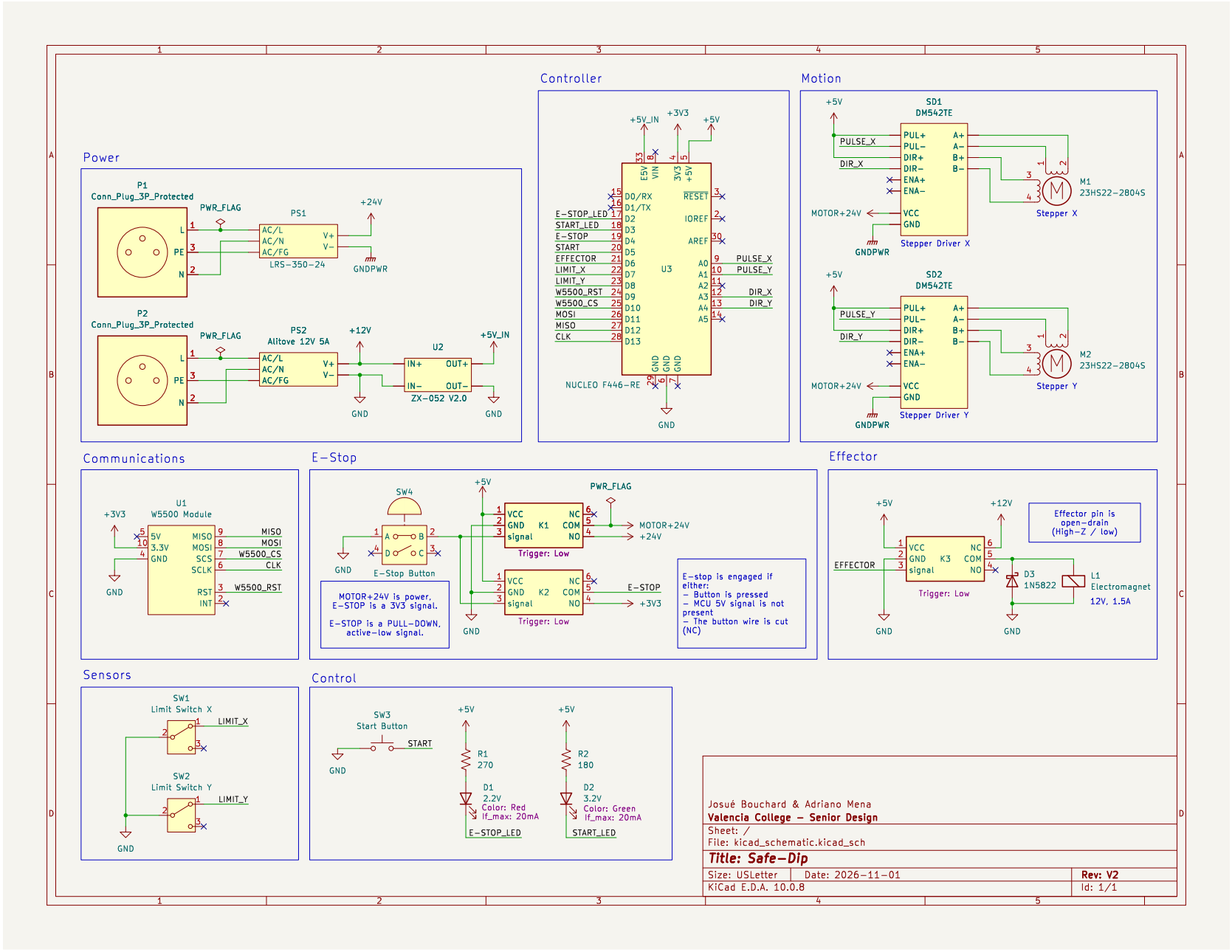


Fig. D.1: Complete electrical schematic of the Safe Dip prototype

Appendix E - Team Biographies

Josué Bouchard is originally from Argentina and is completing his Bachelor's degree in Electrical and Computer Engineering Technology at Valencia College. His interest in engineering began with an Arduino board and grew from a fascination with using software to interact with the physical world. After beginning his studies in Computer Engineering at the Universidad de Buenos Aires, he transferred to Valencia College, where he combined his theoretical background with hands-on experience in electronics, microcontrollers, and embedded systems. Josué also enjoys helping others understand technical concepts through his work as a laboratory aide. After graduation, he hopes to pursue a career in embedded systems and eventually work in aerospace or robotics.

Adriano Mena Duque de Estrada was born in Colón, Matanzas, Cuba, and immigrated to the United States with his family in 2016, crossing the border with very little and no knowledge of English. With faith and perseverance, he is now completing his Bachelor's degree in Electrical and Computer Engineering Technology at Valencia College, having switched from Aerospace Engineering after realizing it wasn't the right fit — though designing spacecraft remains a personal dream. Outside of coursework, Adriano enjoys strategy video games, spends time with his dog and bearded dragon, and loves building electrical projects, which is what ultimately drew him into this career path.